

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines
  - Homework 3 is due next Monday 2/7
  - Lab 1 is due on 2/4
  - Project 1 due on 2/18

Previous lecture …

- How processes (threads) are created and terminated
- Tracing programs with fork() calls

Muddiest Points (fork)

- can you clarify copy-and-write? does the memory automatically get “passed” down or not?
- I was confused how fork can be assigned to variable and whether its the child or parent that gets assigned
- Fork tracing example
- fork tracing!!$
- How can a child process run a different program than their parent?
- what is exec()
- What data should you track when fork tracing

Muddiest Points (wait())

- Why can a child process become a zombie for a parent process, but not the other way around?
- Does most of the parent processes invoke wait( )? Is this the system software engineers' job to ensure every process will invoke wait( ) defaultly to avoid creating zombie processes, or is OS's job and it can arbitrarily invoke wait( ) or not?
- does a process actually wait when wait() is called? like stops running
- Zombie vs Orphan processes
- What are the differences of a zombie and orphan process? The parent doesnt invoke wait in either instance?
- Implementation of how init adopts orphaned processes.
- Can a process be orphan and zombie at the same time?
- Orphan versus zombie processes. The difference between when the parent process calls exit()
- In what situation would a child not be a copy of its parent?

Muddiest Points (Threads)

- Similarities and differences between forks and threads
- The difference between processes and threads in how they're handled by the operating system. Are threads and processes treated the same by the OS beyond the threads sharing memory? Do threads go through the same states and transitions as processes?

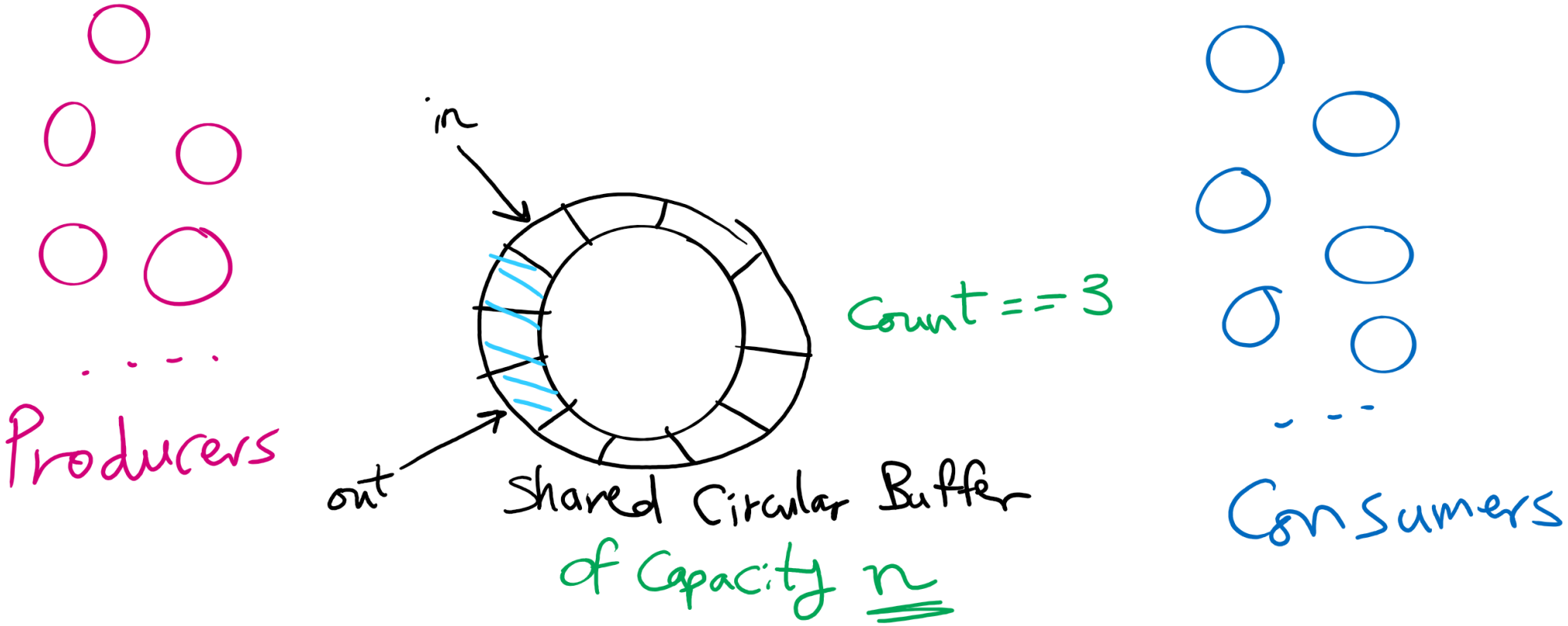
Muddiest Points (Process Termination)

- what does killing a process do, or what is the action the kill program takes to kill another?
- exit vs kill vs abort
- The differences between the exit(), Abort() and Kill(). Specifically, the difference between kill() and abort().

Problem of the Day

- Bounded Buffer Problem
  - aka Producers Consumers Problem
- A shared circular-array buffer with capacity *n*
- A set of *producer* processes/threads
- As set of *consumer* processes/threads
- Requirements:
  - Never exceed the buffer capacity
  - Producers wait if the buffer is full
  - Consumers wait if the buffer is empty

Produces Consumers Problem

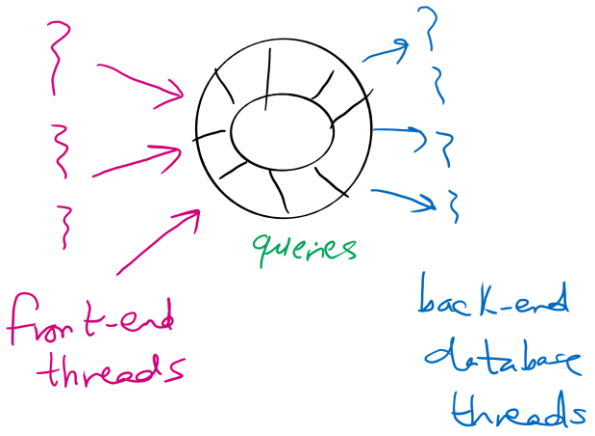


Producers Consumers Problem is everywhere!

- Access to User Interface elements in mobile applications
  - Main UI thread
  - Background threads
    - e.g., download files, check email on mail server
  - Background threads send requests to UI thread to update UI elements
  - Requests are stored in a shared bounded buffer
  - Which threads are the producers?
  - Who threads are the consumers?

Producers Consumers Problem is everywhere!

- Web Server
  - front-end processes/threads that interact with the HTTP connection from the client (e.g., browser)
  - Back-end processes/threads that execute database queries
  - Queries are inserted by front-end processes into a shared buffer
  - Which threads are the producers?
  - Who threads are the consumers?



Solving Producers Consumers using Semaphores

Semaphore empty(n), full(0)
Mutex sem(1);

Producer

```
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
Count++
up(sem)
up(full)
```

Consumer

```
down(full)
down(sem)
item = buffer[out]
out += 1 % n
Count--
up(sem)
up(empty)
```

How to trace the solution?

Let’s define some events.

Producer arrives

Moves as far as possible until the solid line

Producer

down(empty)

down(sem)

---

buffer[in] = new item

in += 1 % n

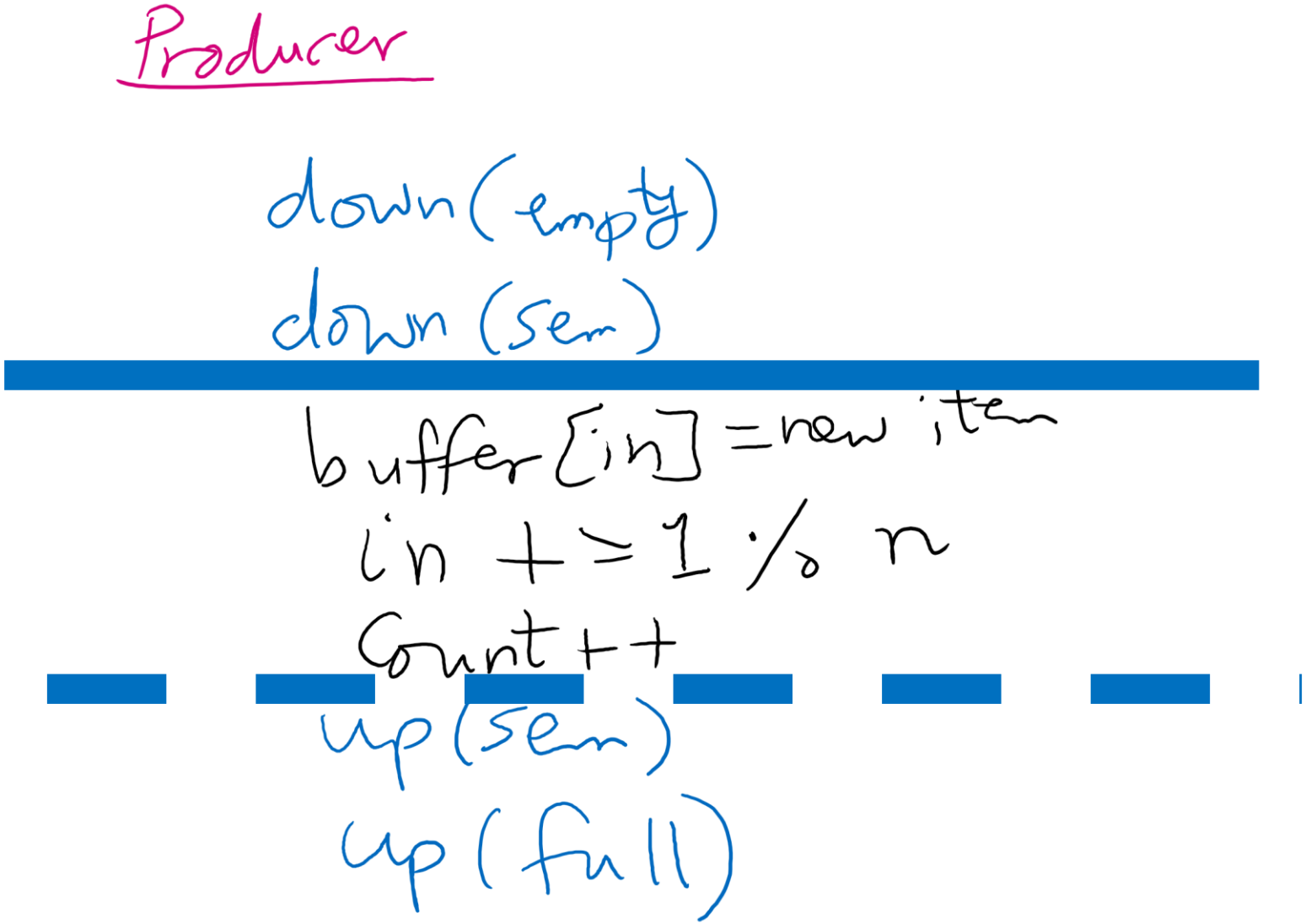
count++

up(sem)

up(full)

Producer enters

Moves as far as possible past the solid line and until the dashed line



Producer leaves

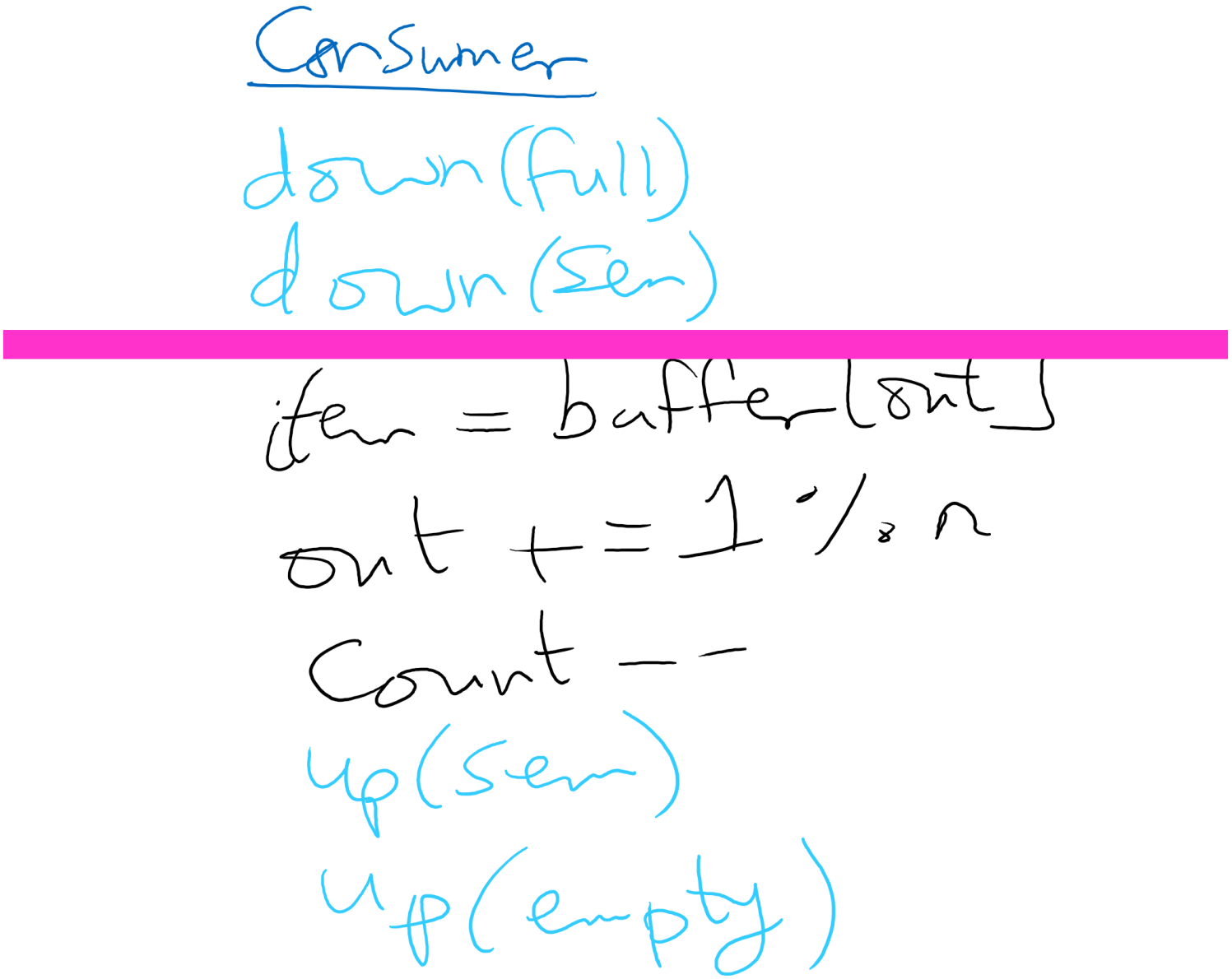
Moves as far as possible until the dotted line

Producer

down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)

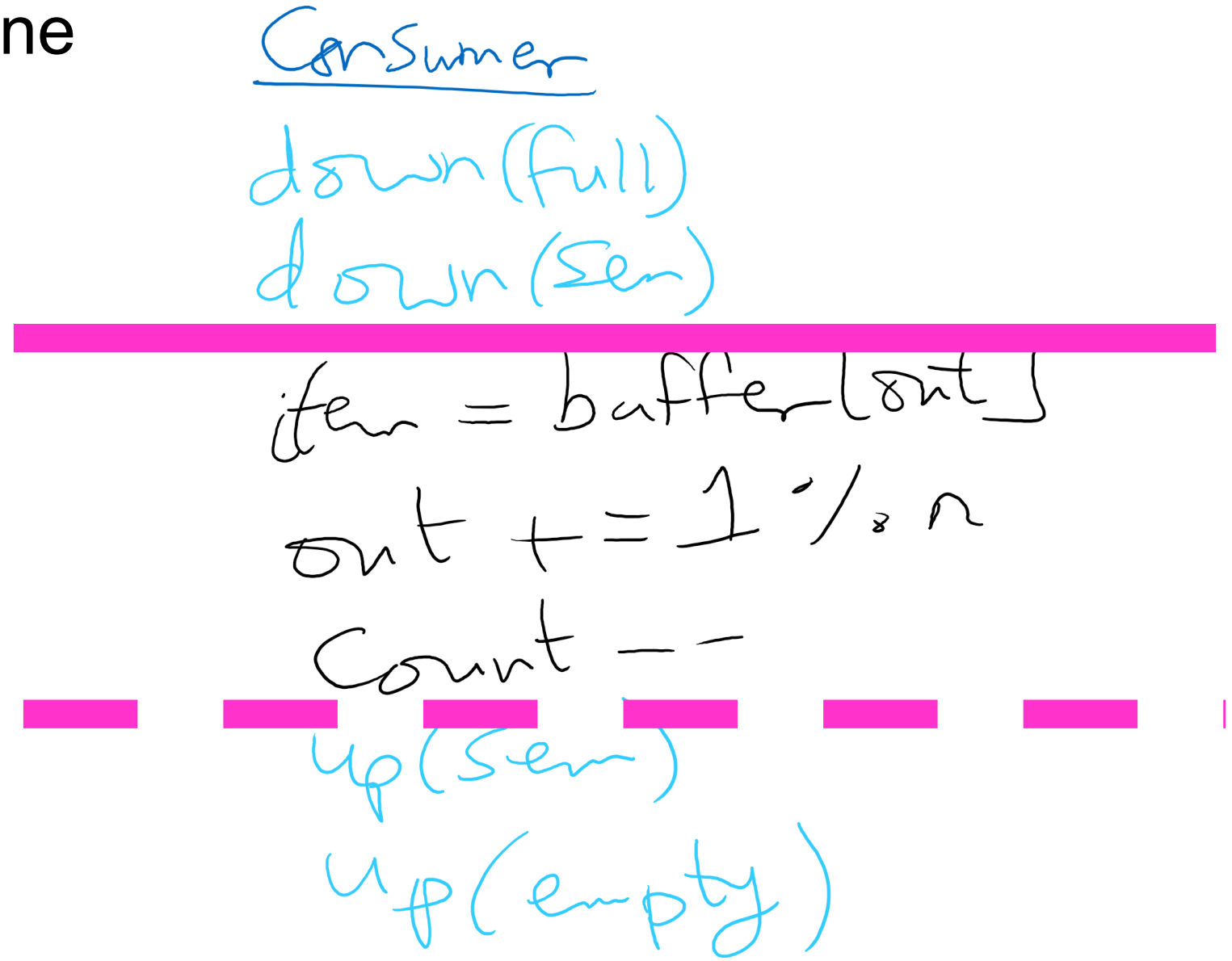
Consumer arrives

Moves as far as possible until the solid line



Consumer enters

Moves as far as possible past the solid line and until the dashed line



Consumer leaves

Moves as far as possible past the dashed line and until the dotted line

Consumer

down(full)

down(sem)

item = buffer[out]

out += 1 % n

count--

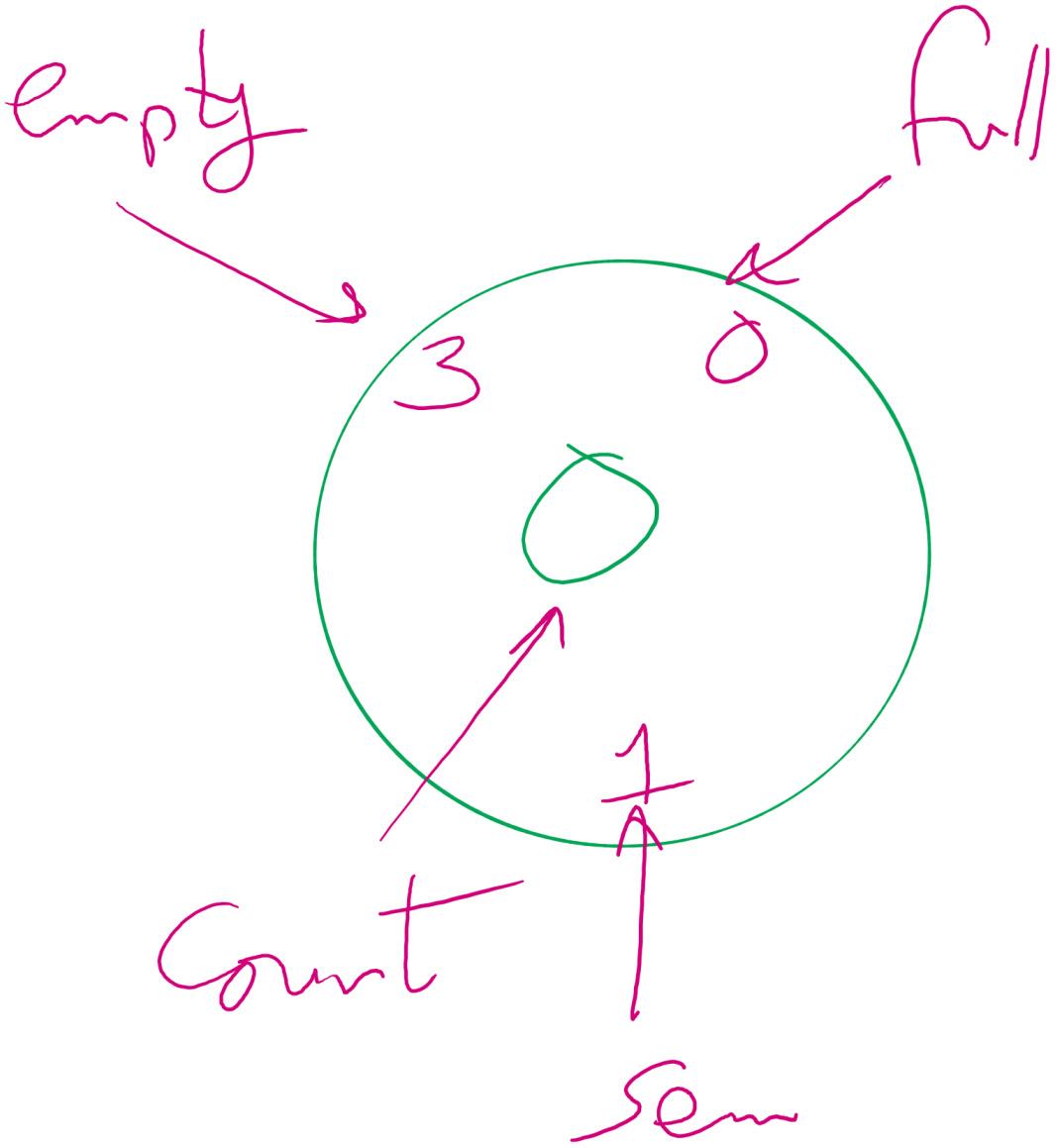
up(sem)

up(empty)

Tracing

Given a sequence of events, is the sequence *feasible*?
If yes, what is the *system state* at the end of the sequence?

System State

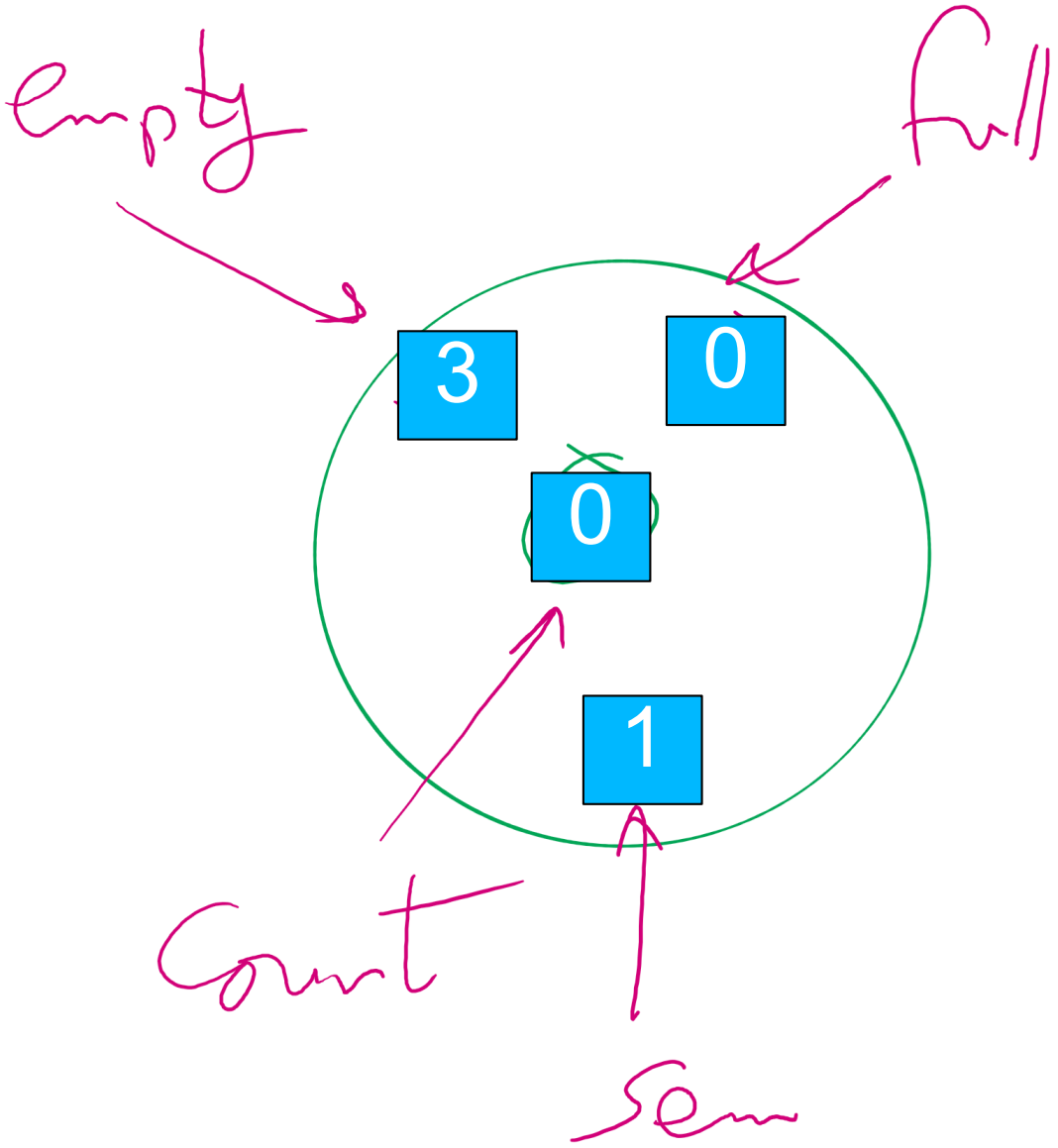


Example 1

n == 3

Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

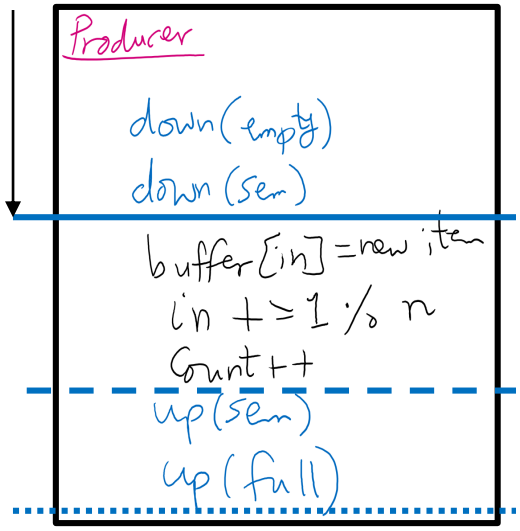
Initial state

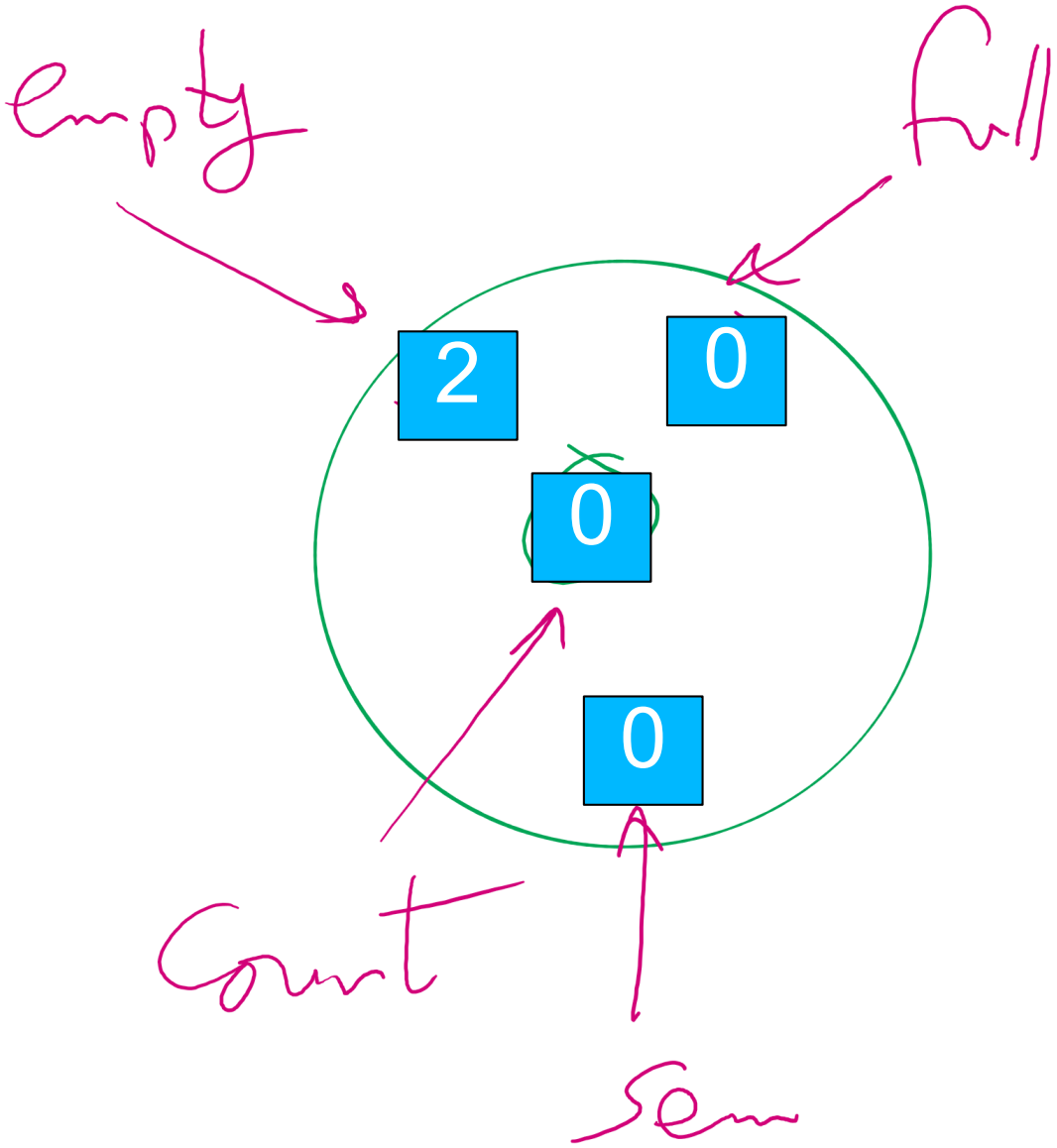


n == 3

Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

P0 arrives

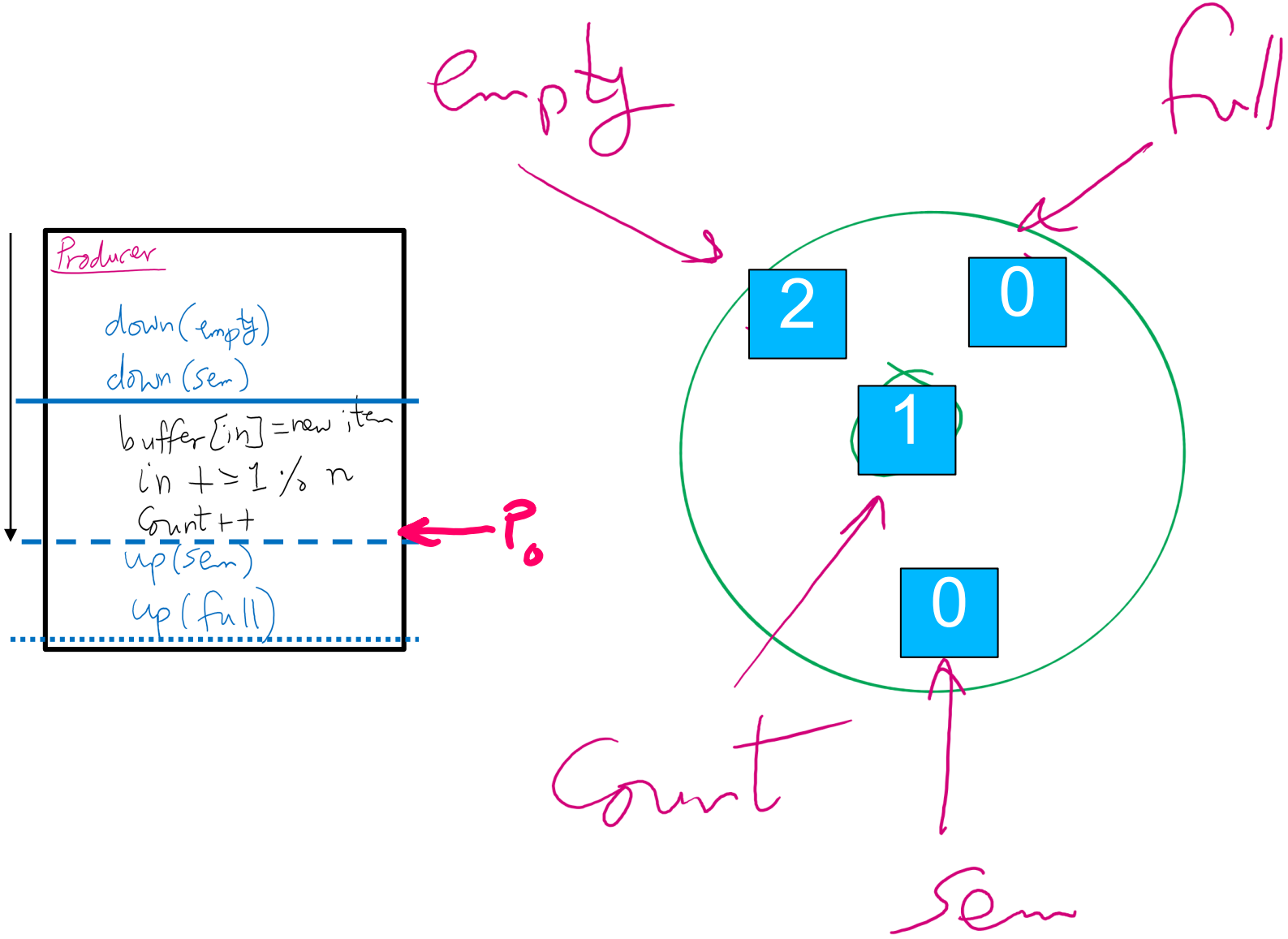




n == 3

Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

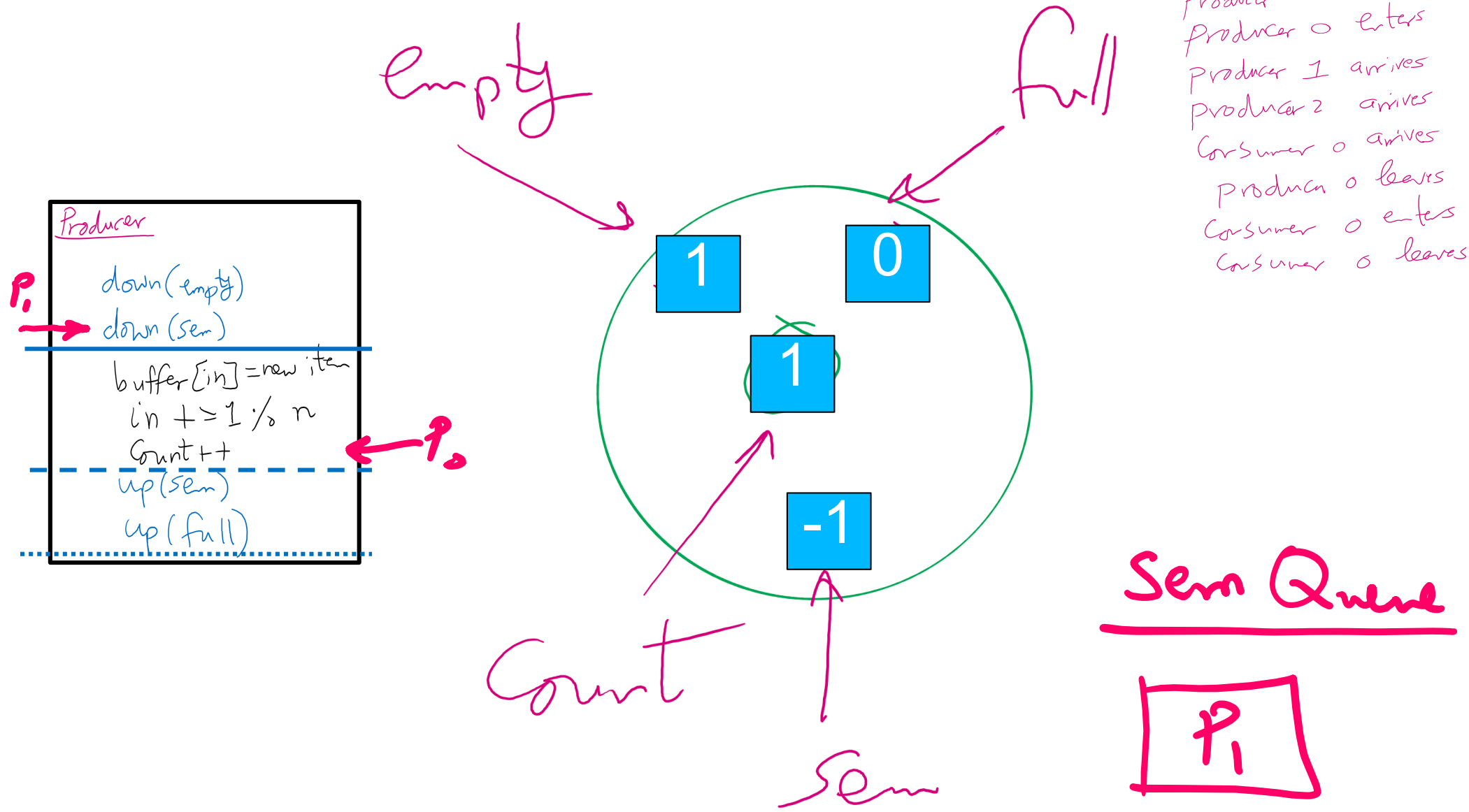
P0 enters



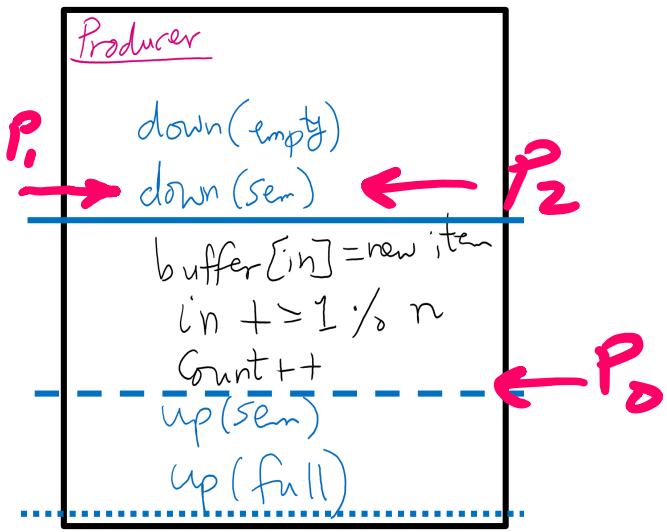
n == 3

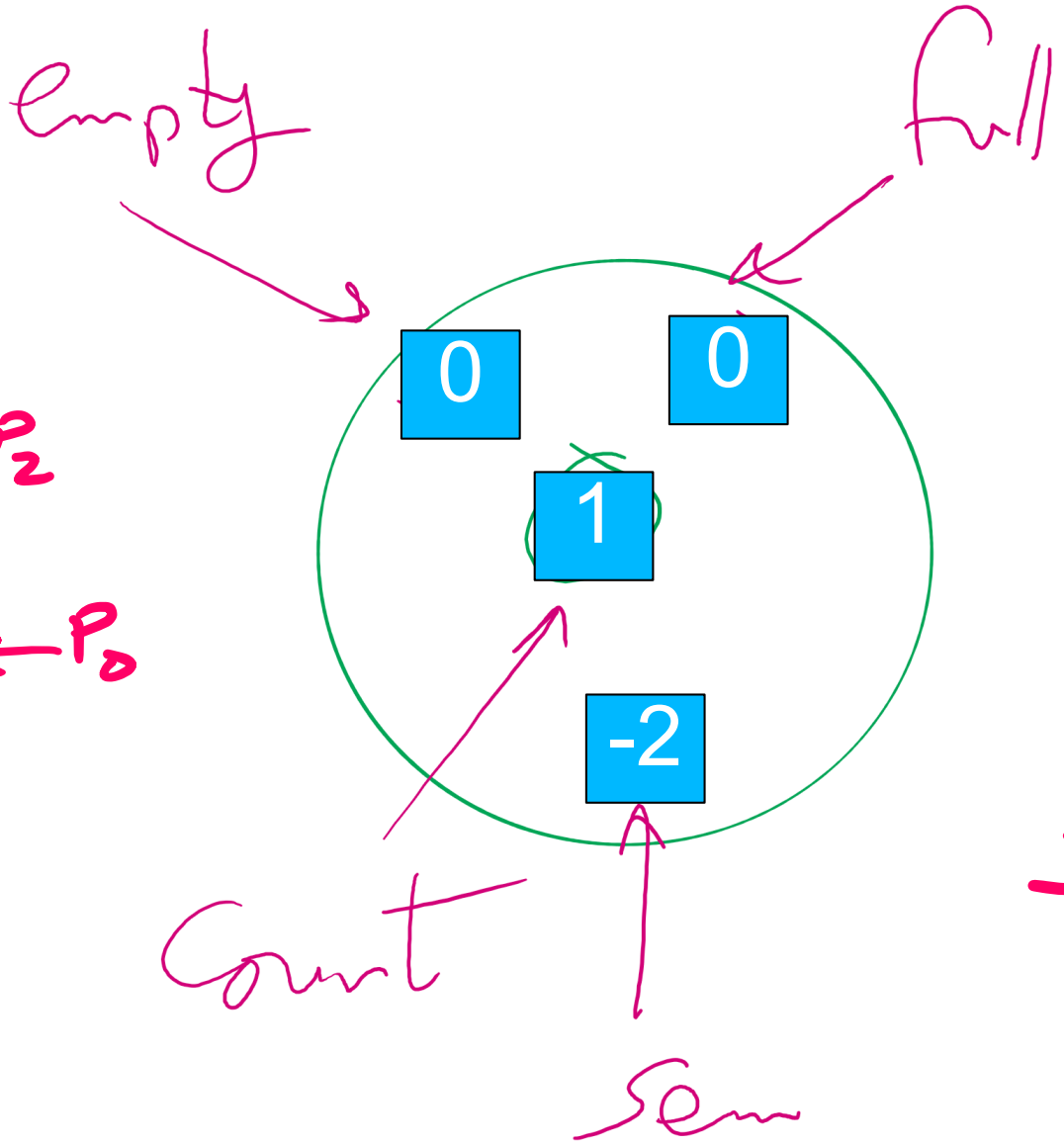
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

P1 arrives


n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Producer
down(empty)
down(sem)
buffer[in] = new item
in = (in + 1) % n
count++
up(sem)
up(full)
P1
P2
empty
full
1
0
1
-1
count
sem
Sem Queue
P1

P2 arrives

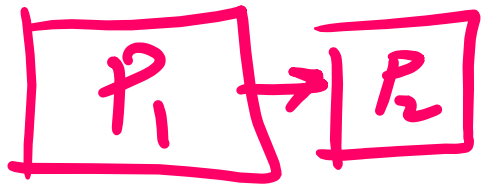




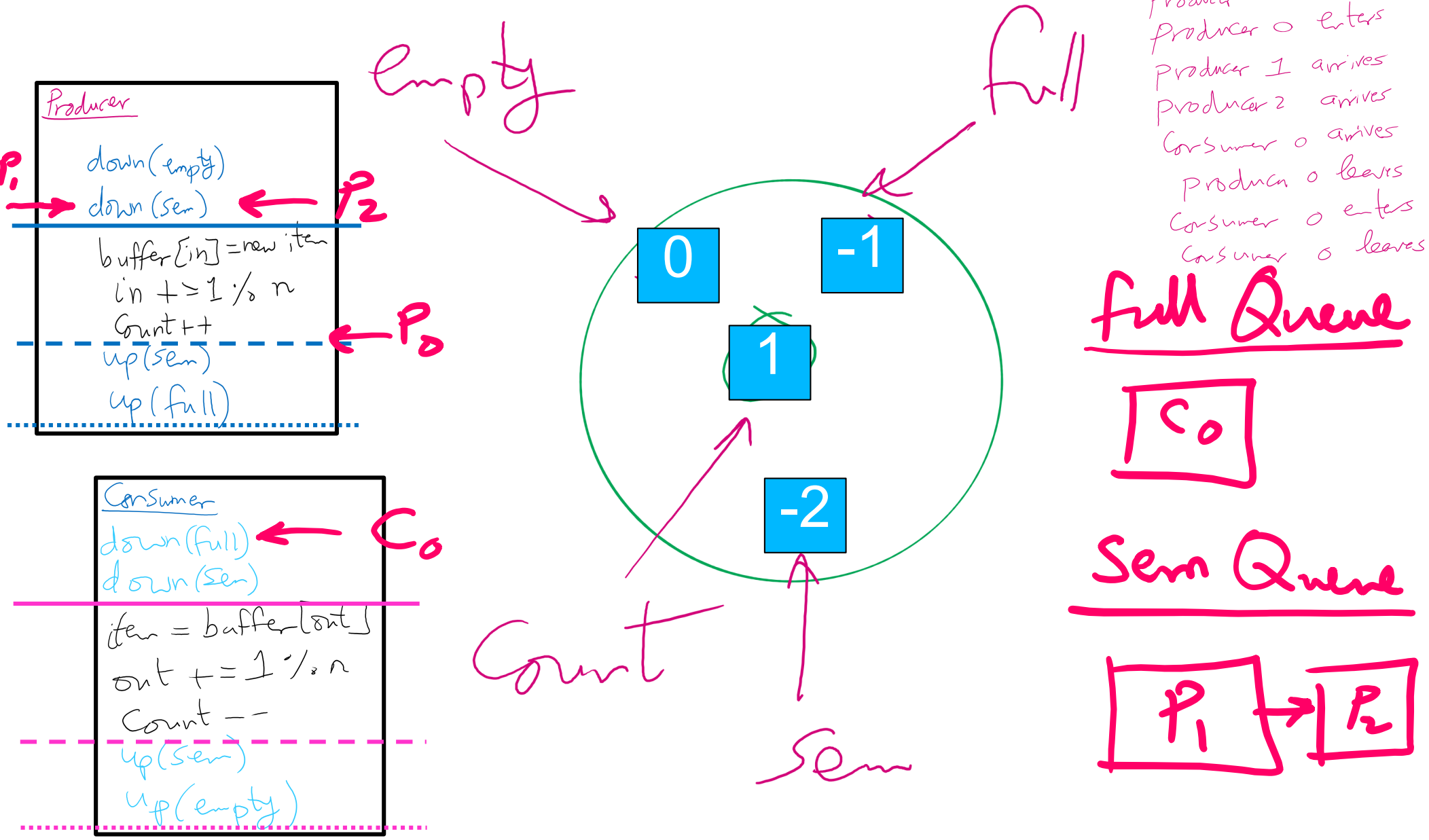
n == 3

Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

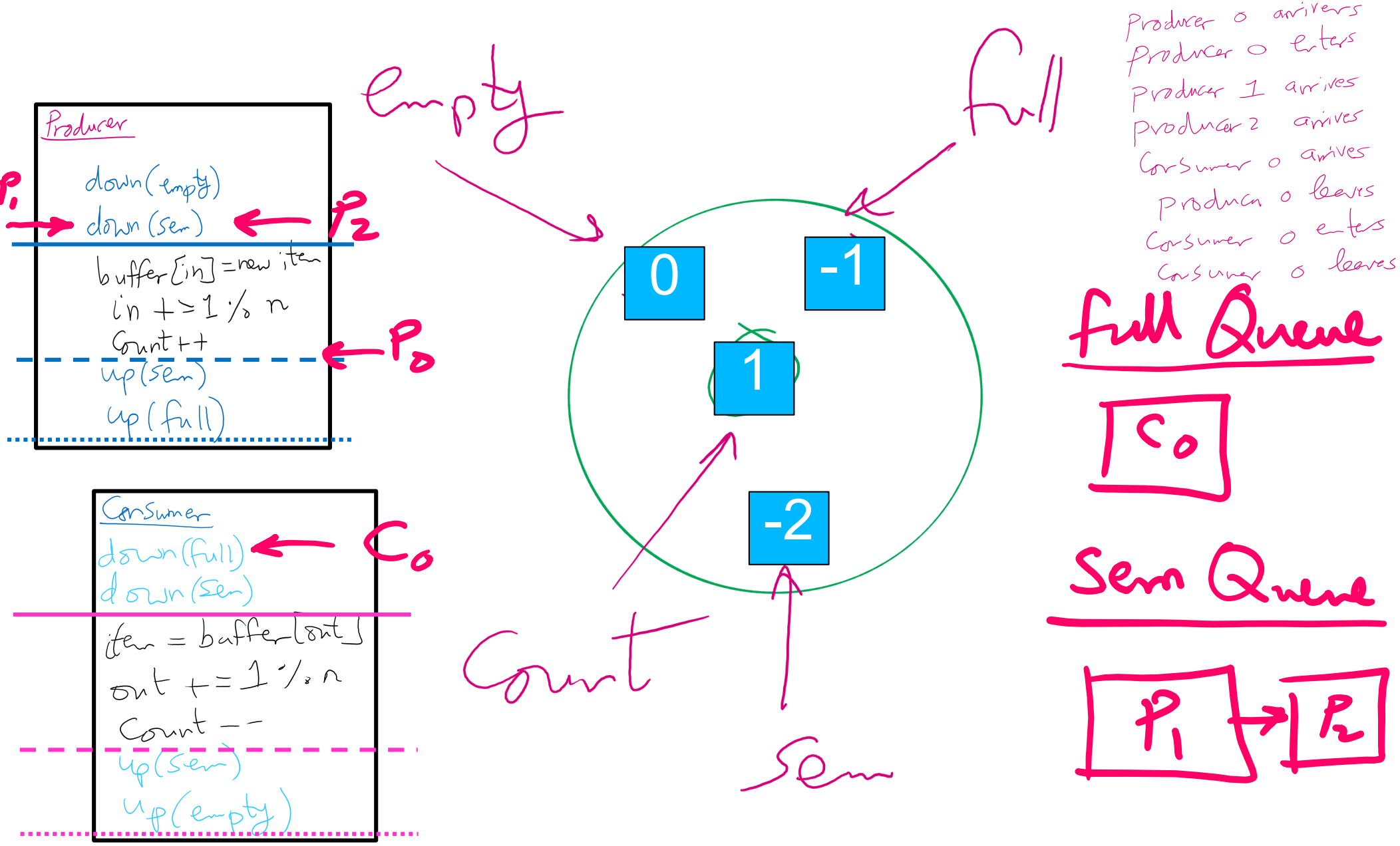
Sem Queue



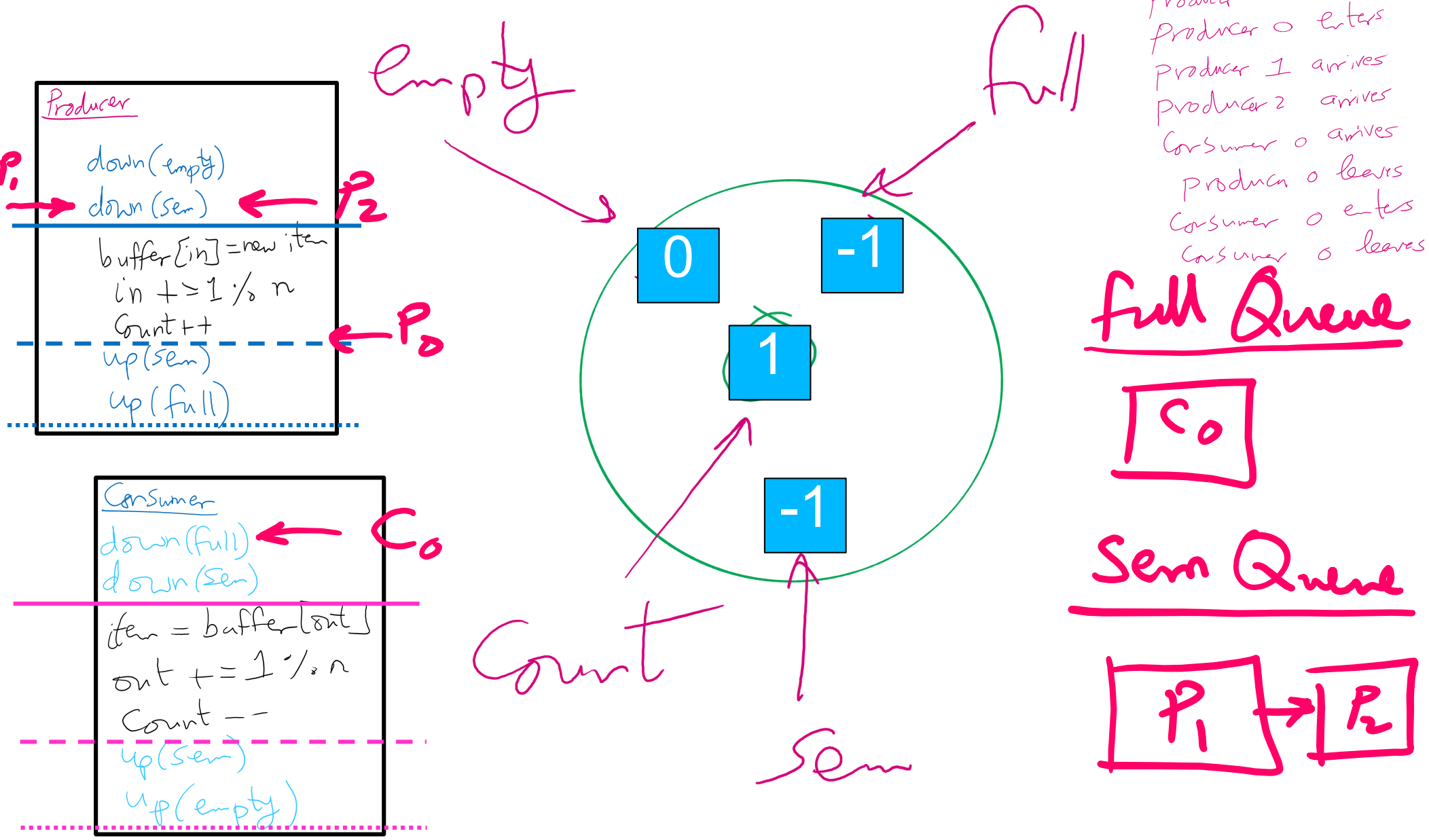
C0 arrives


n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
-1
1
-2
count
sem
full Queue
C0
sem Queue
P1
P2

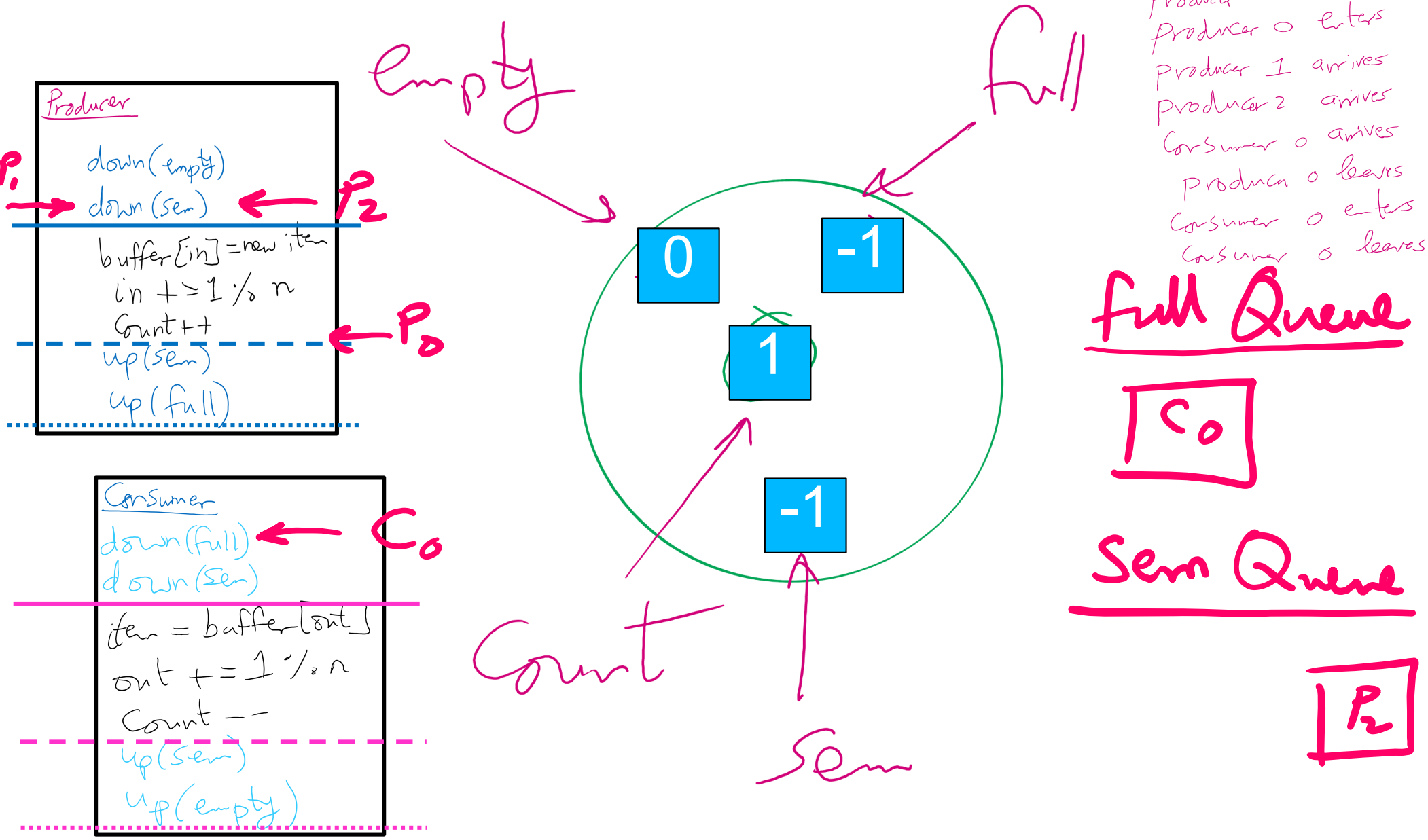
P0 leaves


n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
-1
1
-2
count
sem
full Queue
C0
Sem Queue
P1
P2

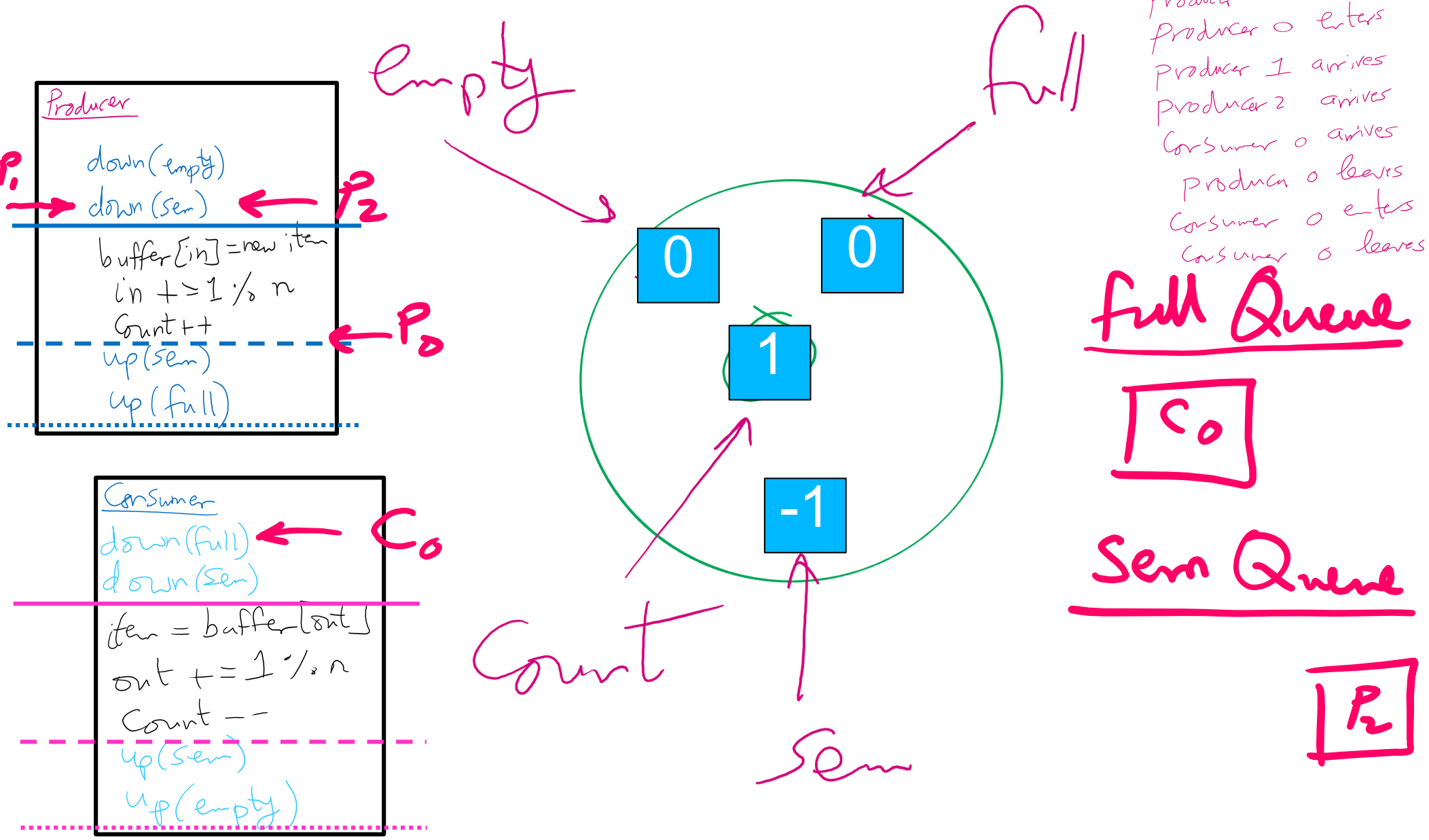
P0 leaves


n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
-1
1
-1
count
sem
full Queue
C0
sem Queue
P1
P2

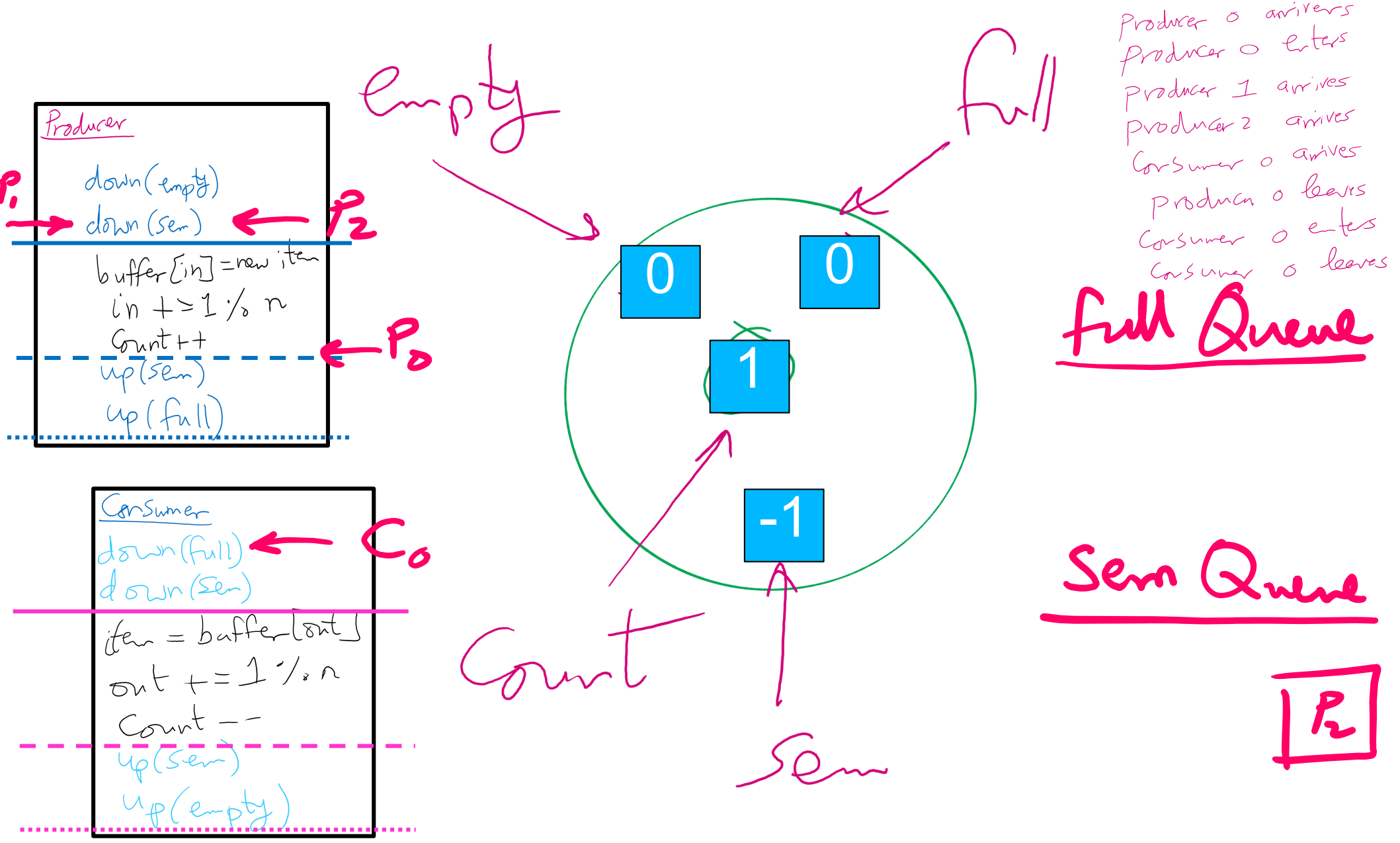
P0 leaves


Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
-1
1
-1
count
sem
n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
full Queue
C0
Sem Queue
P2

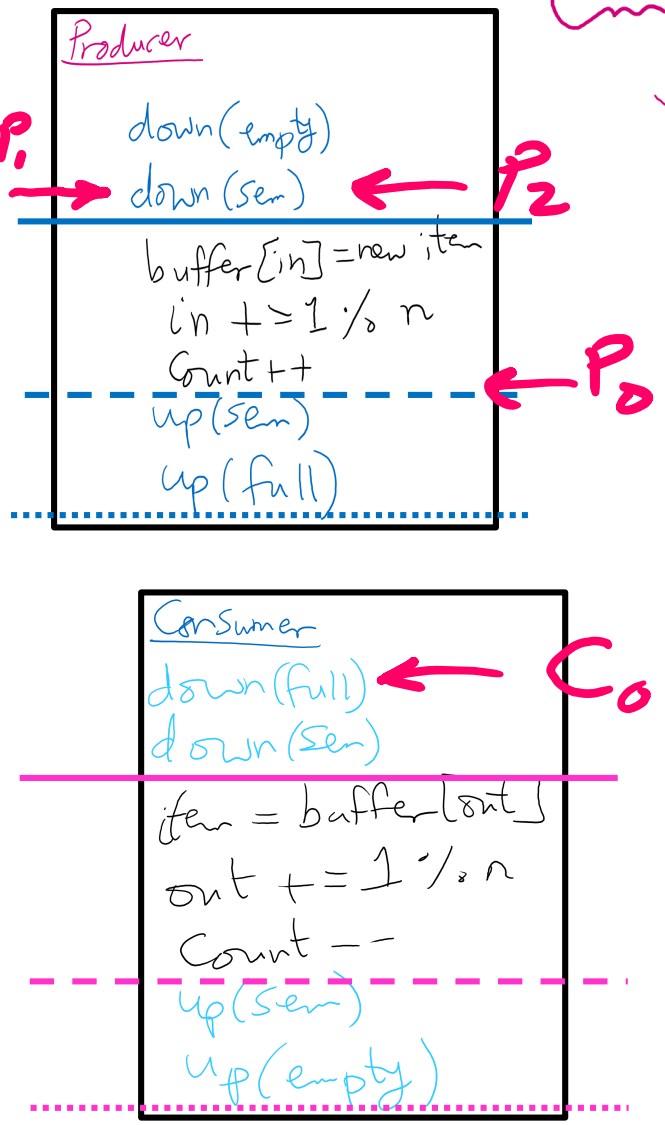
P0 leaves

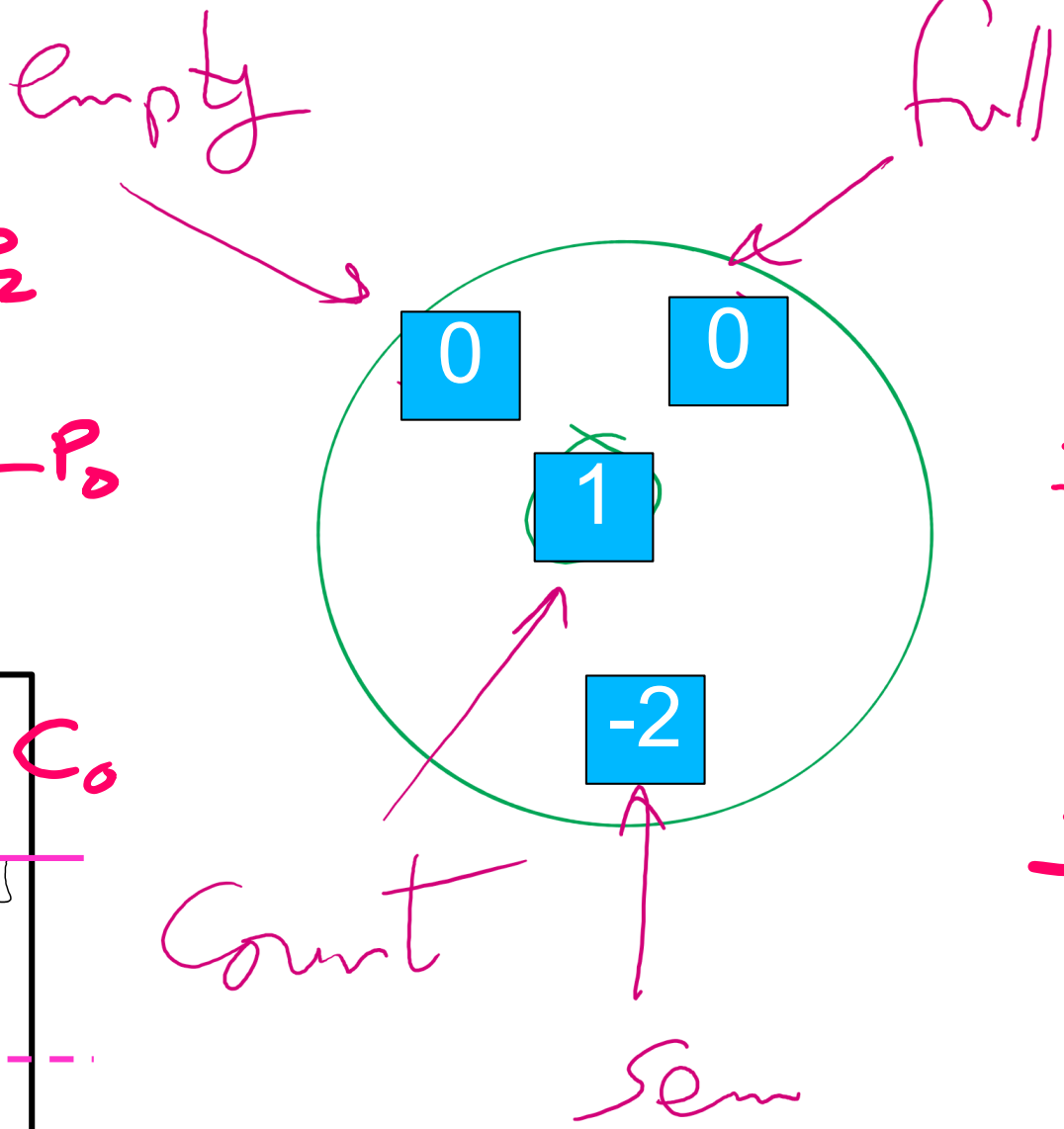

Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
0
1
-1
count
sem
n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
full Queue
C0
Sem Queue
P2

C0 enters


Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P1
P2
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
0
1
-1
count
sem
n == 3
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
full Queue
Sem Queue
P2

C0 enters





n == 3

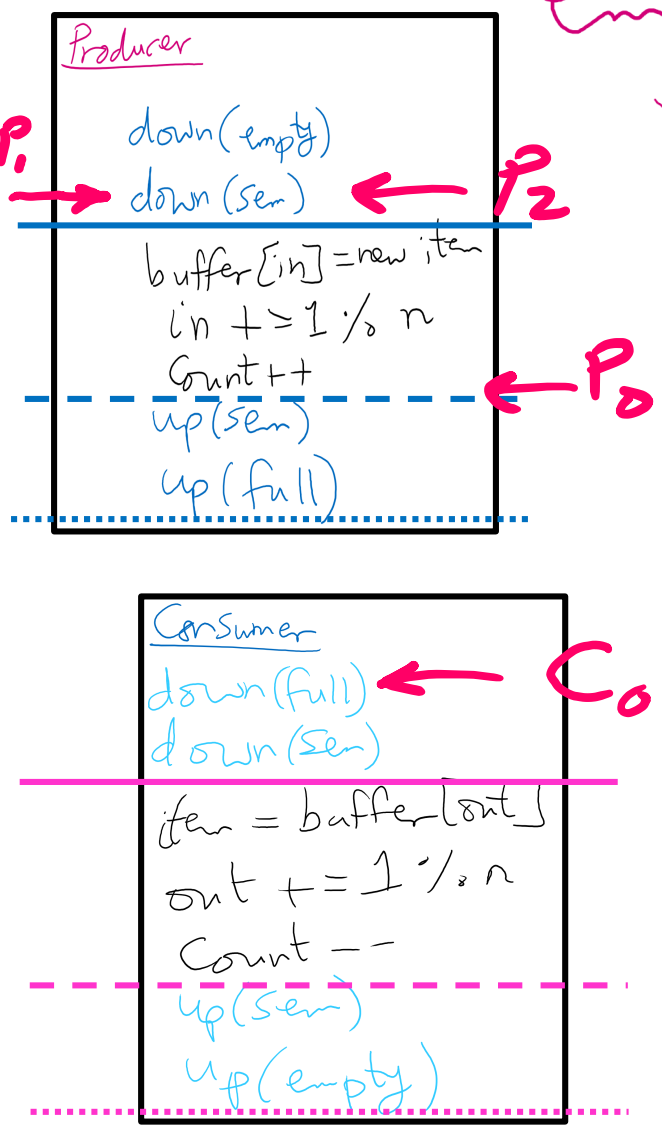
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

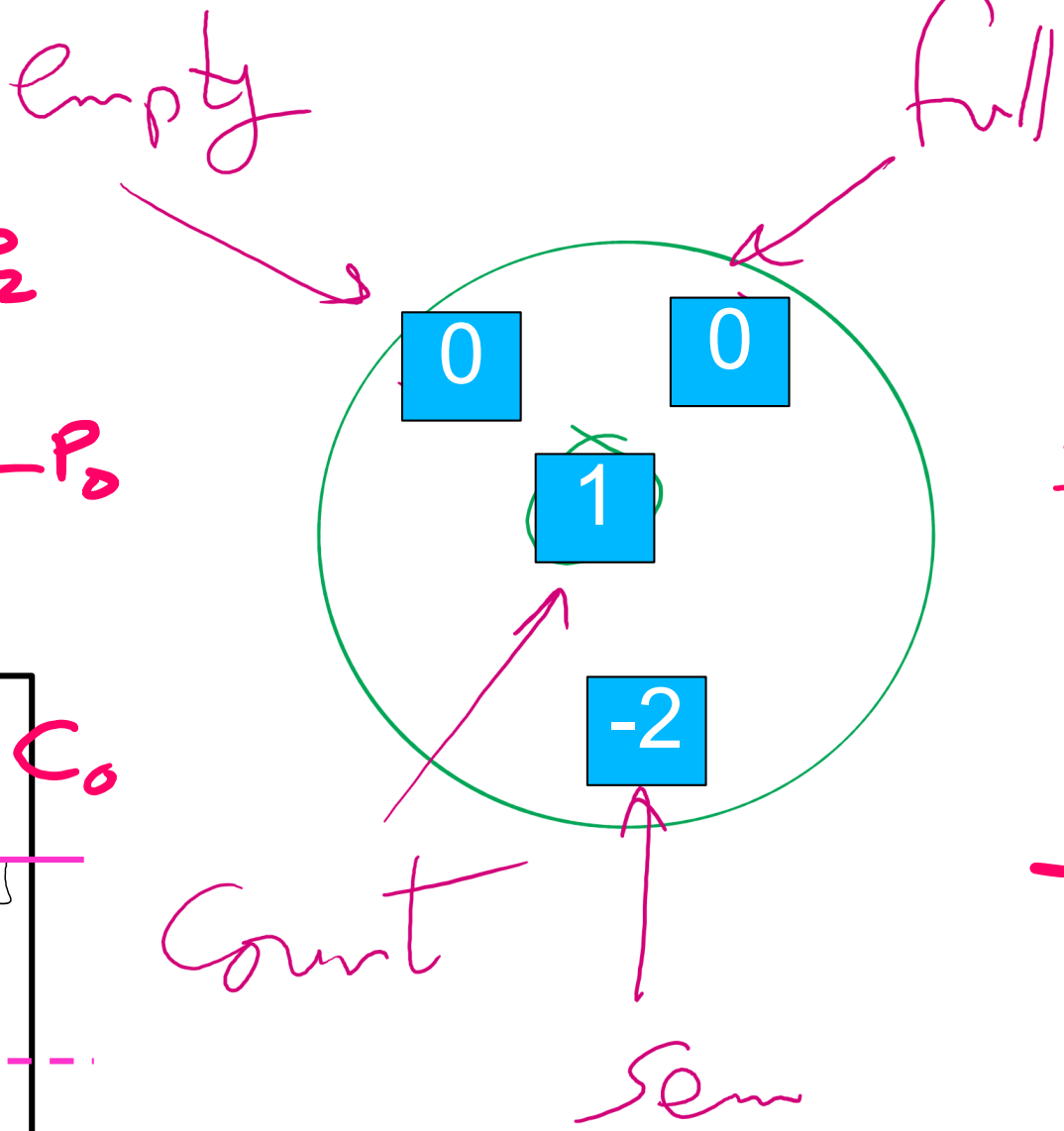
full Queue

Sem Queue

P2

C0 enters





n == 3

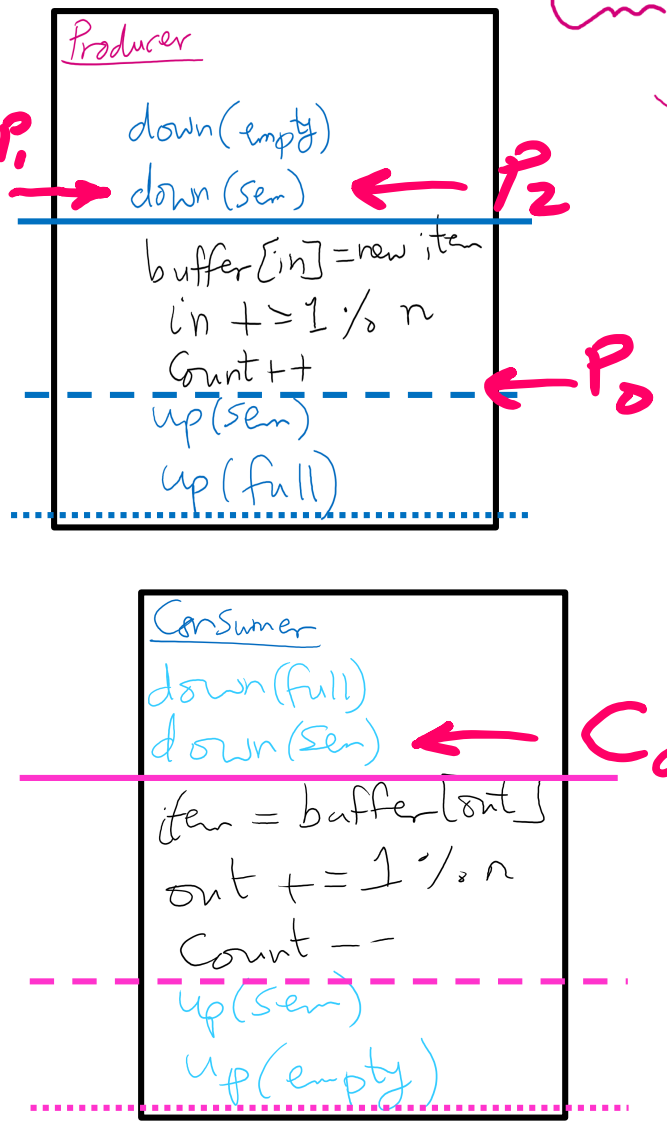
Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

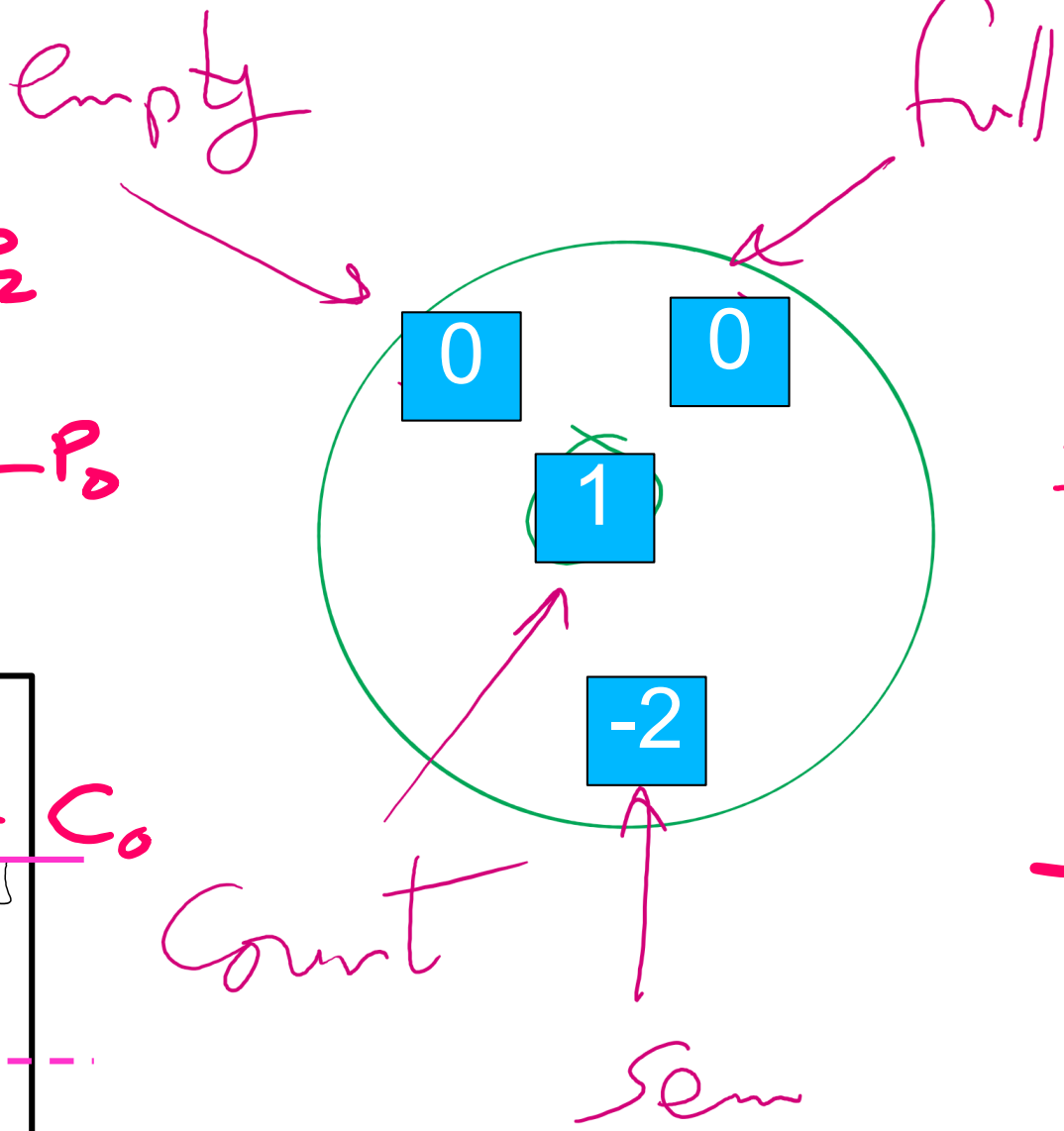
full Queue

Sem Queue



Can C0 enter?





n == 3

Producer 0 arrives
Producer 0 enters
Producer 1 arrives
Producer 2 arrives
Consumer 0 arrives
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

full Queue

Sem Queue



Example 2

n == 1

Consumer 0 arrives

Producer 0 arrives

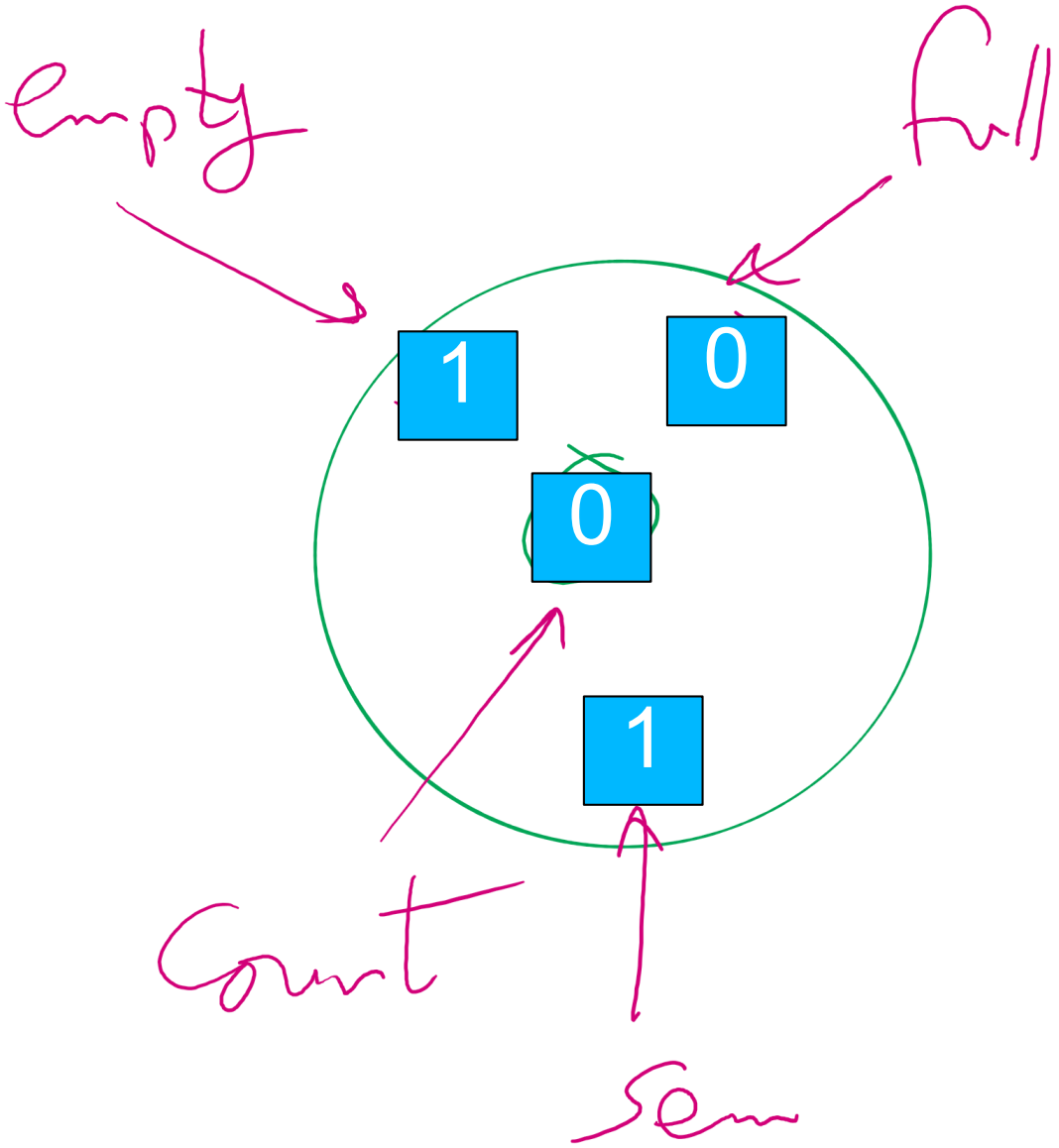
Producer 0 enters

Producer 0 leaves

Consumer 0 enters

Consumer 0 leaves

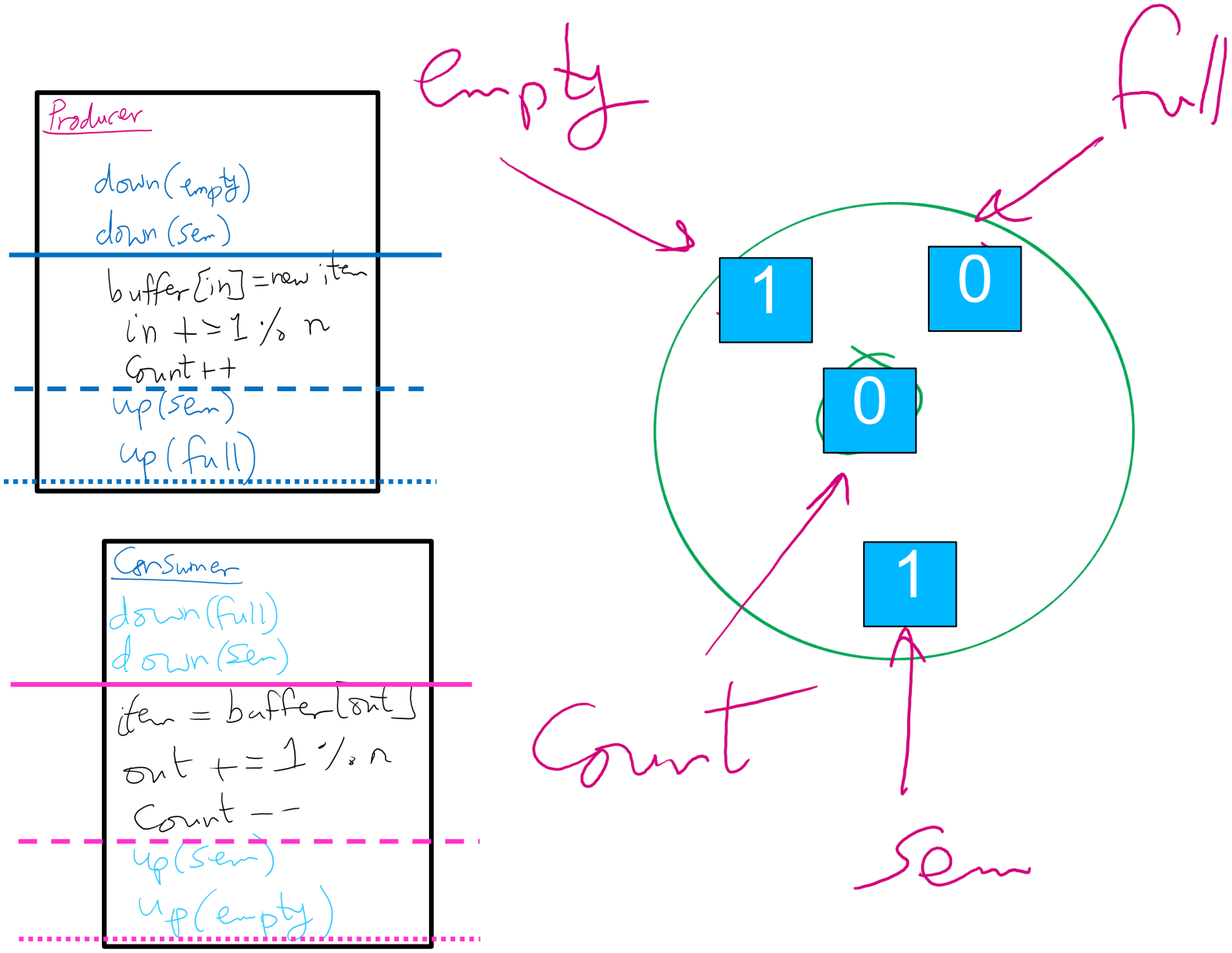
Initial state



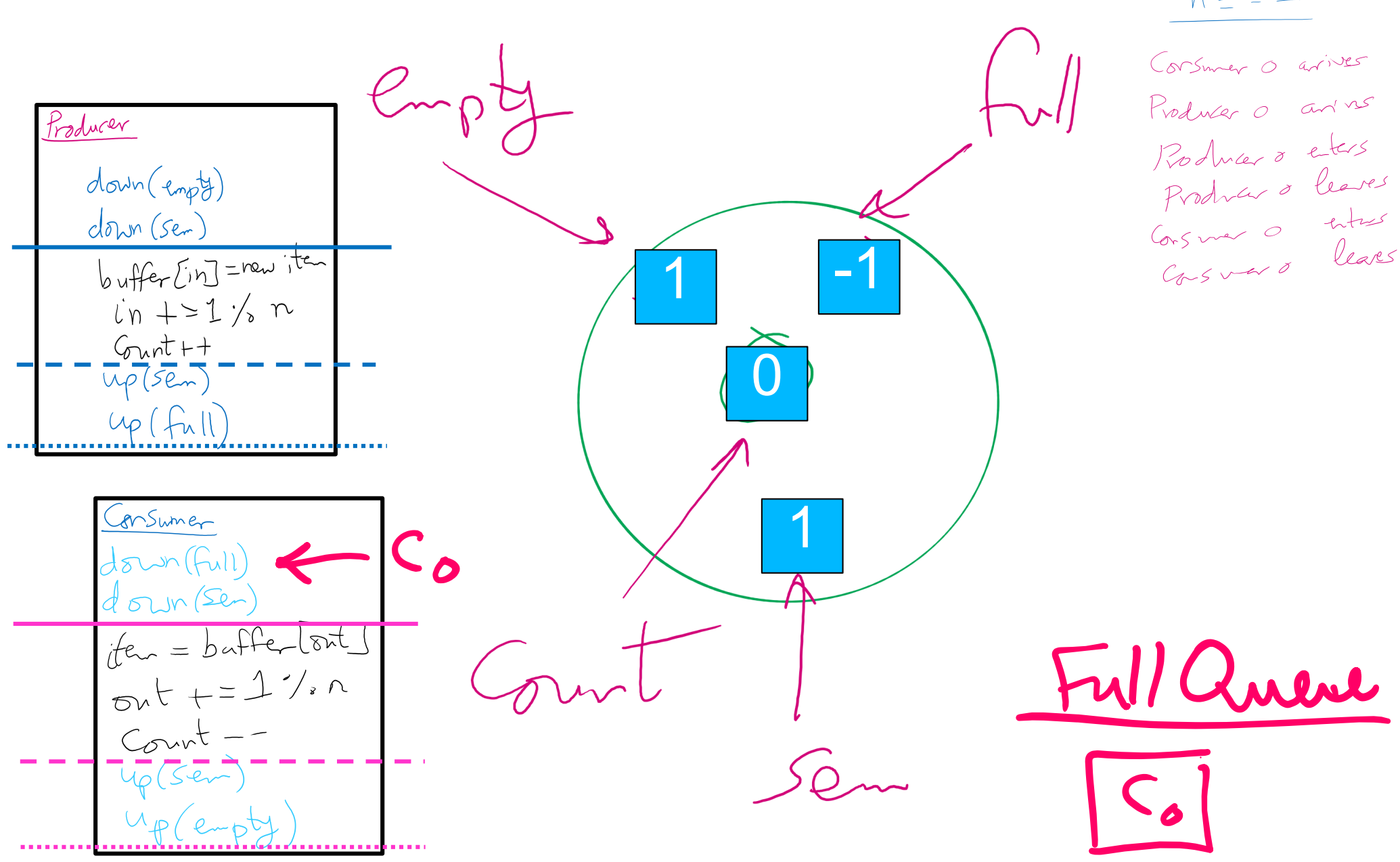
n == 1

Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

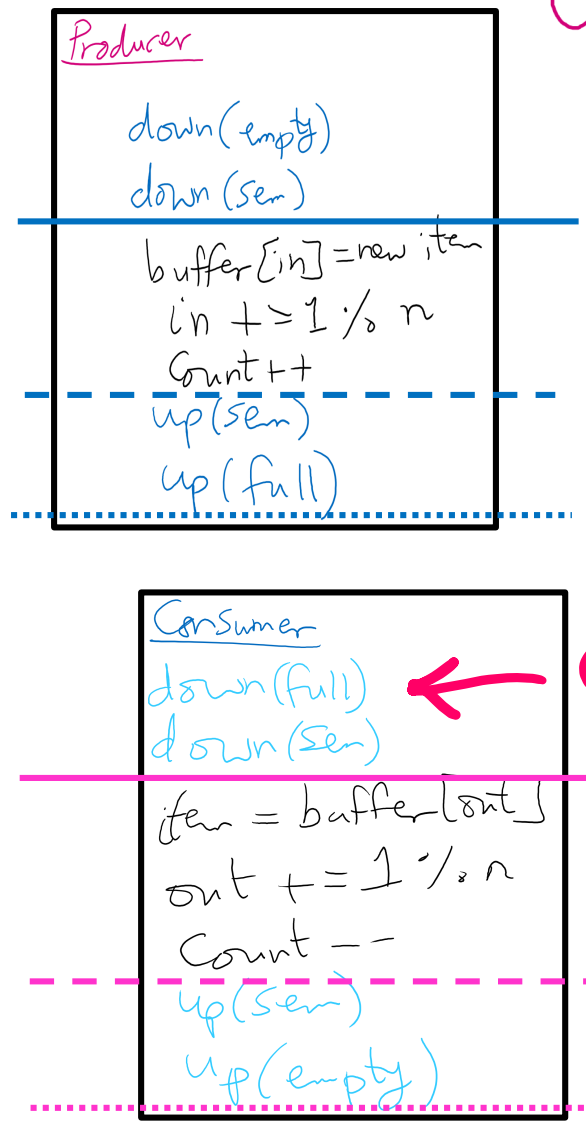
C0 arrives

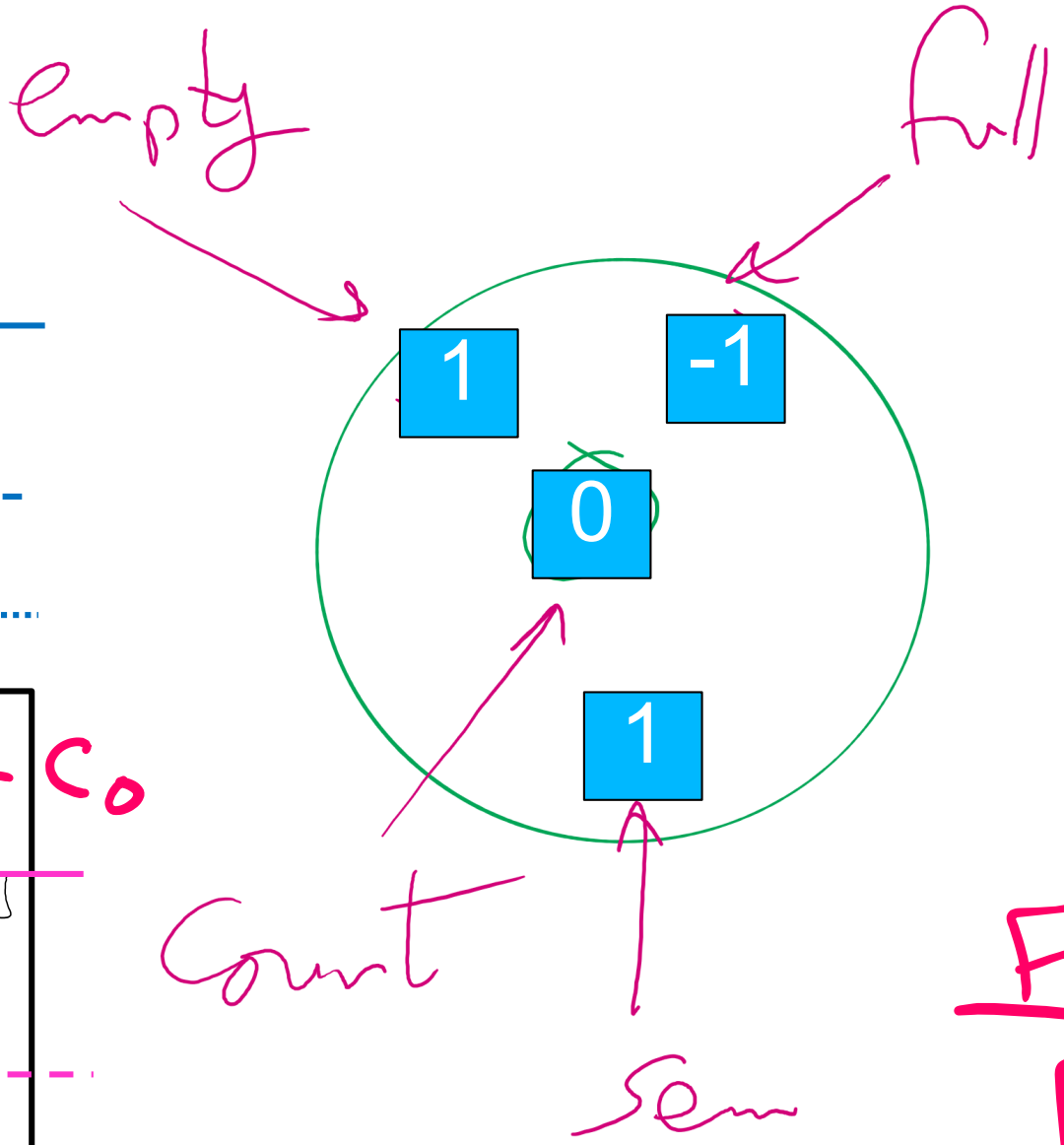

Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
empty
full
1
0
0
1
count
sem
n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

C0 arrives


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
1
-1
0
1
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue
C0

P0 arrives





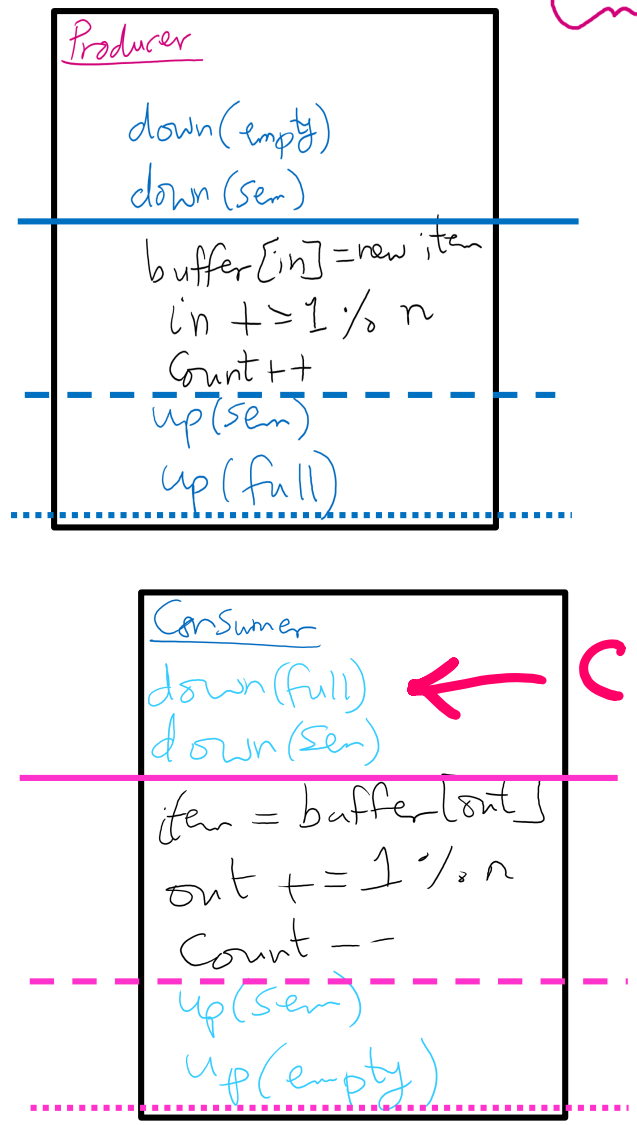
n == 1

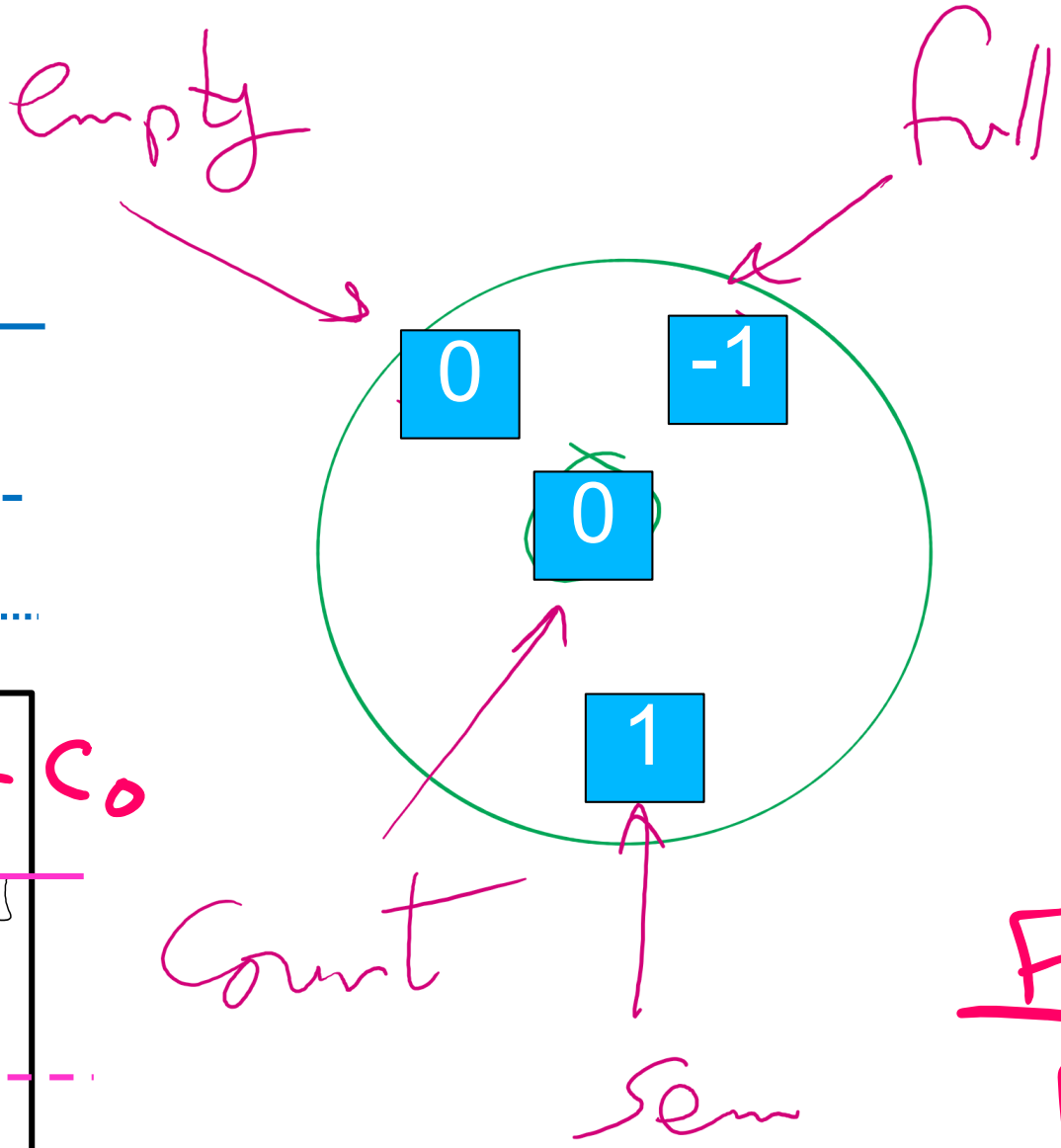
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

Full Queue

C0

P0 arrives





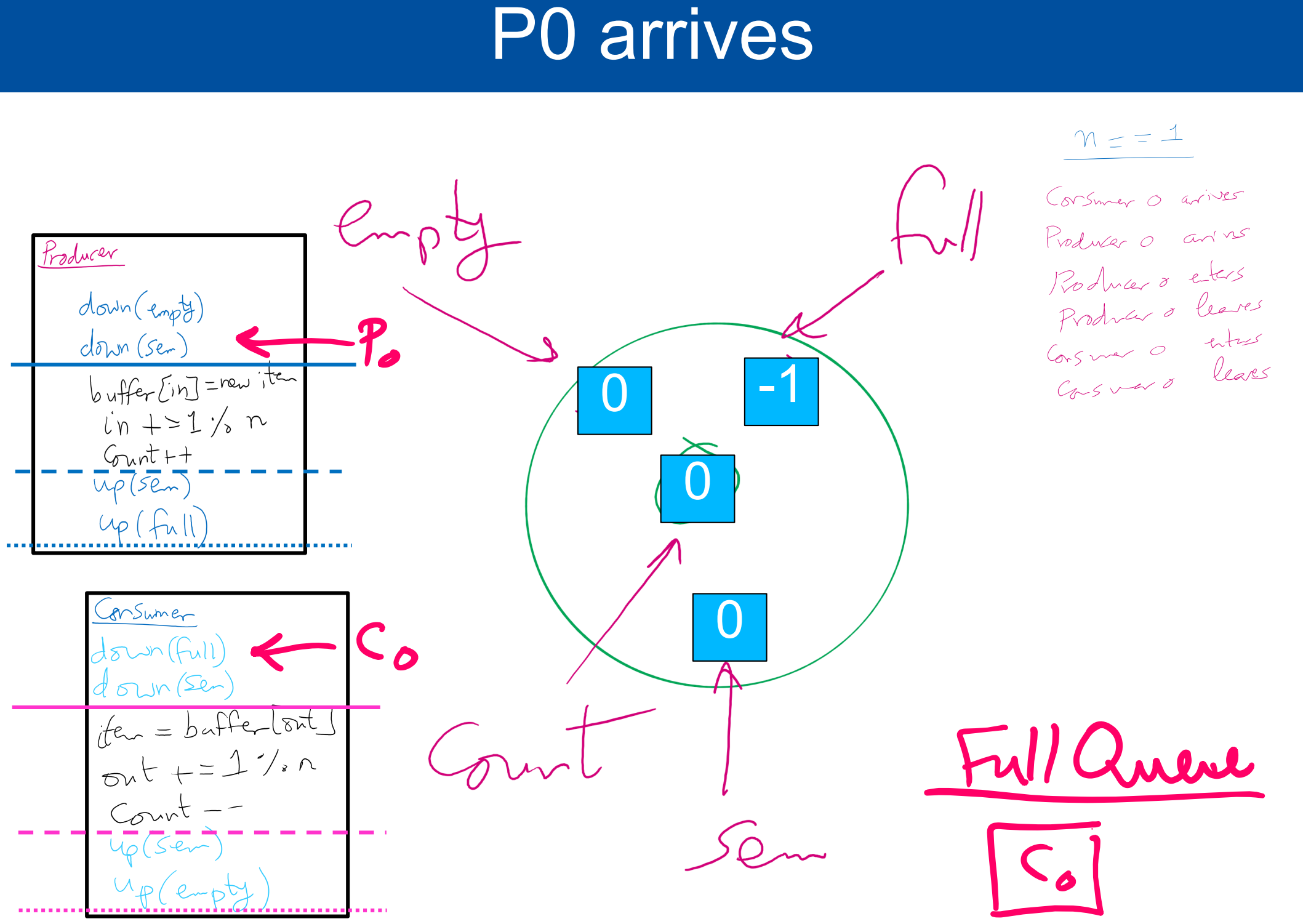
n == 1

Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves

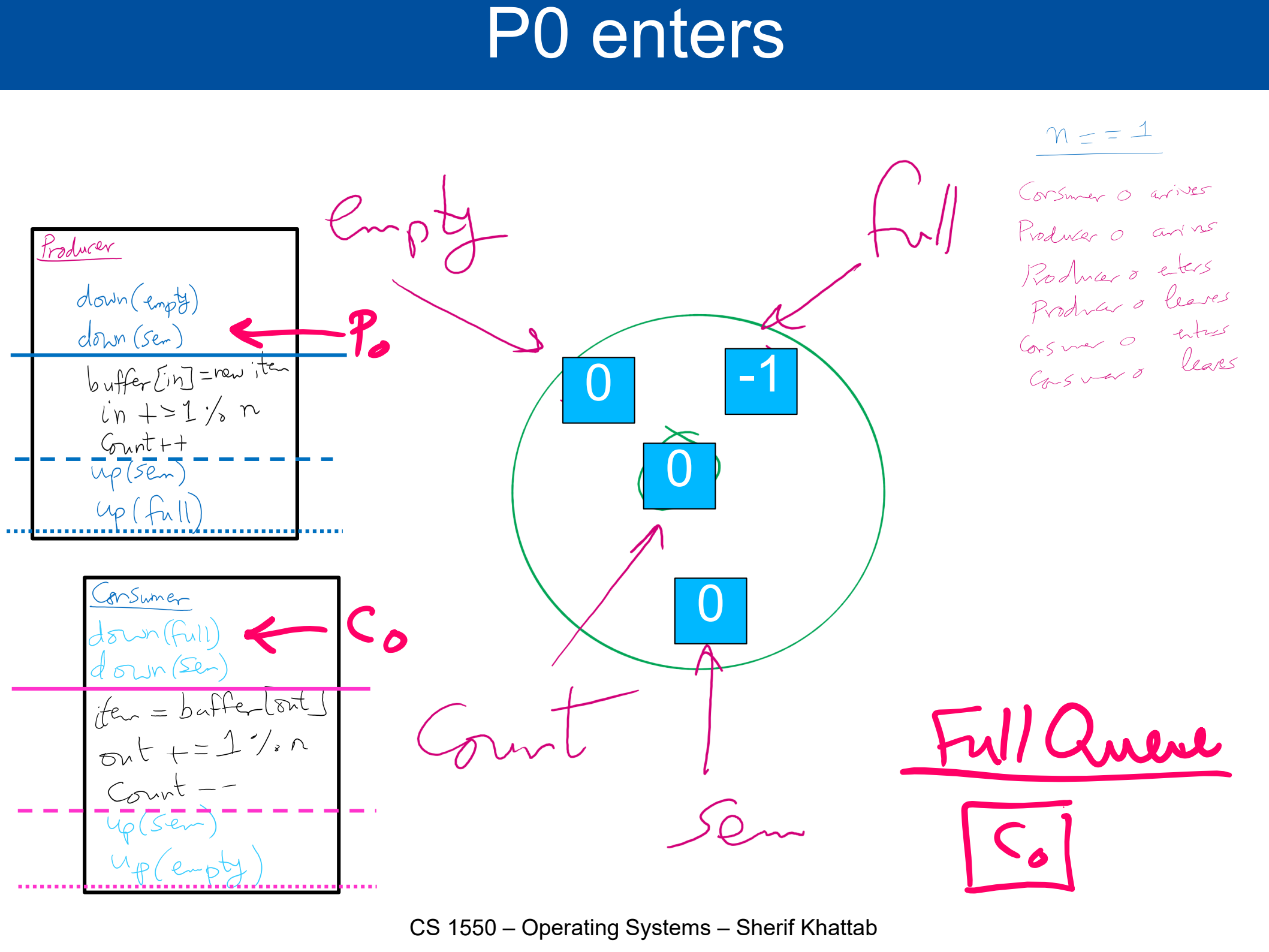
Full Queue

C0

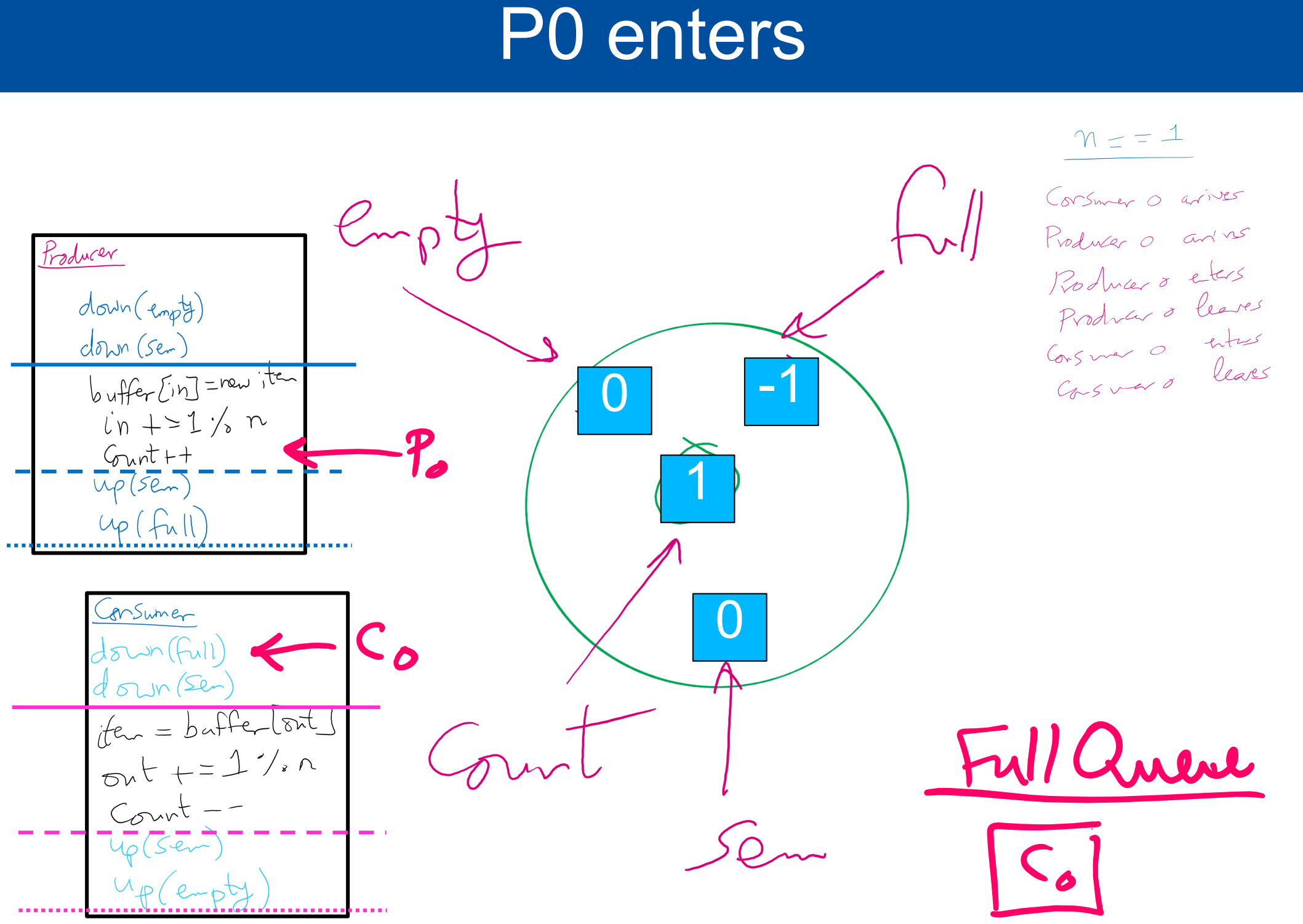
P0 arrives


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
-1
0
0
count
sem
Full Queue
C0

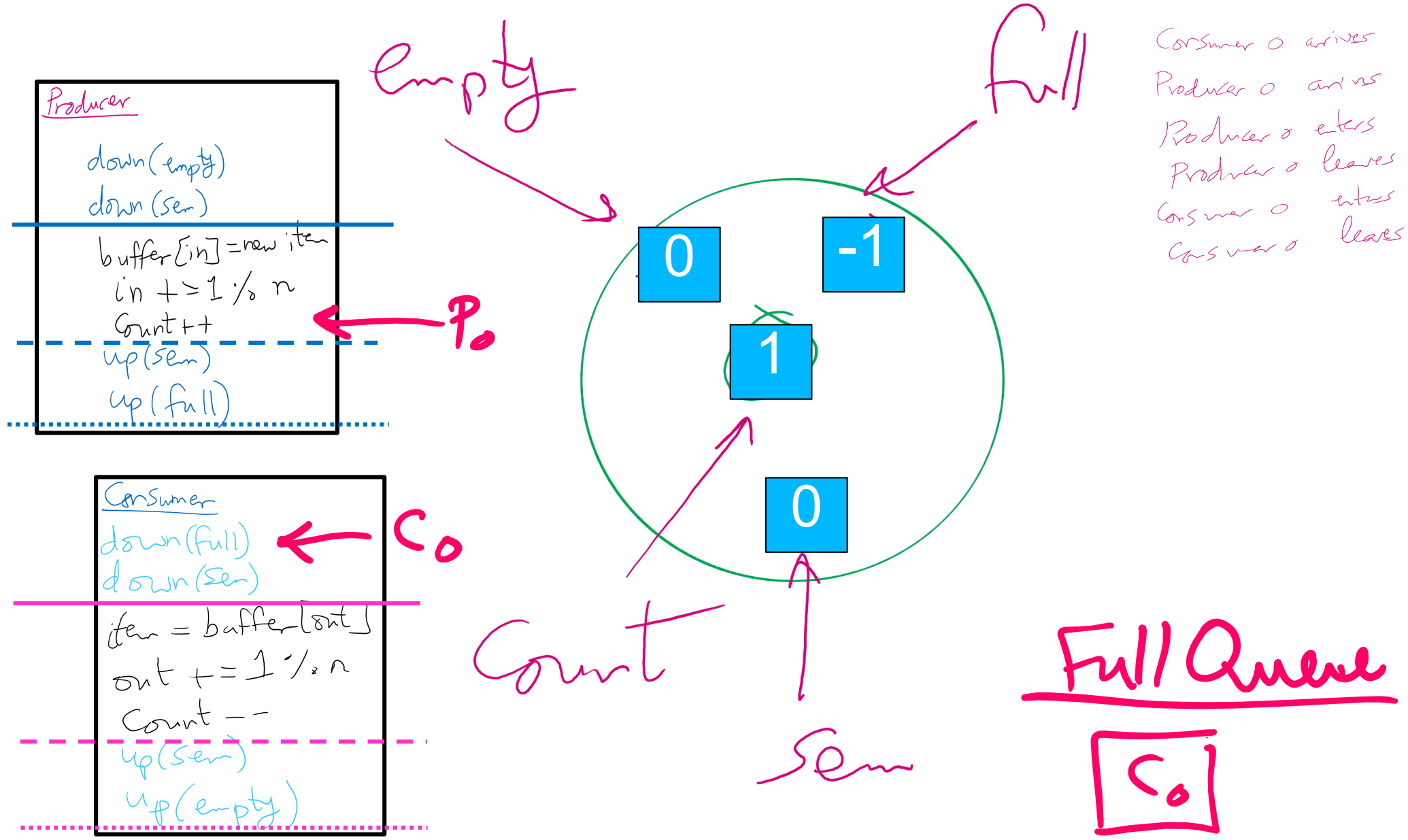
P0 enters


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
0
-1
0
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue
C0

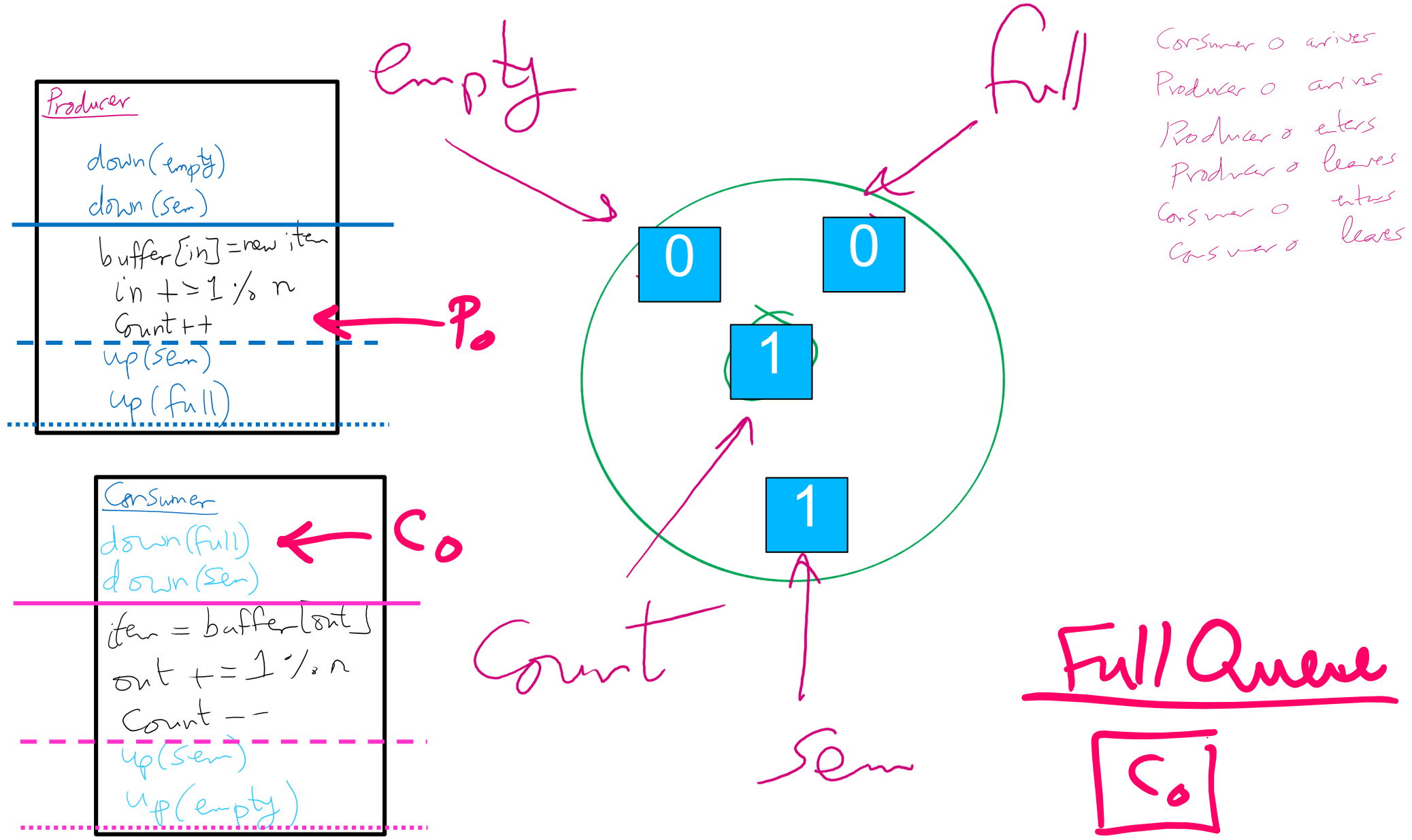
P0 enters


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
0
-1
1
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue
C0

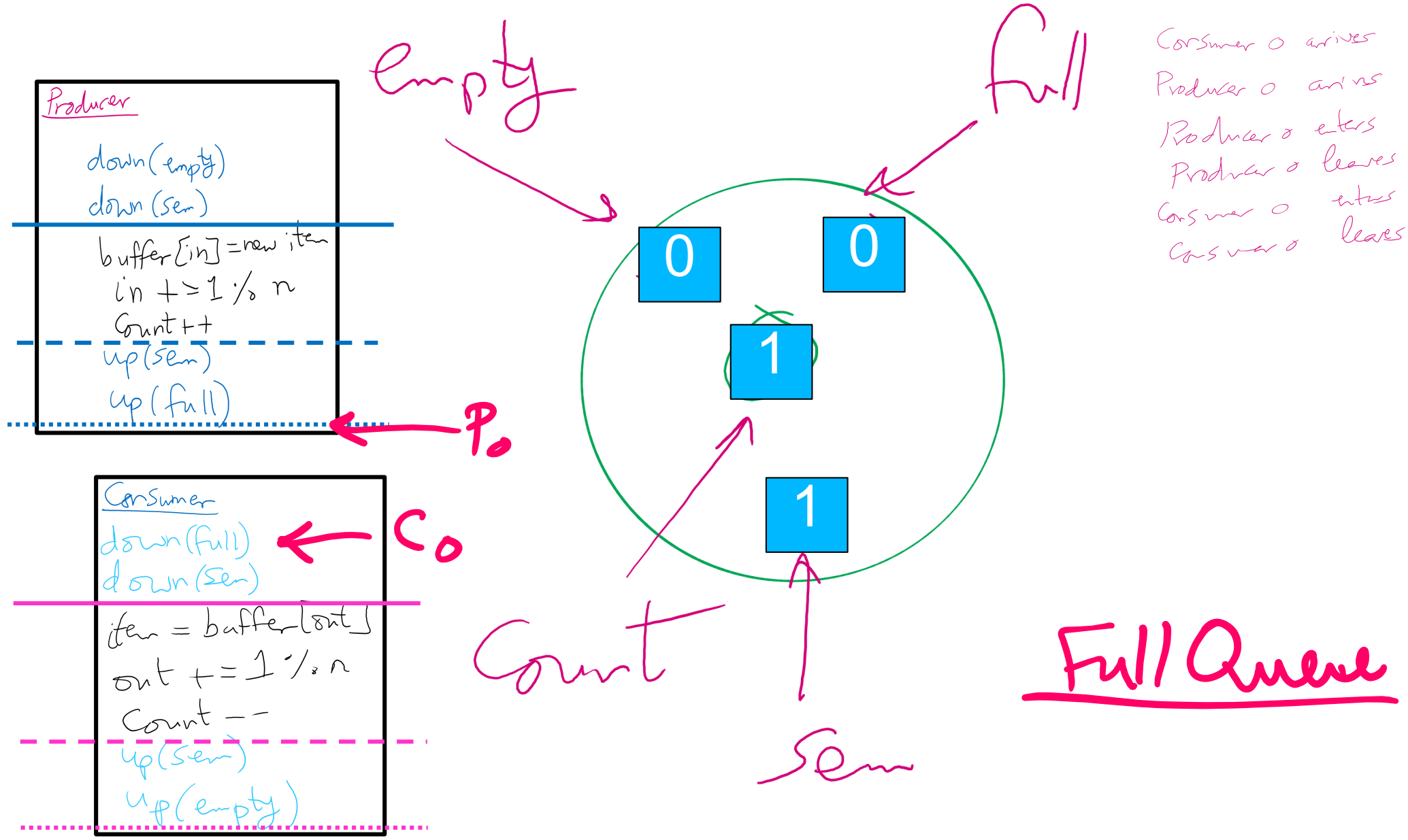
P0 leaves


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
0
-1
1
0
count
sem
Full Queue
C0

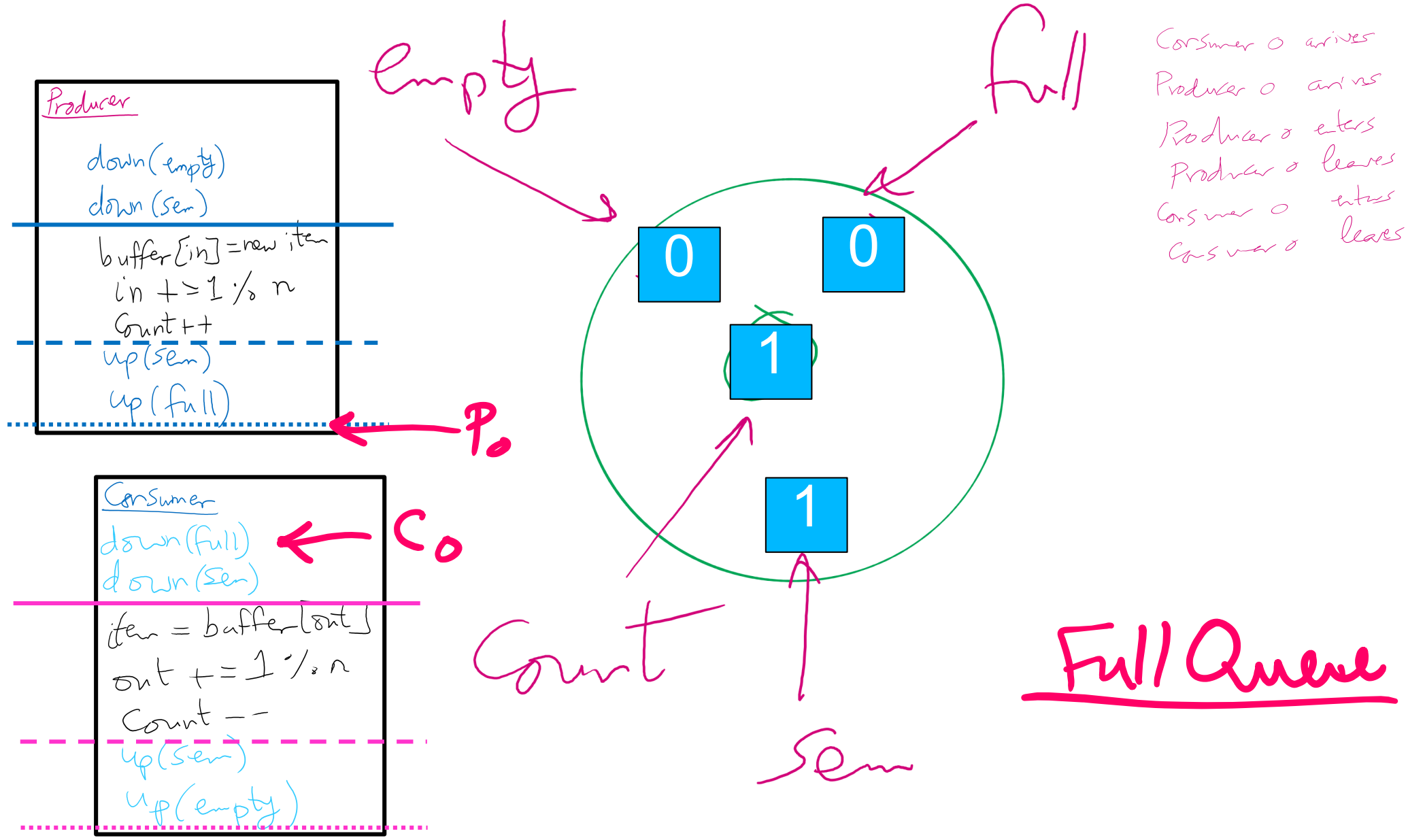
P0 leaves


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
0
0
1
1
count
sem
Full Queue
C0

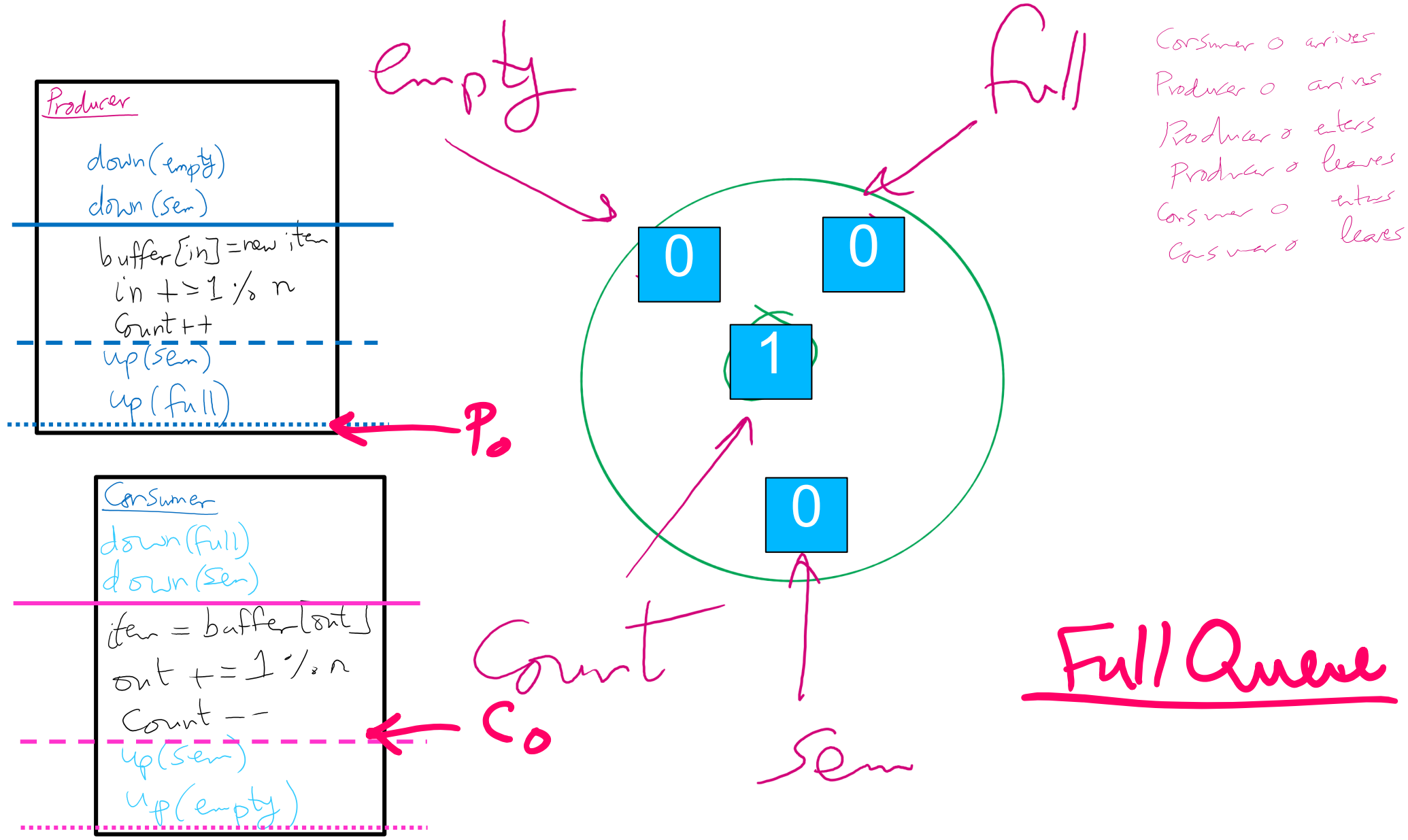
P0 leaves


Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
0
1
1
count
sem
n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Full Queue

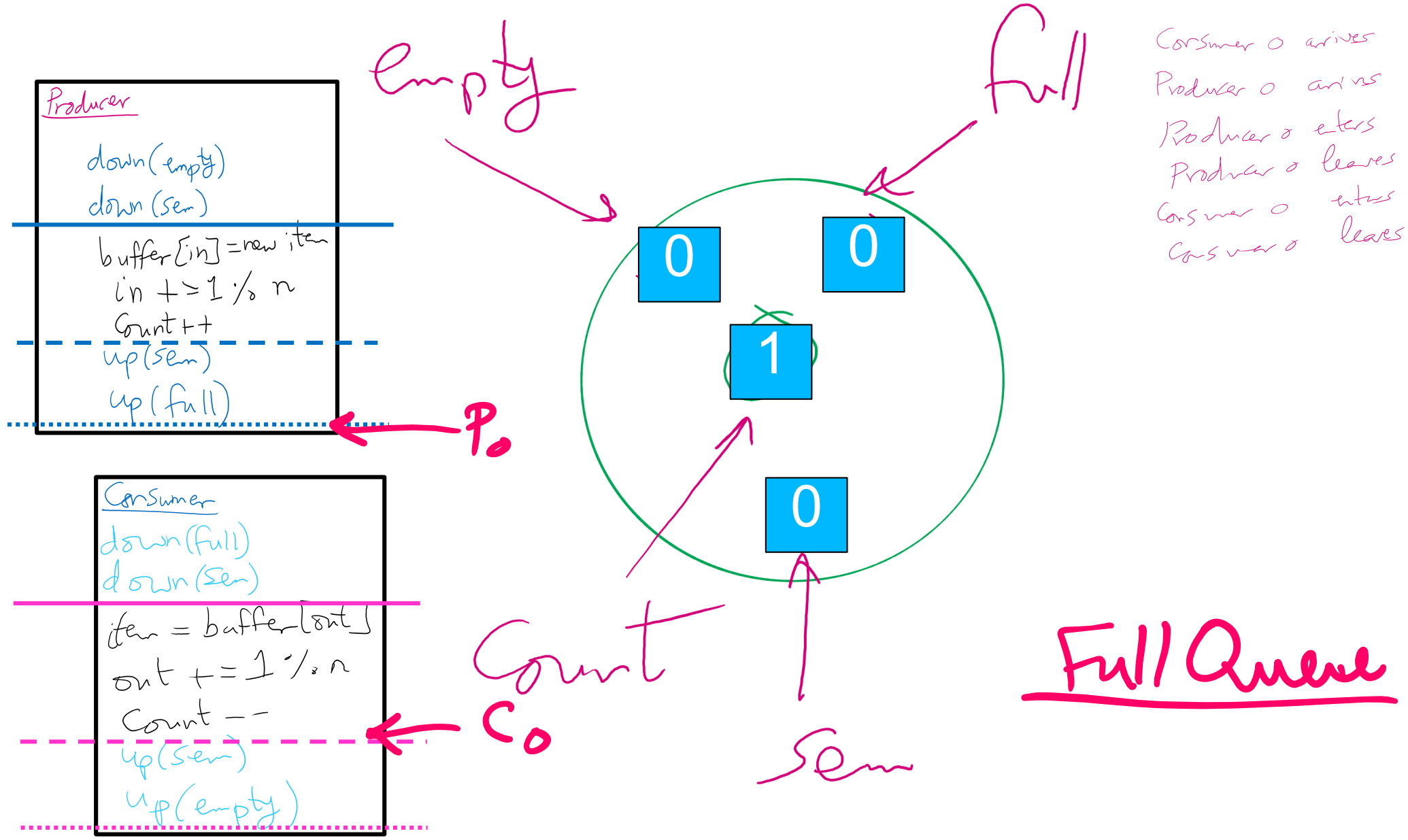
C0 enters


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
0
0
1
1
count
sem
Full Queue

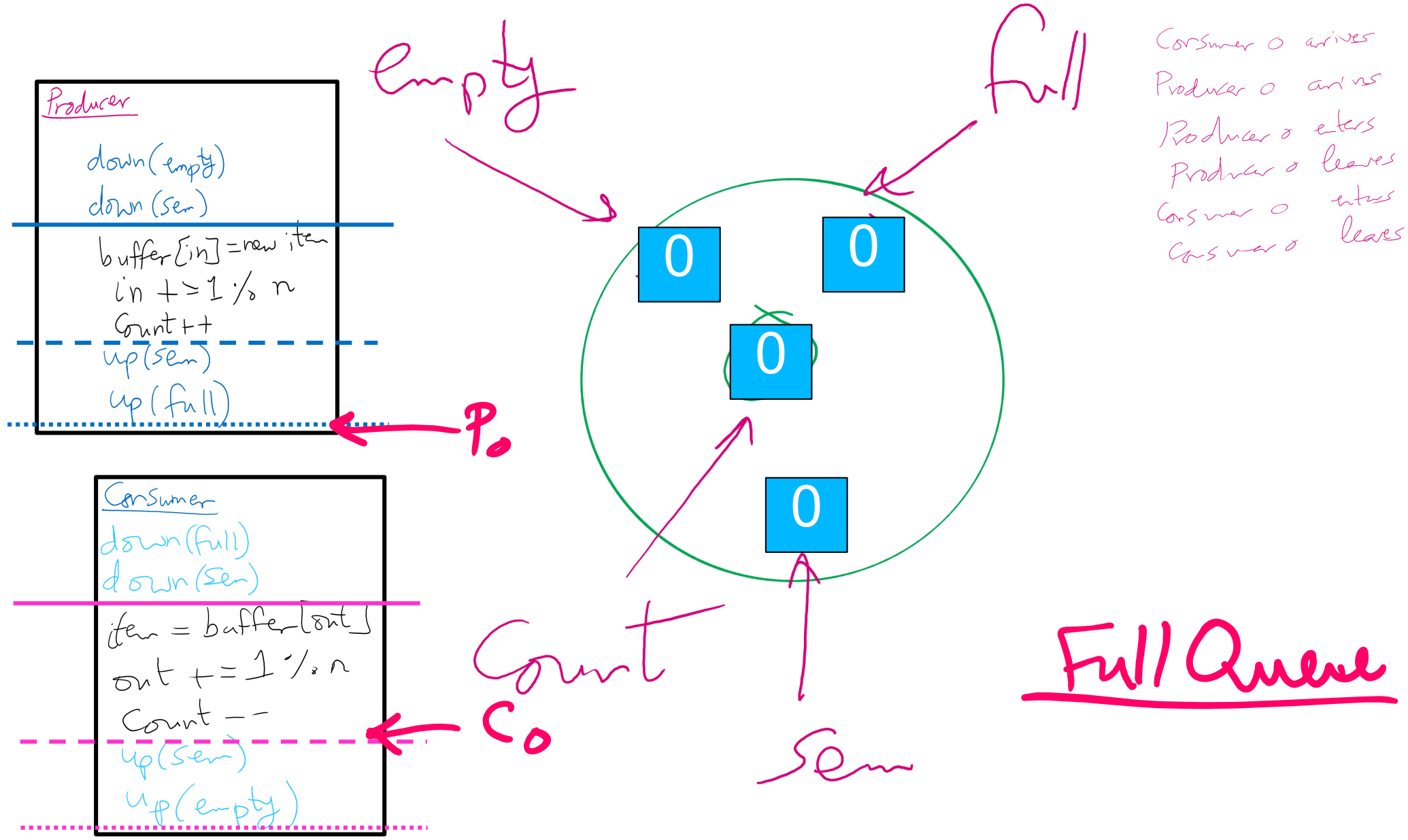
C0 enters


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
0
0
1
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue

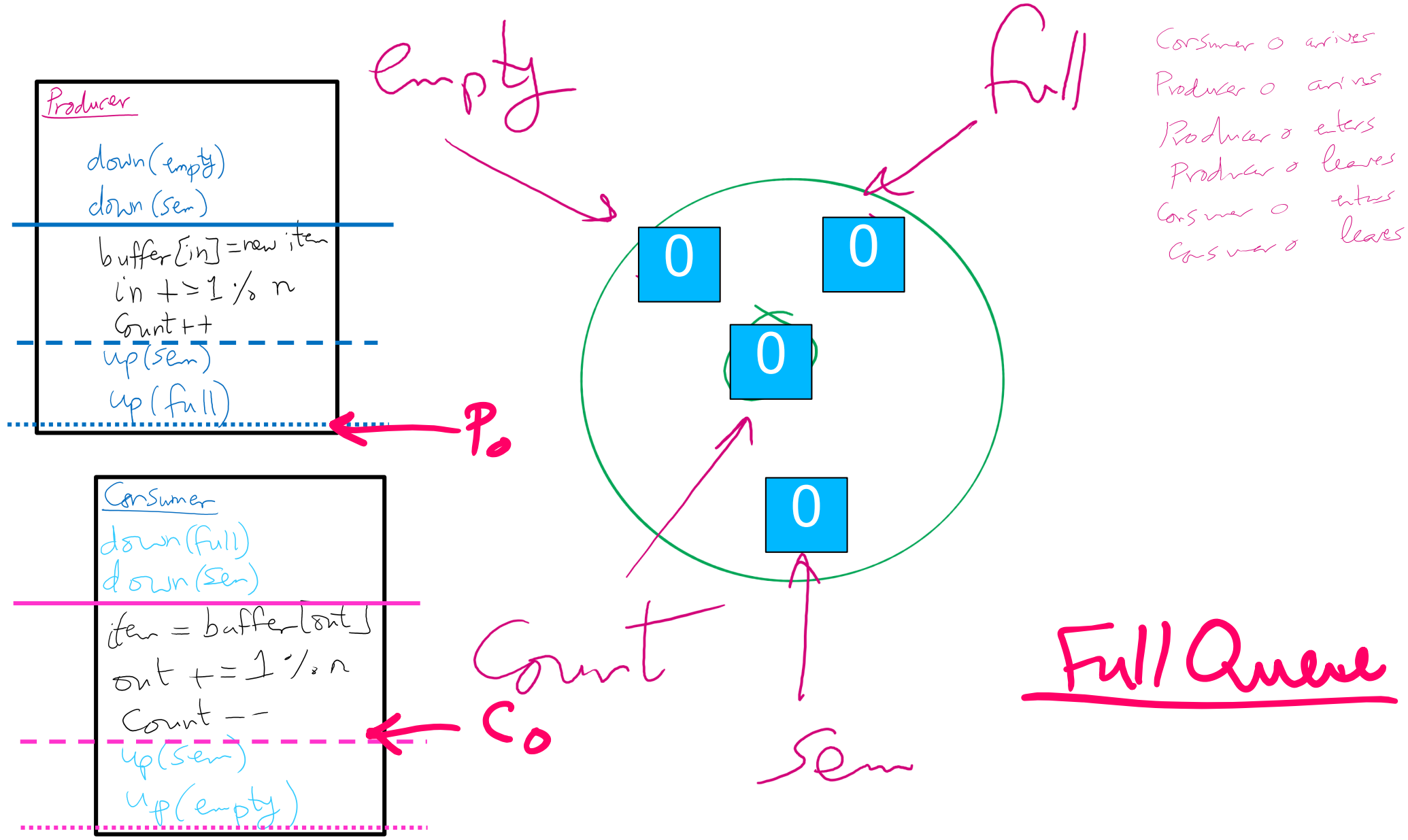
Can C0 enter?


Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
empty
full
0
0
1
0
count
sem
n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
Full Queue

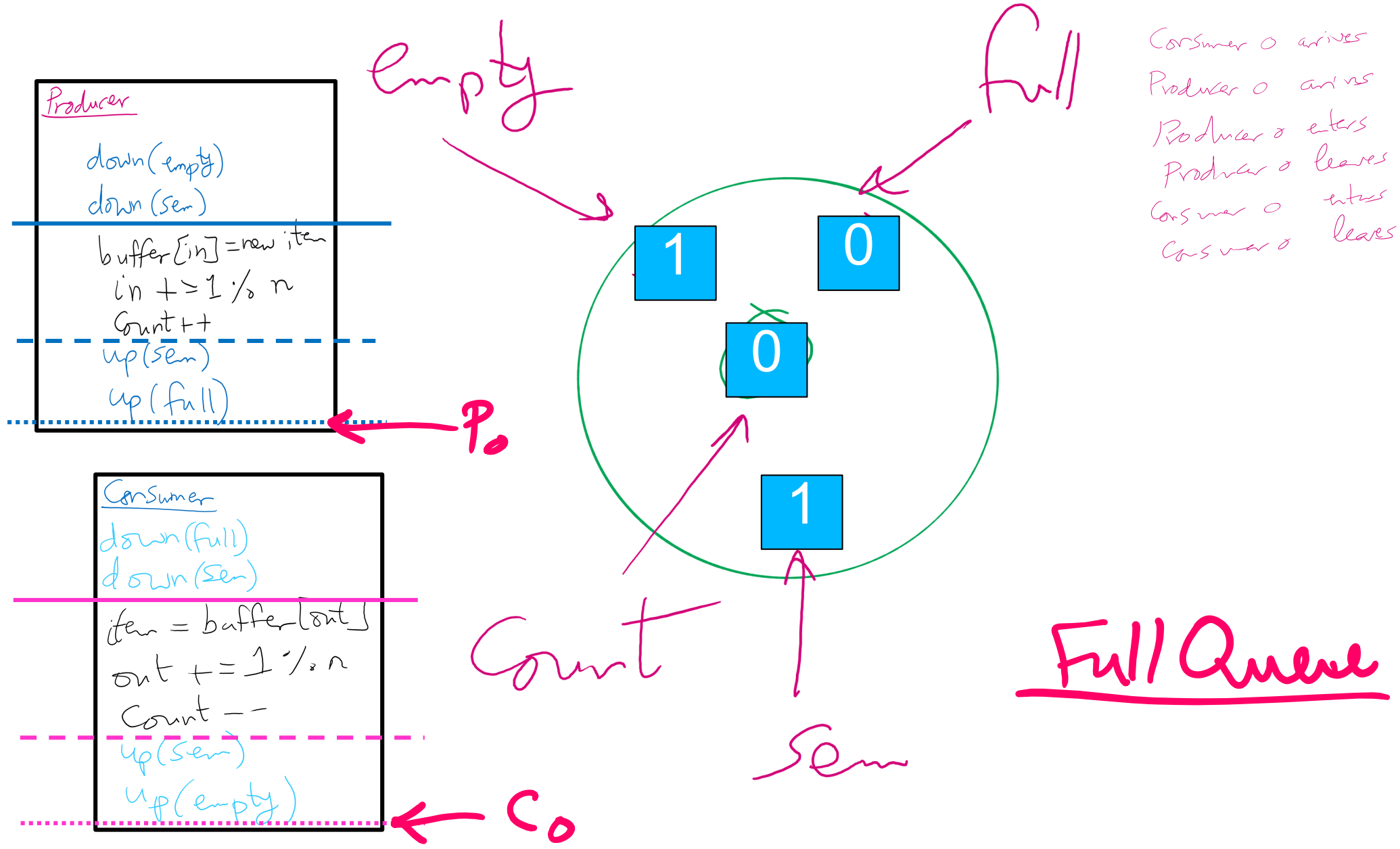
C0 enters


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
0
0
0
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue

C0 leaves


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
0
0
0
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue

C0 leaves


n == 1
Consumer 0 arrives
Producer 0 arrives
Producer 0 enters
Producer 0 leaves
Consumer 0 enters
Consumer 0 leaves
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
1
0
0
1
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0
count
sem
Full Queue

Example 3

$n == 2$

C_0 arrives

C_1 arrives

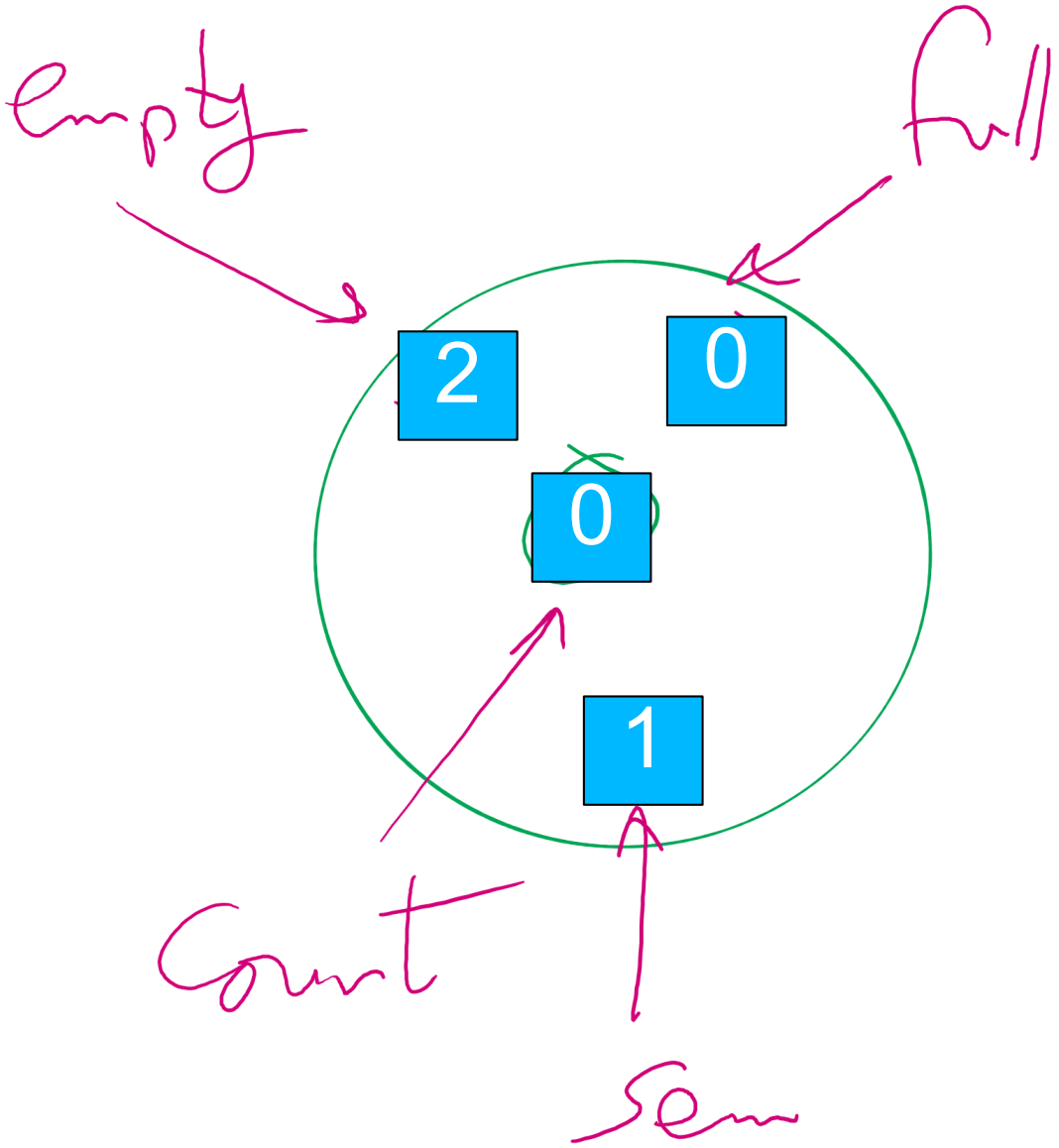
C_2 arrives

P_0 arrives

P_0 enters

C_0 enters

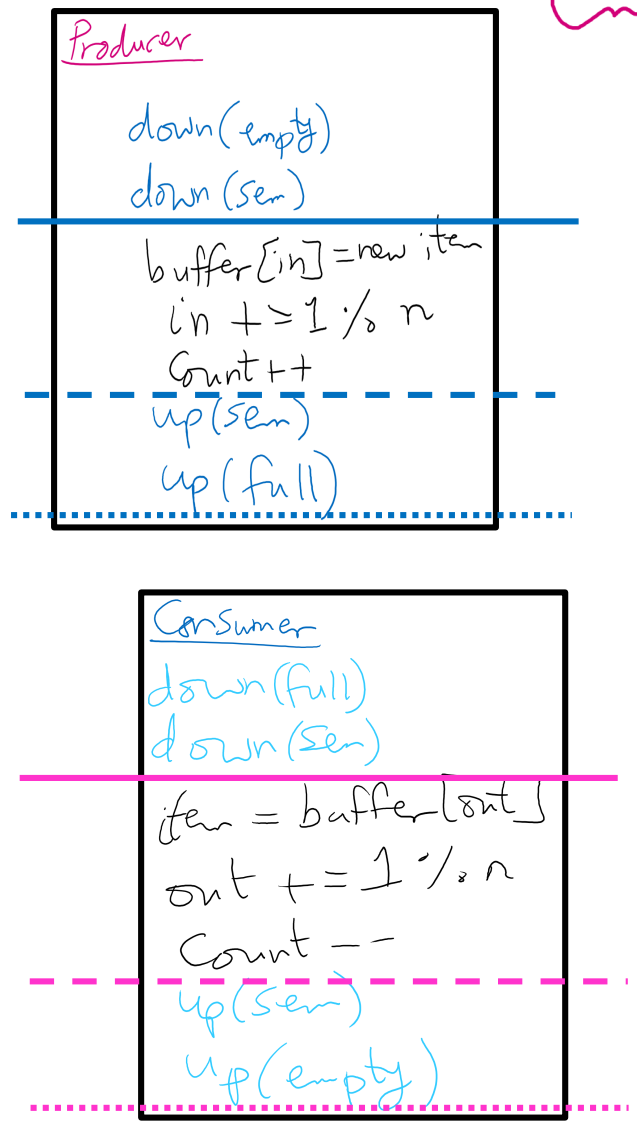
Initial state

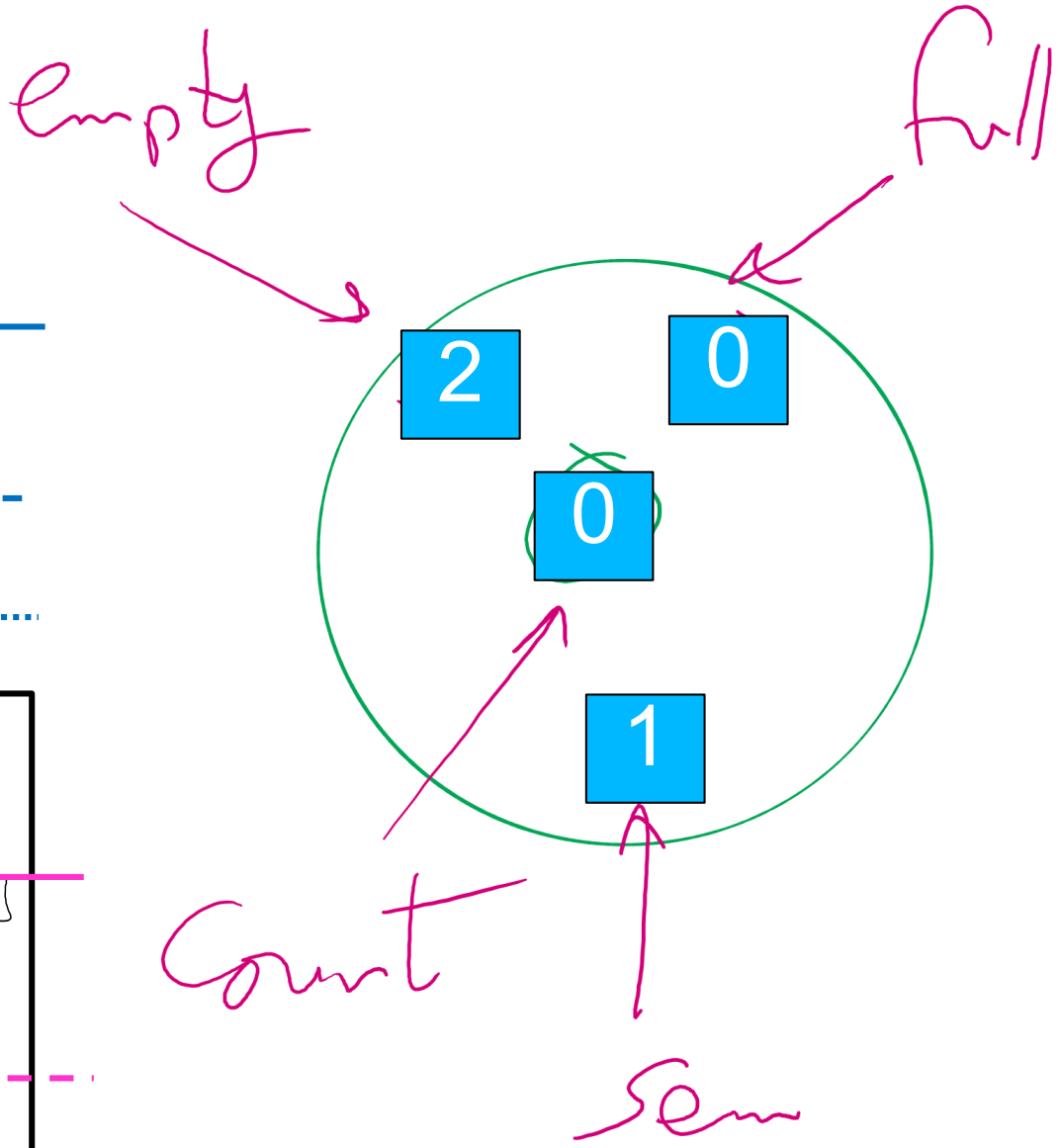


n == 2

C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

C0 arrives

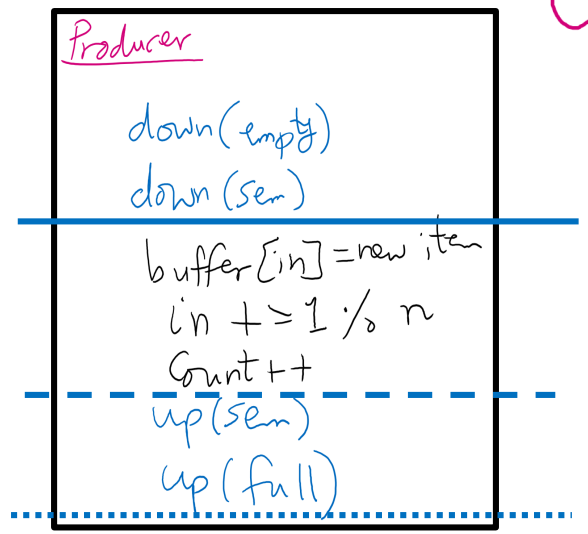


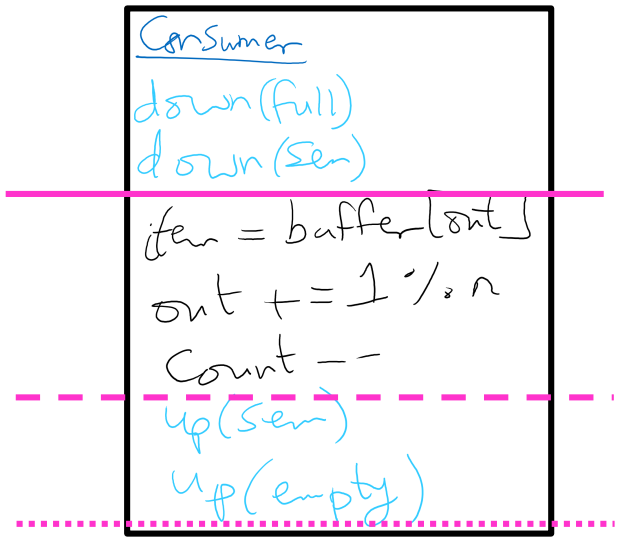


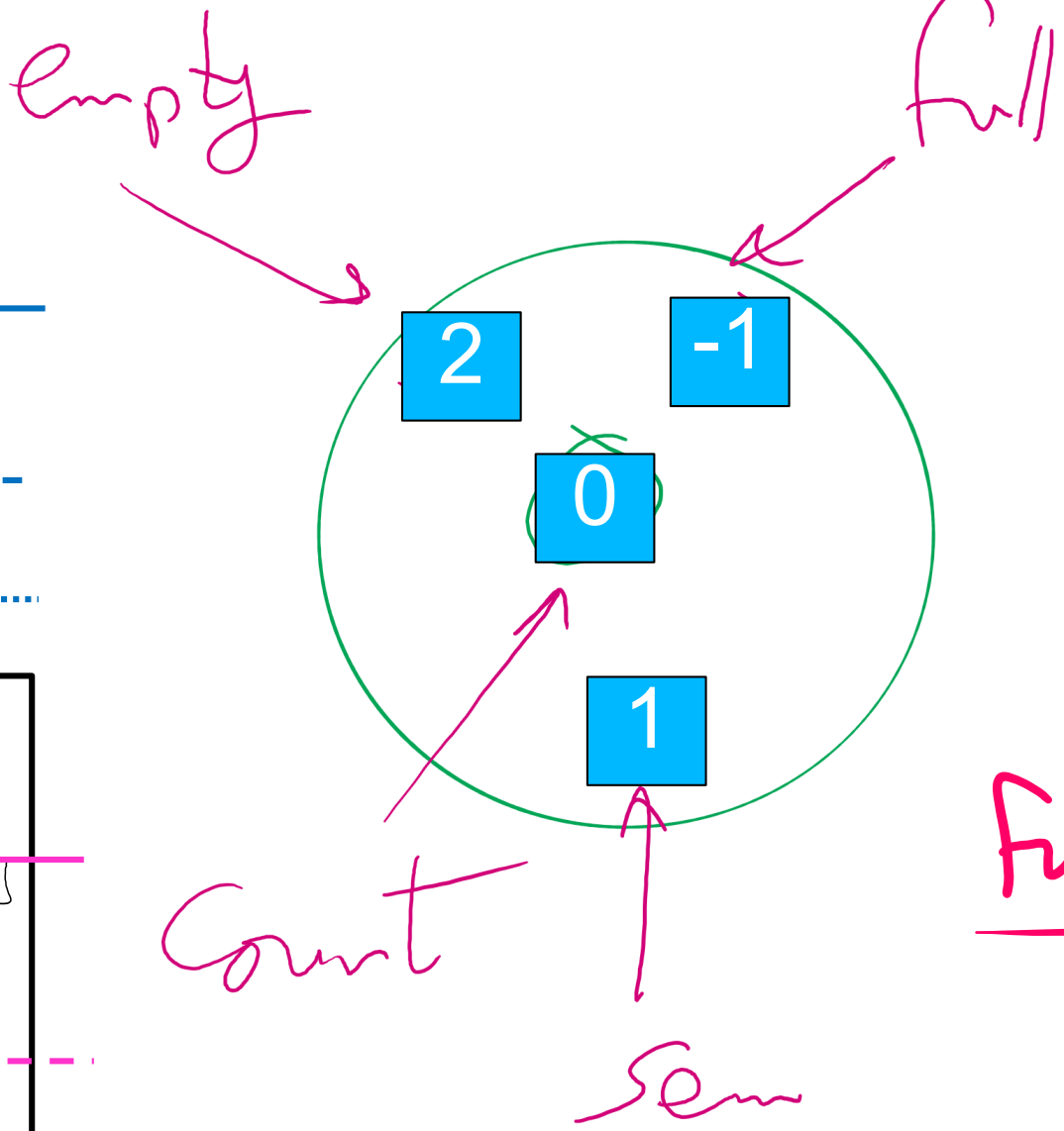
n == 2

C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters

C0 arrives



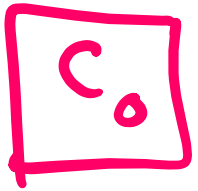




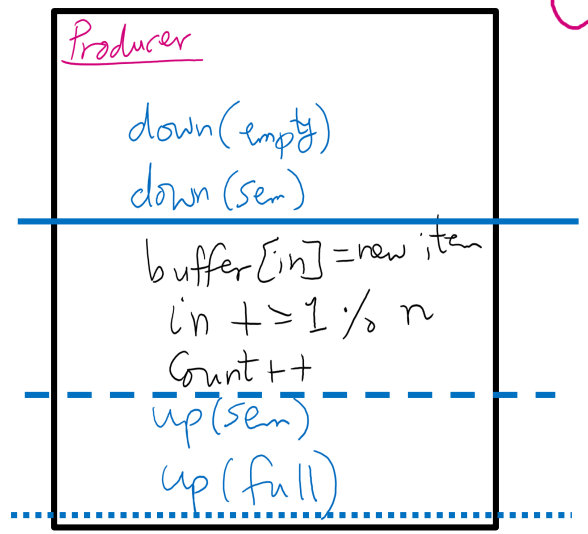
n == 2

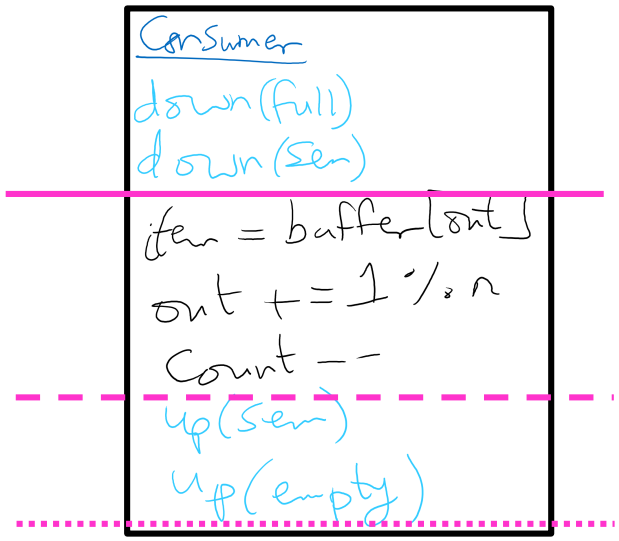
C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters

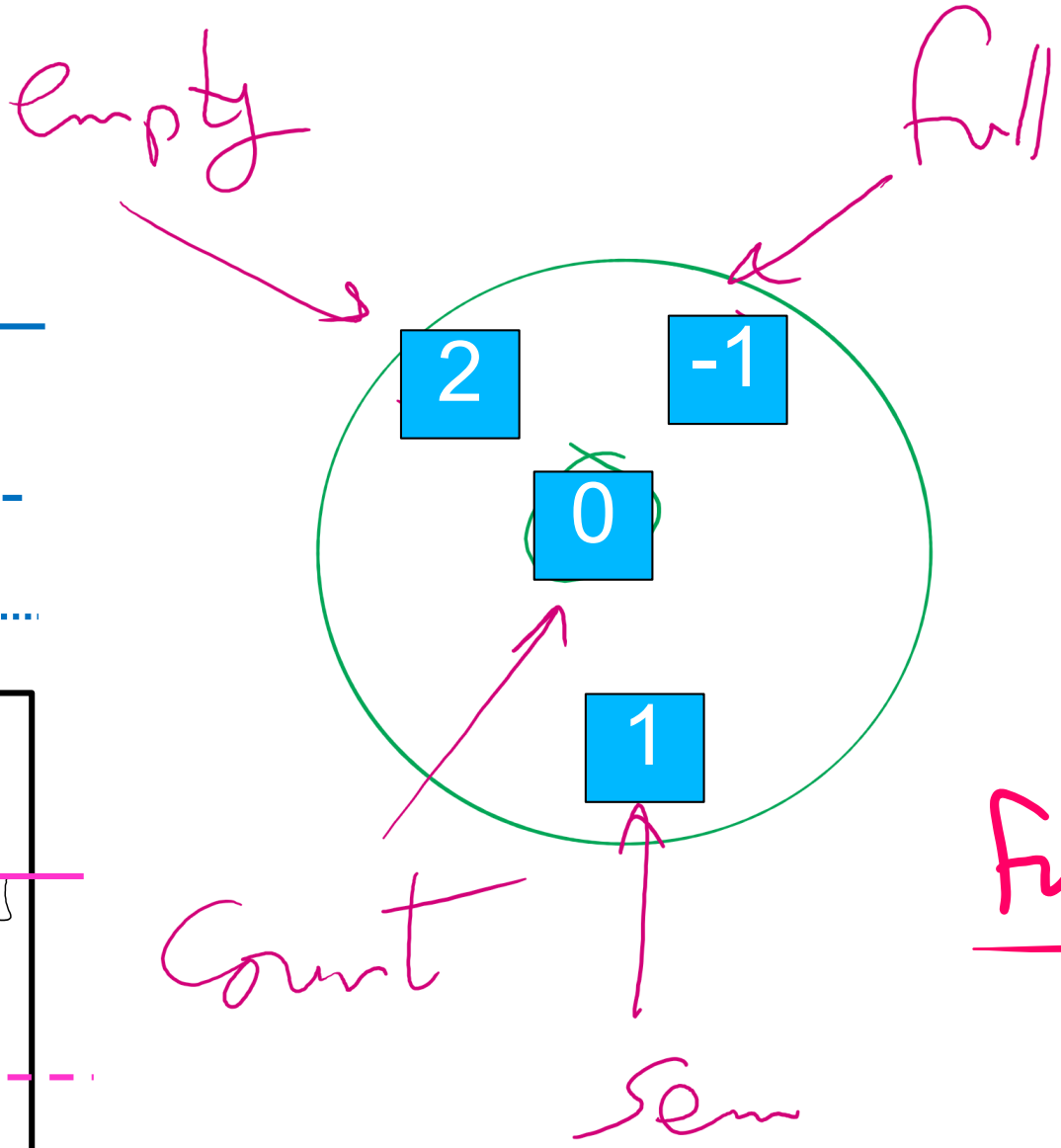
Full Queue



C1 arrives



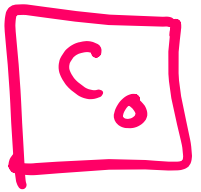




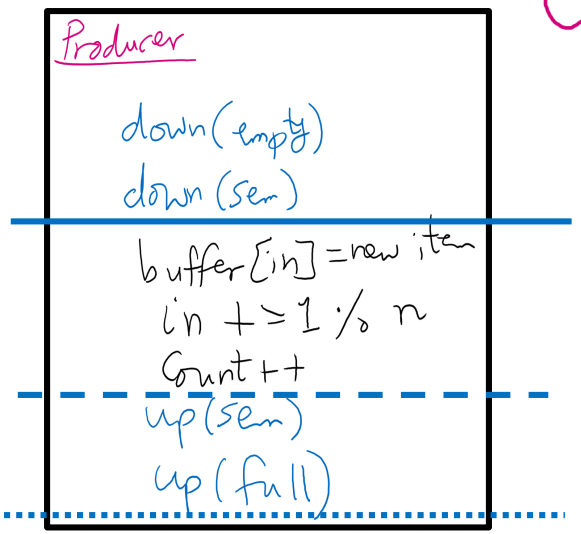
n == 2

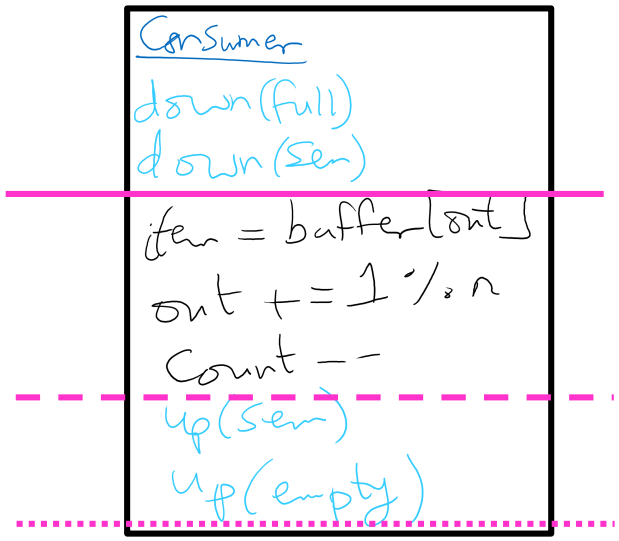
C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

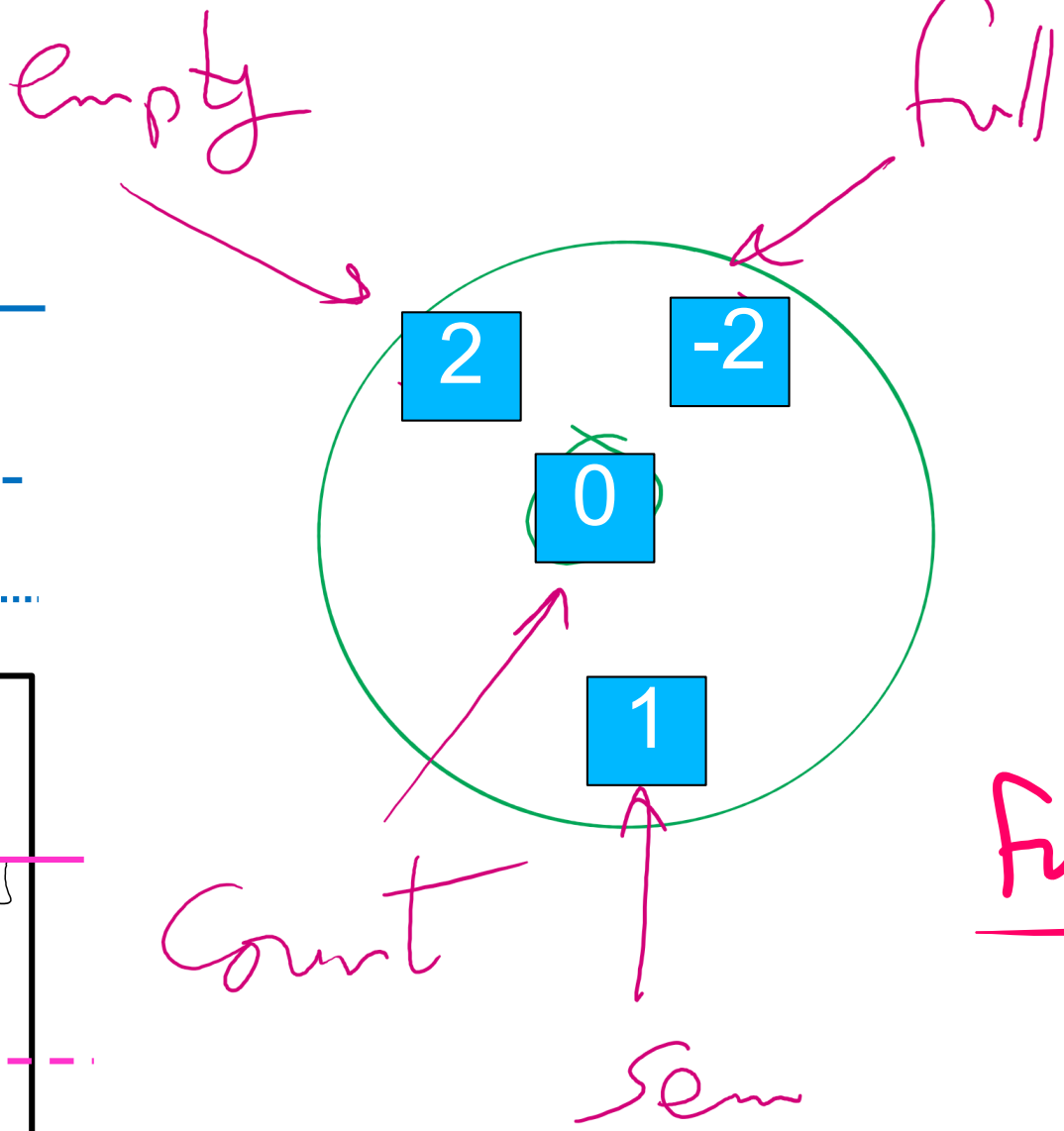
Full Queue



C1 arrives



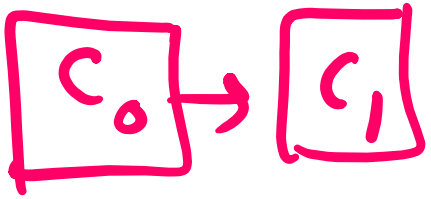




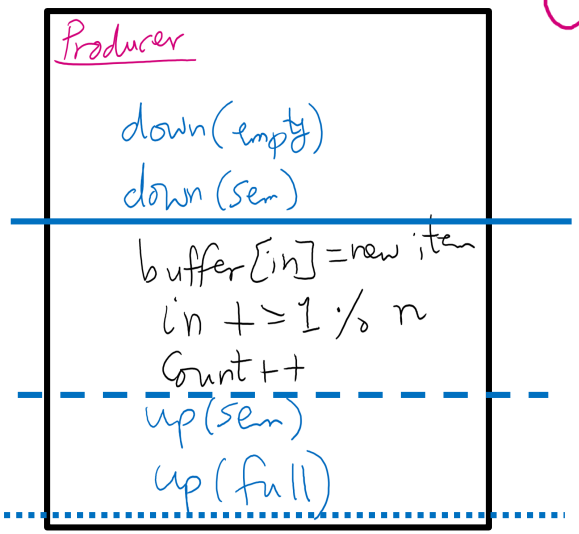
n == 2

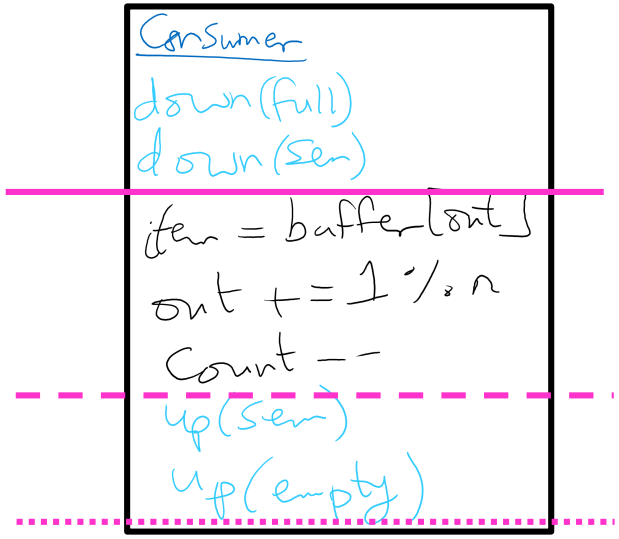
C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

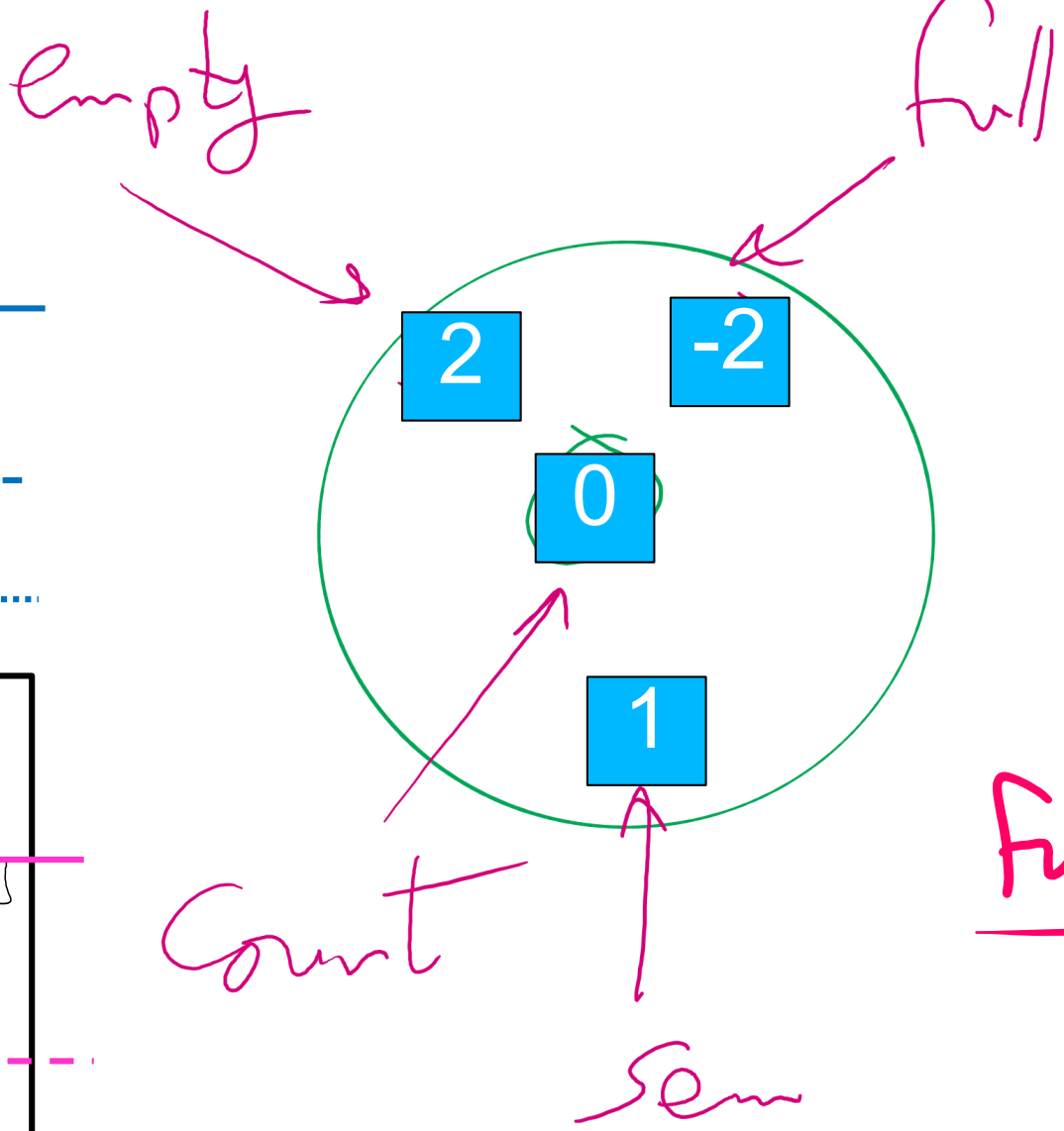
Full Queue



C2 arrives



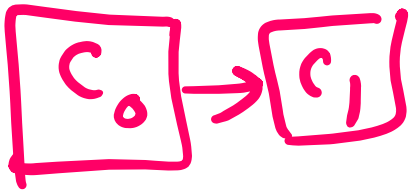




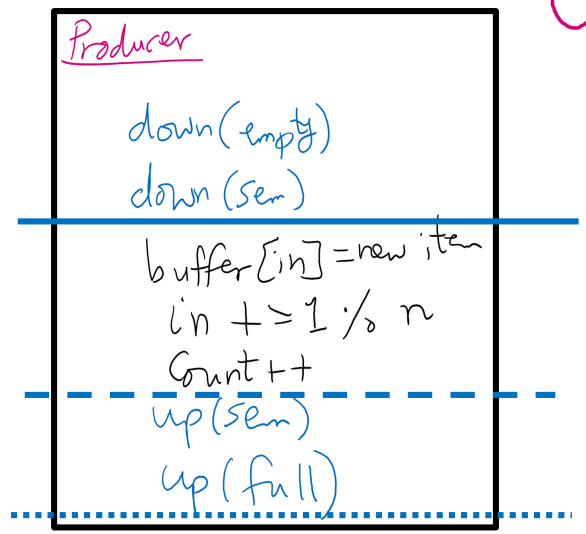
n == 2

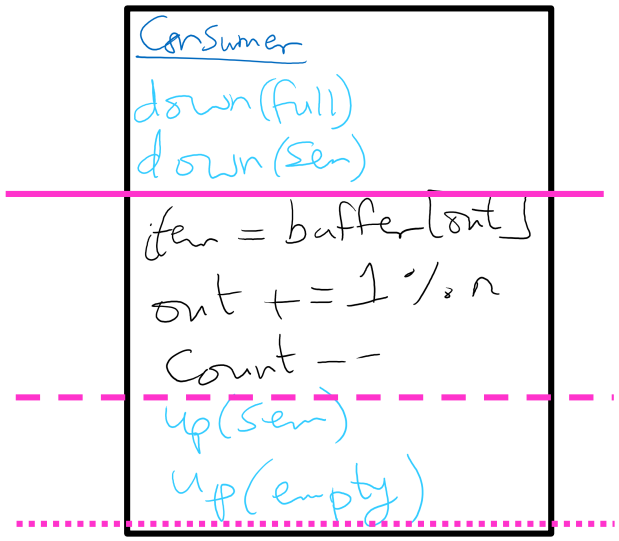
C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

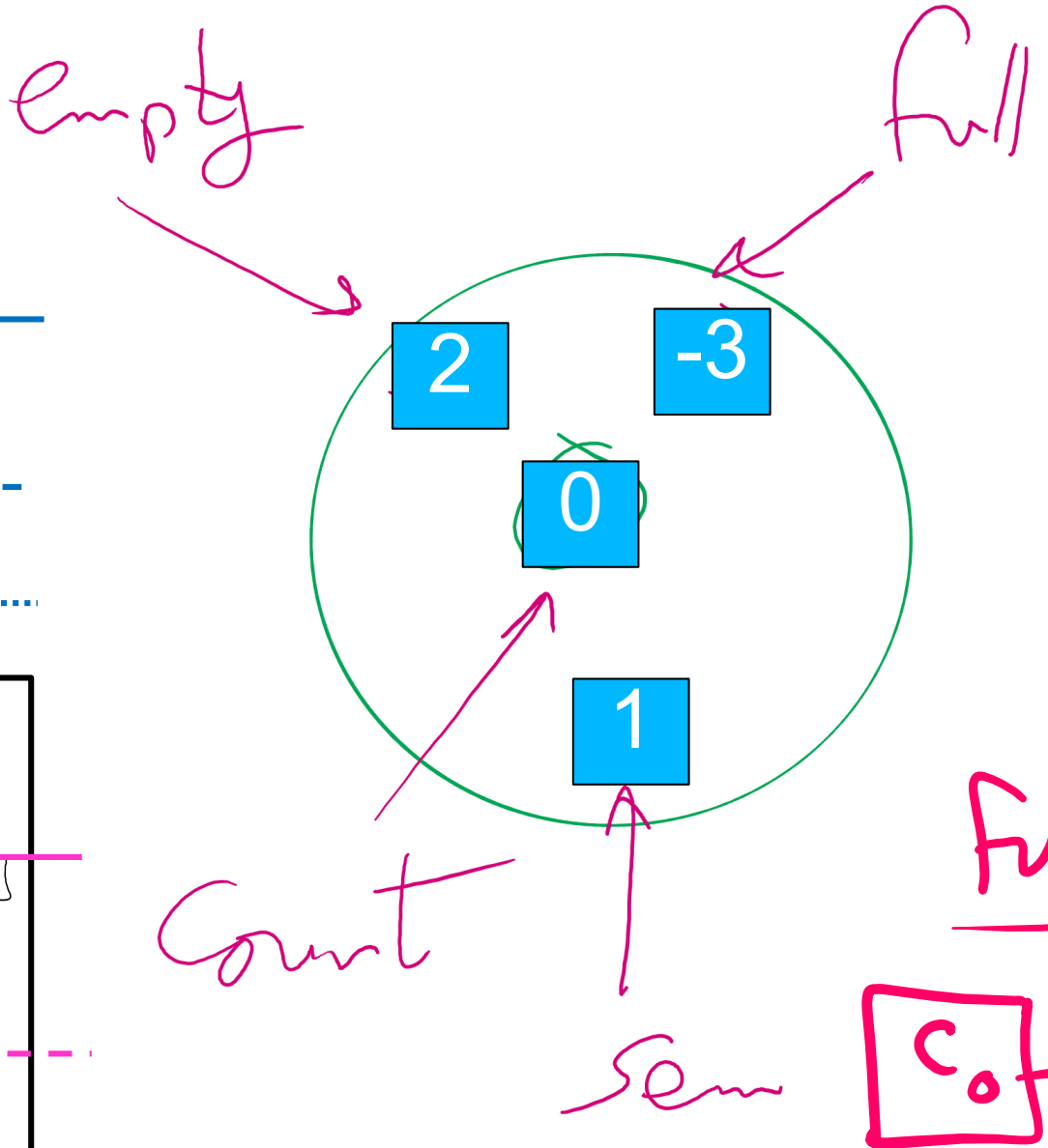
Full Queue



C2 arrives



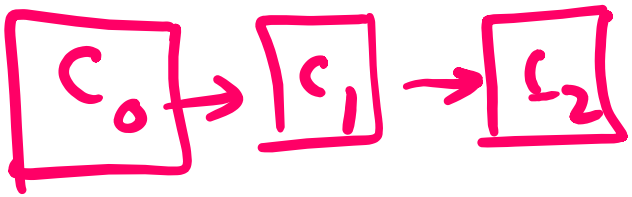




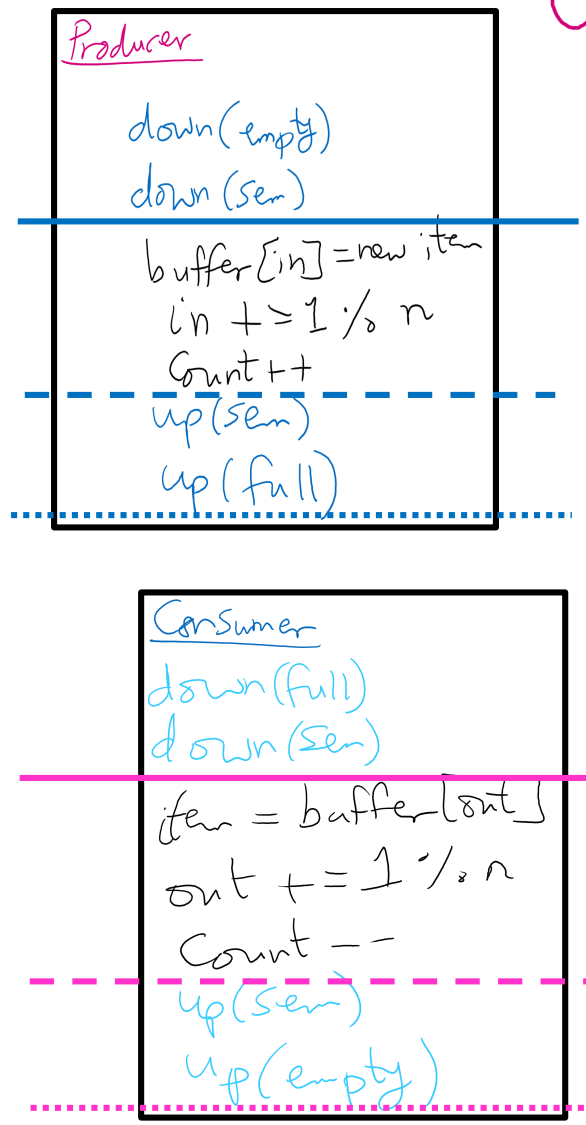
n == 2

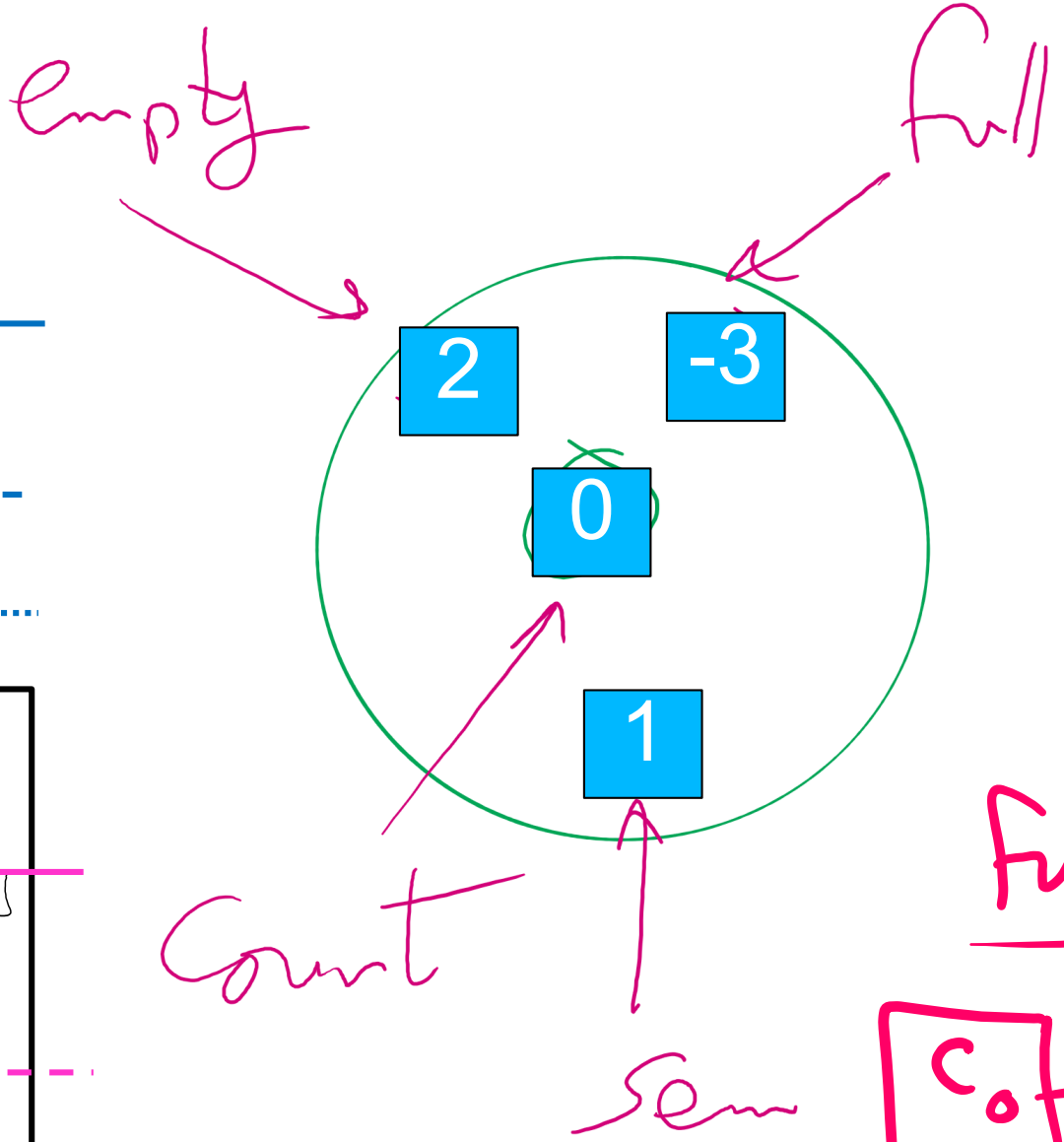
C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

Full Queue



P0 arrives

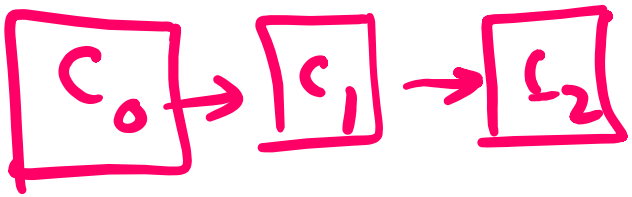




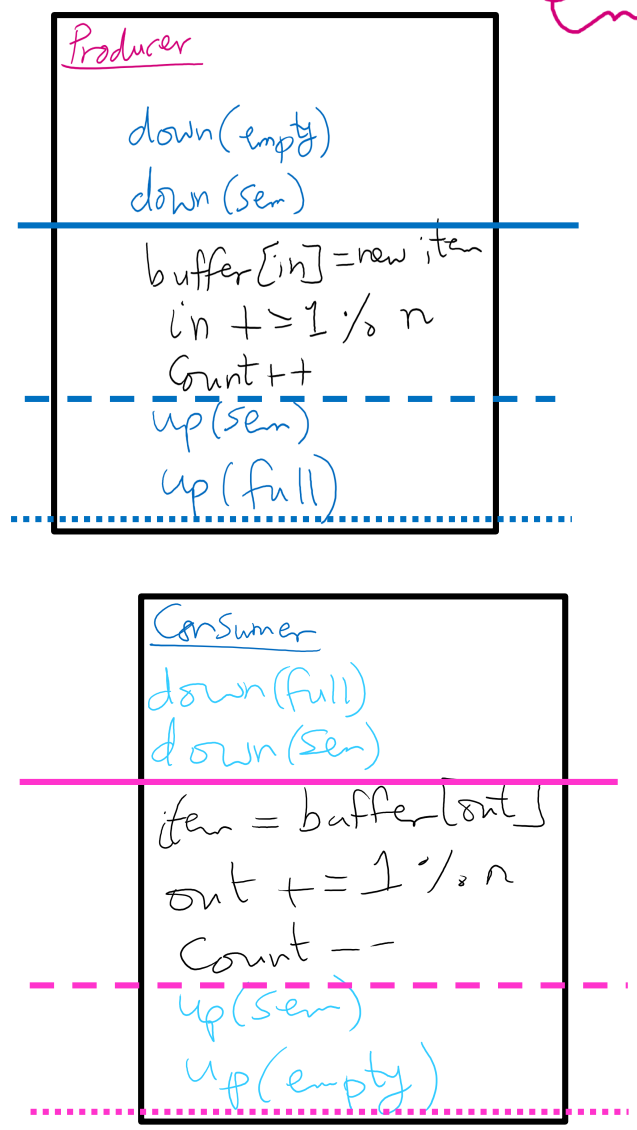
n == 2

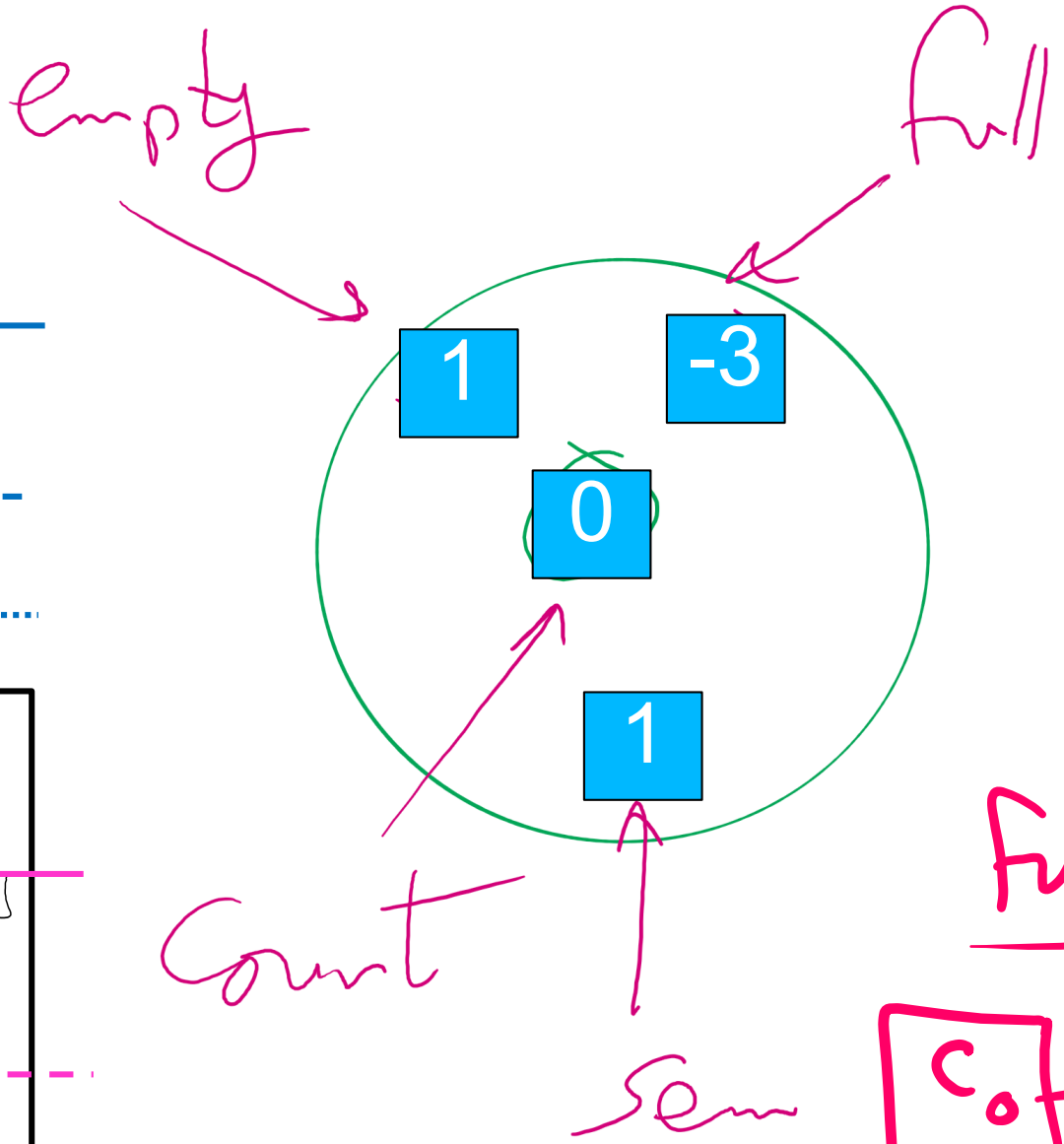
C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

Full Queue



P0 arrives

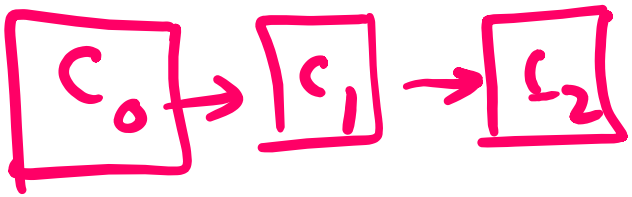




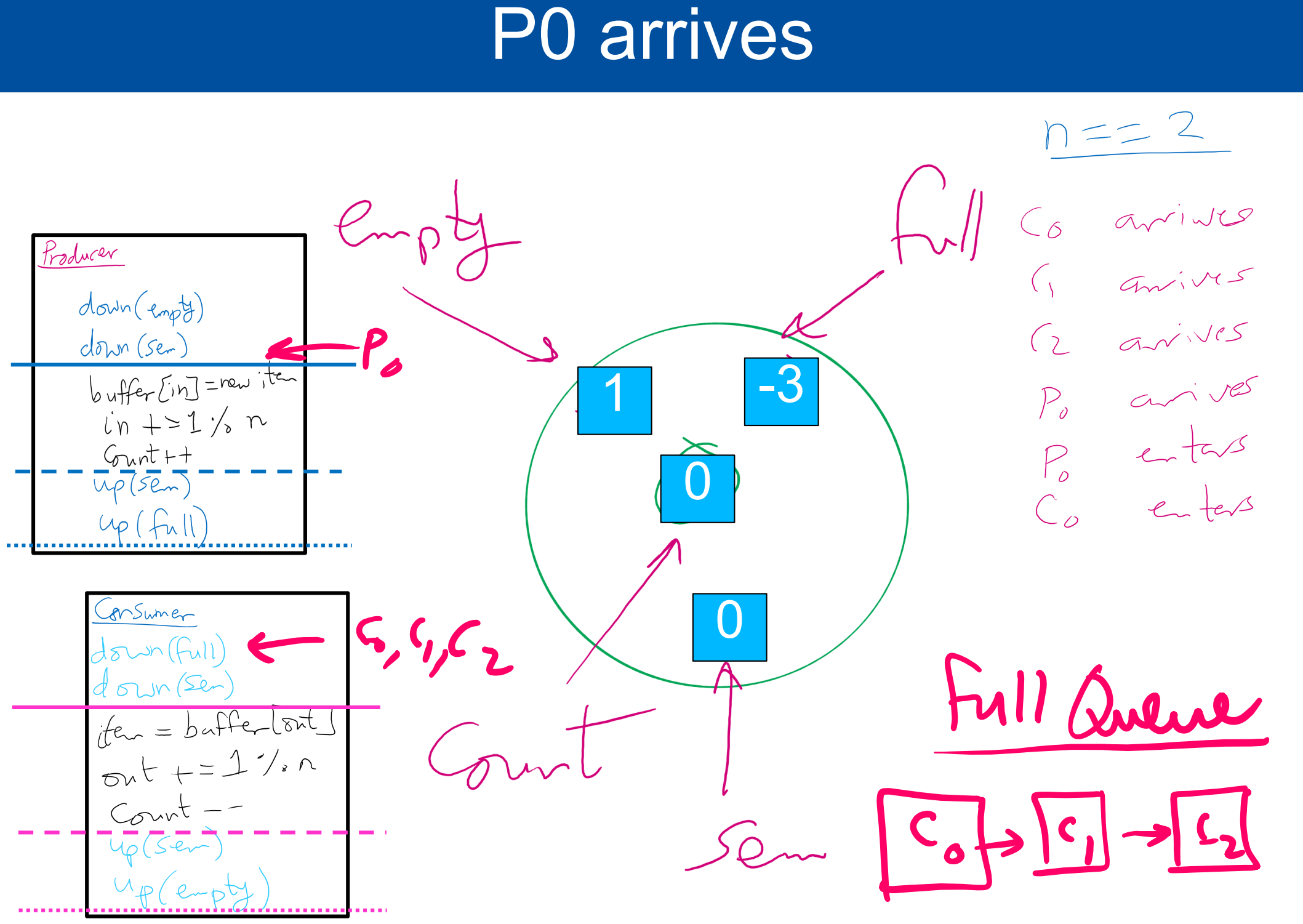
n == 2

C_0 arrives
C_1 arrives
C_2 arrives
P_0 arrives
P_0 enters
C_0 enters

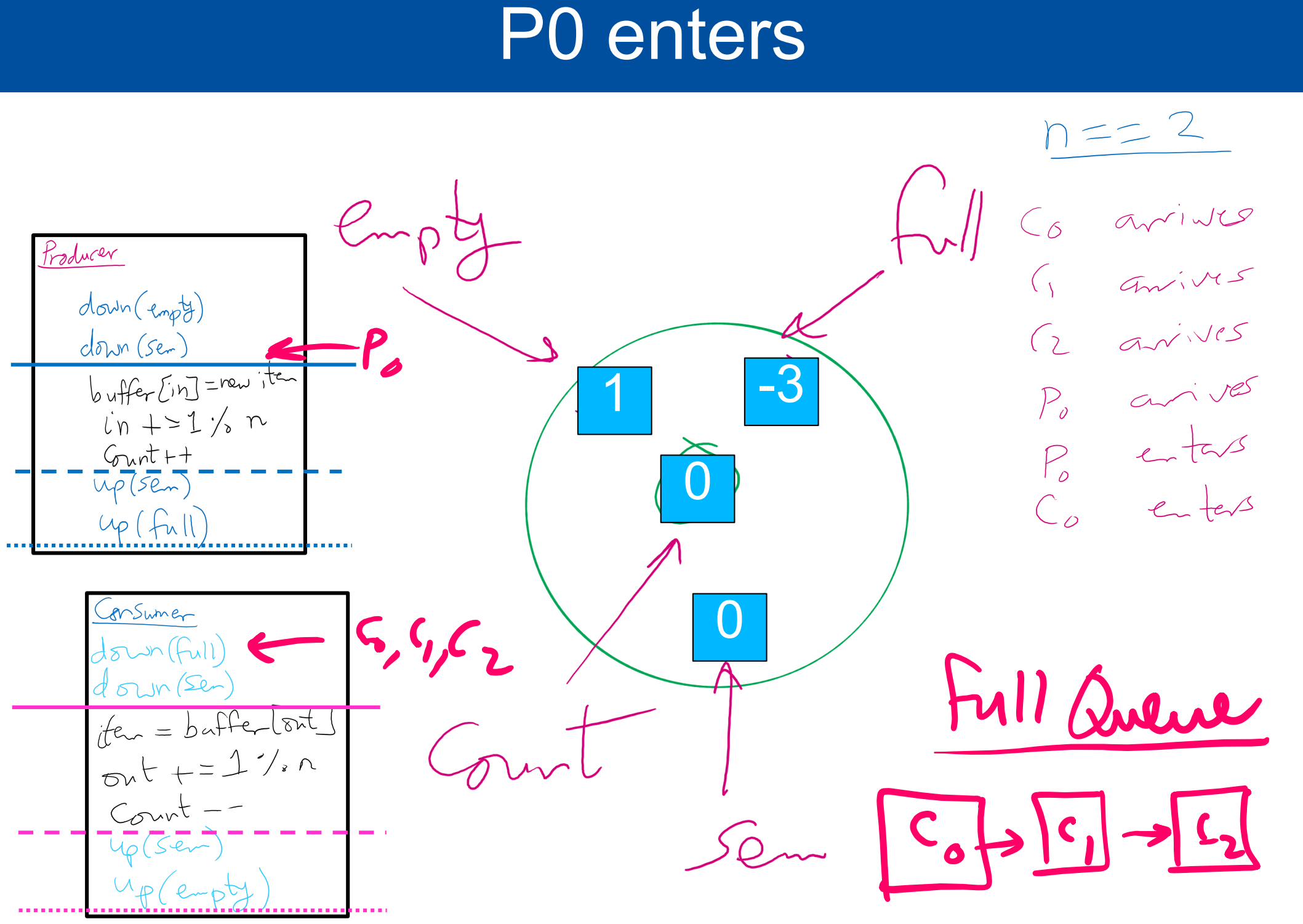
Full Queue



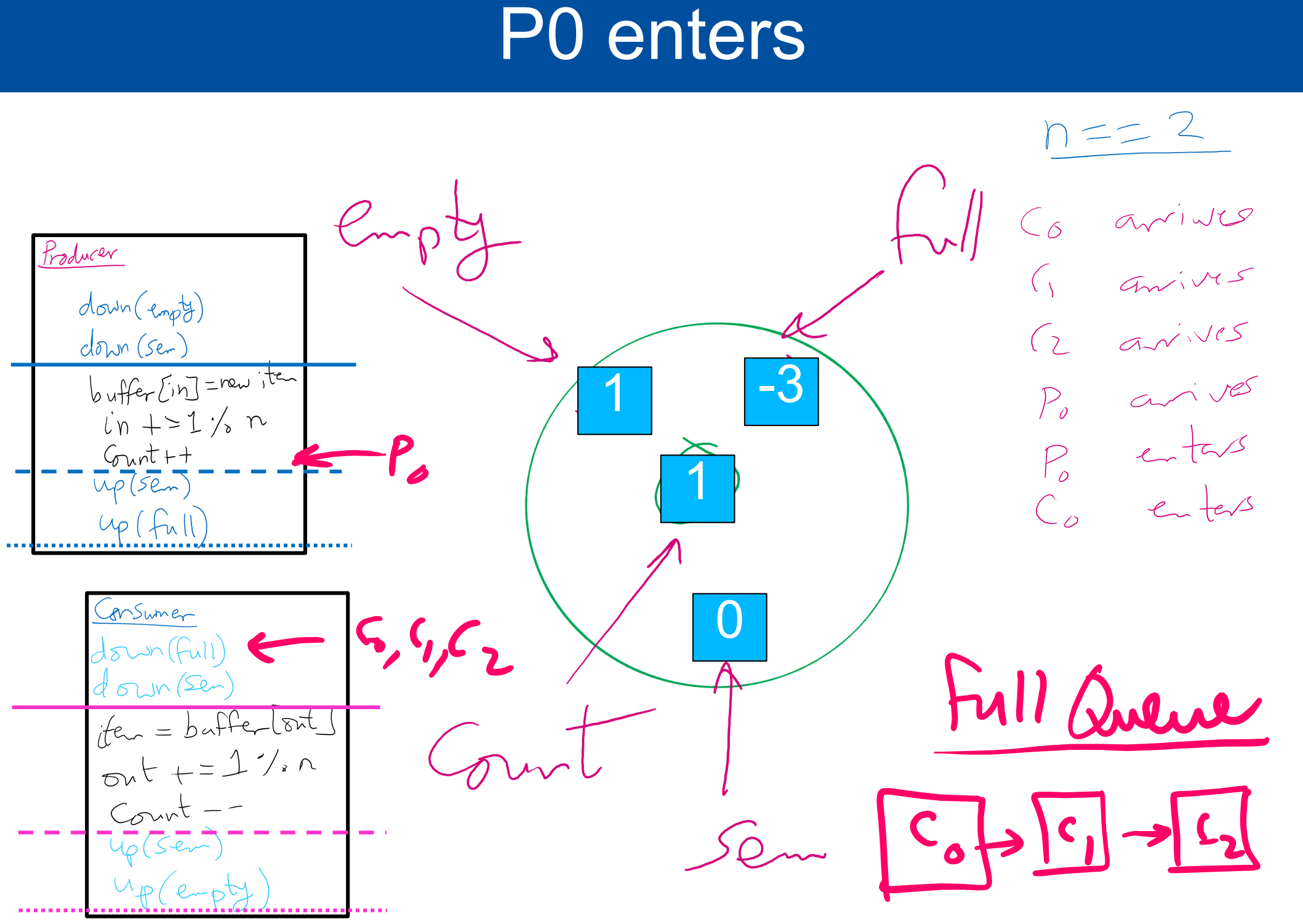
P0 arrives


n == 2
C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
1
-3
0
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0, C1, C2
count
sem
Full Queue
C0
C1
C2

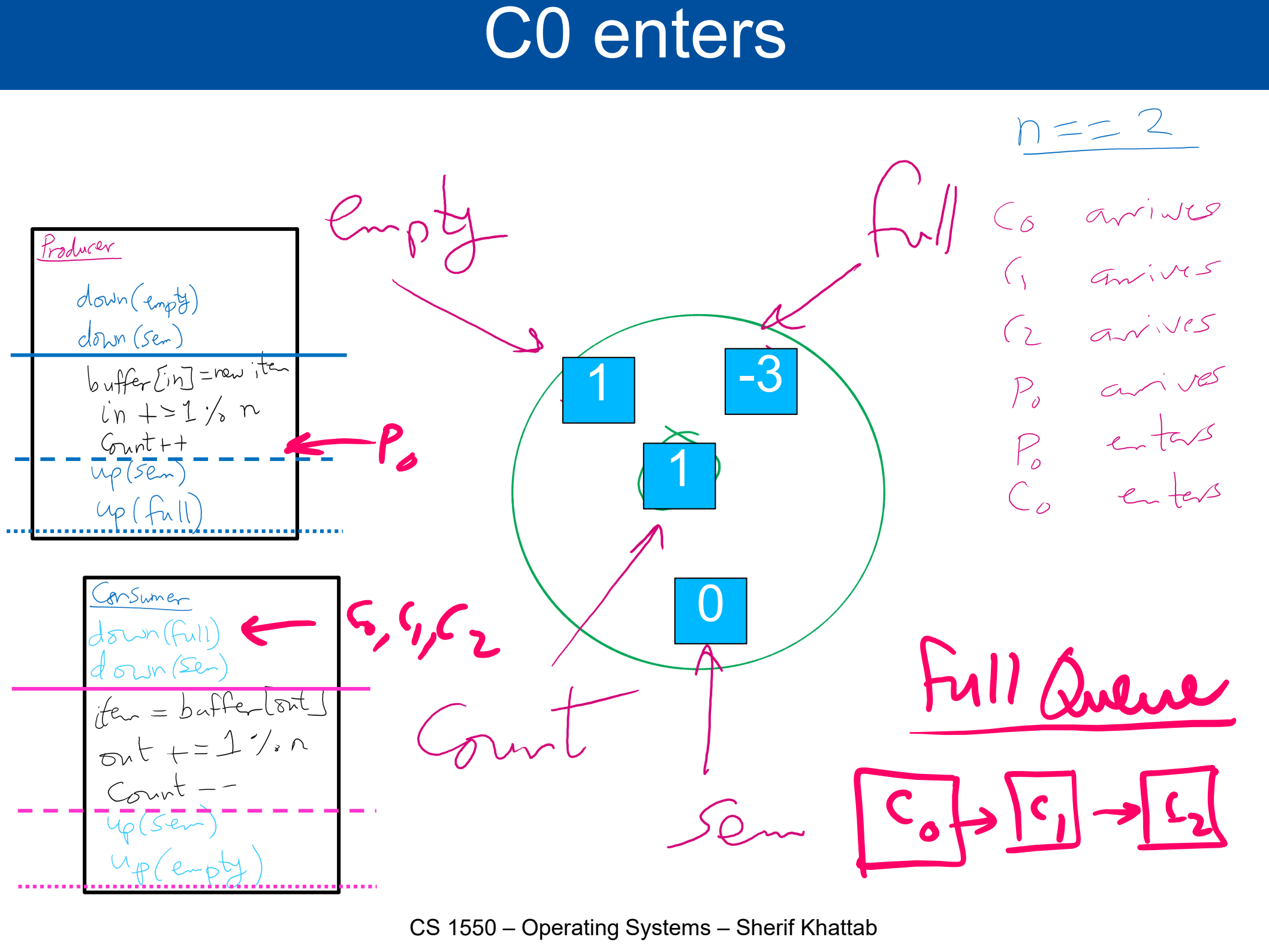
P0 enters


n == 2
Co arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
Co enters
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
1
-3
0
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0, C1, C2
count
sem
Full Queue
C0
C1
C2

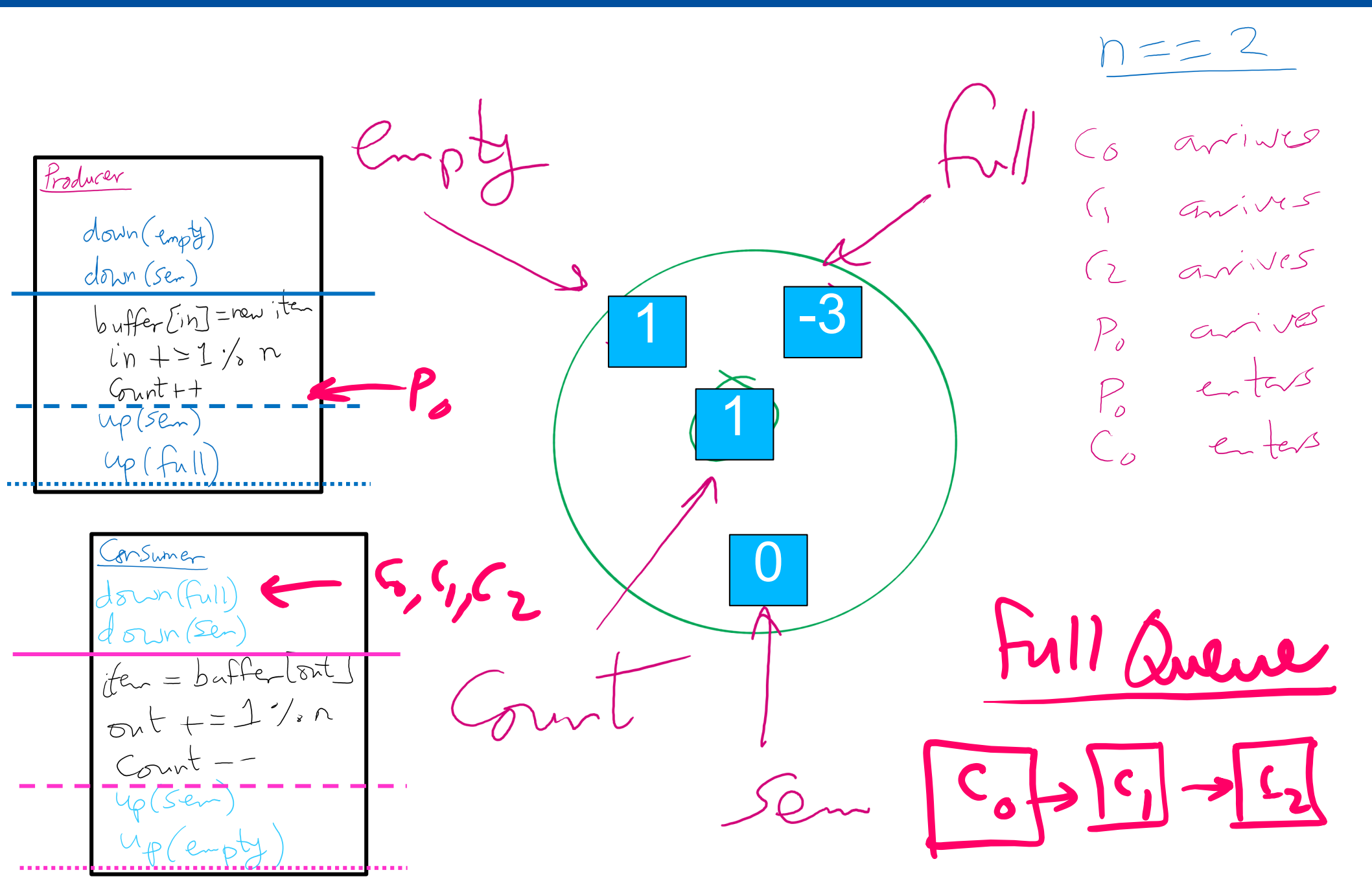
P0 enters


n == 2
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
empty
full
1
-3
1
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0, C1, C2
count
sem
C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters
Full Queue
C0
C1
C2

C0 enters


n == 2
C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
1
-3
1
0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0, C1, C2
count
sem
Full Queue
C0
C1
C2

Can C0 enter?


n == 2
C0 arrives
C1 arrives
C2 arrives
P0 arrives
P0 enters
C0 enters
empty
full
Producer
down(empty)
down(sem)
buffer[in] = new item
in += 1 % n
count++
up(sem)
up(full)
P0
Consumer
down(full)
down(sem)
item = buffer[out]
out += 1 % n
count--
up(sem)
up(empty)
C0, C1, C2
1
-3
1
0
count
sem
Full Queue
C0
C1
C2